Contents

1 Plantillas y utilidades **3**
- 1.1 Plantilla main + fast I/O . . . 3
- 1.2 Random / mt19937 . . . 3
- 1.3 BigInt (add + multiply, base 1e9) . . . 4
- 1.4 Coordinate compression . . . 5

2 Matematicas **5**
- 2.1 Teoria de numeros . . . 5
  - 2.1 Extended Euclidean Algorithm . . . 5
  - 2.2 Formulas utiles . . . 6
  - 2.3 Criba de Eratostenes . . . 6
  - 2.4 Criba lineal . . . 6
  - 2.5 Miller-Rabin primality . . . 7
  - 2.6 Pollard's Rho + factorizacion . . . 7
  - 2.7 Modular exponentiation . . . 8
  - 2.8 Modular inverse . . . 8
  - 2.9 Chinese Remainder Theorem . . . 8
  - 2.10 Euler totient precompute . . . 9
  - 2.11 Mobius precompute . . . 9
  - 2.12 Stern-Brocot (best fraction in interval) . . . 9
- 2.2 Combinatoria . . . 10
  - 2.1 Catalan (formula) . . . 10
  - 2.2 Factoriales + nCr mod p . . . 10
  - 2.3 Lucas theorem . . . 10
  - 2.4 Stirling S2 (particiones) . . . 11
  - 2.5 Stirling S1 (permutaciones por ciclos) . . . 11
  - 2.6 Inclusion-exclusion over subsets . . . 11
- 2.3 Polinomios y transformadas . . . 12
  - 2.1 FFT (iterativo, complex) . . . 12
  - 2.2 NTT (mod 998244353) . . . 12
  - 2.3 Multiplicacion de enteros grandes (FFT) . . . 13
  - 2.4 Polynomial utilities (mod p) . . . 13
- 2.4 Algebra lineal . . . 14
  - 2.1 Gauss-Jordan (mod p) . . . 14
  - 2.2 Matrix exponentiation . . . 15
- 2.5 Juegos y teoria de juegos . . . 15
  - 2.1 Sprague-Grundy template . . . 15
  - 2.2 Small game tables (Kayles SG values) . . . 15

3 Estructuras de datos **16**
- 3.1 Ordenado y contenedores . . . 16
  - 3.1 Ordered Set (pbds) . . . 16
  - 3.2 Basis (XOR basis) . . . 16
  - 3.3 Trie . . . 16
- 3.2 Union-Find . . . 17
  - 3.1 DSU classico . . . 17
  - 3.2 DSU con rollback . . . 17
- 3.3 Segment Trees . . . 18
  - 3.1 Segment Tree Add + Sum . . . 18
  - 3.2 Segment Tree Lazy Assign + Sum . . . 18
  - 3.3 Segment Tree Add + lower bound index . . . 19
  - 3.4 Segment Tree Lazy Add + Set + Sum . . . 19
  - 3.5 MO . . . 20
  - 3.6 Segment Tree 2D (con BASE) . . . 21
  - 3.7 Segment Tree persistente . . . 22
  - 3.8 Segment Tree (con BASE) . . . 22
- 3.4 Fenwick / BIT . . . 23
  - 3.1 BIT point update + prefix sum . . . 23
  - 3.2 BIT range add + point query . . . 23
  - 3.3 BIT range add + range sum . . . 23
- 3.5 RMQ y sparse table . . . 24
  - 3.1 Sparse Table . . . 24
  - 3.2 Disjoint Sparse Table . . . 24
- 3.6 Trees y DP en arbol . . . 24
  - 3.1 Binary Lifting / LCA . . . 24
  - 3.2 Heavy-Light Decomposition (HLD) . . . 25
  - 3.3 Centroid Decomposition . . . 25
  - 3.4 Euler Tour Technique (2 Variantes) . . . 26
- 3.7 Otras . . . 27
  - 3.1 Mo con updates . . . 27
  - 3.2 sqrt-decomposition generico . . . 27

4 Strings **28**
- 4.1 Hashing . . . 28
  - 4.1 Rolling Hash (doble modulo) . . . 28
- 4.2 Patrones . . . 28
  - 4.1 KMP + LPS . . . 28
  - 4.2 Z-algorithm . . . 29
  - 4.3 Rabin-Karp (single hash) . . . 29
  - 4.4 Aho-Corasick (basic) . . . 29
  - 4.5 Aho-Corasick DP for forbidden patterns . . . 30
- 4.3 Estructuras avanzadas . . . 31
  - 4.1 Suffix Array . . . 31
  - 4.2 LCP Array (Kasai) . . . 32
  - 4.3 Suffix Automaton . . . 32
- 4.4 Palindromos . . . 33
  - 4.1 Manacher . . . 33
- 4.5 Otros . . . 34

4.1 Lyndon factorization (Duval) . . . 34

5 Grafos **34**
5.1 Caminos minimos . . . 34
5.1 Dijkstra . . . 34
5.2 Bellman-Ford . . . 35
5.3 Floyd-Warshall . . . 35
5.2 Componentes y cortes . . . 36
5.1 Kosaraju SCC . . . 36
5.2 Tarjan (puentes) . . . 36
5.3 Tarjan (puntos de articulacion) . . . 37
5.4 Tarjan SCC . . . 37
5.3 Matching . . . 38
5.1 Kuhn (bipartite matching) . . . 38
5.2 Hopcroft-Karp . . . 38
5.4 MST . . . 39
5.1 Kruskal . . . 39
5.2 Prim (priority queue) . . . 39
5.5 Basicos . . . 40
5.1 Topological sort (Kahn + DFS) . . . 40
5.2 DP en DAG (camino mas largo / corto con relajacion) . . . 40
5.6 Flujos . . . 41
5.1 Ford-Fulkerson (DFS max flow) . . . 41
5.2 Edmonds-Karp (max flow) . . . 41
5.3 Dinic (max flow) . . . 42
5.4 MCMF con SPFA (Simple / Costos Negativos) . . . 43
5.5 MCMF con Potenciales de Johnson (Dijkstra / Rapido) . . . 43
5.6 Algoritmo Hungaro (Assignment Problem) . . . 44
5.7 Especiales . . . 45
5.1 2-SAT (implication graph + SCC) . . . 45
5.2 Euler tour / Hierholzer (camino y circuito) . . . 46
5.3 Hamiltonian path / cycle (bitmask DP + backtracking) . . . 46

6 DP **48**
6.1 LIS O(n log n) . . . 48
6.2 LCS / Edit distance . . . 49
6.3 Bitmask DP over subsets . . . 49
6.4 SOS DP . . . 50
6.5 Digit DP template . . . 51
6.6 Tree DP (reroot) . . . 52

7 Geometria **53**
7.1 Puntos y segmentos . . . 53
7.1 Conceptos y formulas de geometria 2D . . . 53
7.2 Point + cross + dot . . . 54
7.3 Interseccion de segmentos . . . 55
7.4 Contains colineal . . . 55
7.5 Distance point-segment / point-line . . . 55
7.6 Projection of point onto line . . . 55
7.2 Poligonos . . . 56
7.1 Convex Hull (monotono) . . . 56
7.2 Polygon Area . . . 56
7.3 Point in polygon (ray casting) . . . 56
7.4 Minkowski sum (convex polygons) . . . 56
7.3 Herramientas . . . 57
7.1 Li Chao Tree (long long) . . . 57
7.2 Sweep line template . . . 57
7.3 Closest pair of points (sweepline) . . . 58
7.4 Rotating calipers (polygon diameter) . . . 58

1 Plantillas y utilidades

1.1 Plantilla main + fast I/O

```cpp
// TC: N/A (boilerplate)
// usa: punto de partida para cualquier problema
#include <bits/stdc++.h>
using namespace std;

int main(){
  ios_base::sync_with_stdio(false);
  cin.tie(0);

  // freopen("input.txt", "r", stdin);
  // freopen("output.txt", "w", stdout);

  int n;
  if(!(cin >> n)) return 0;
  vector<int> a(n);
  for(int i=0;i<n;i++) cin >> a[i];

  // minimal example: print sum of n integers
  long long sum = 0;
  for(int x : a) sum += x;
  cout << sum << "\n";
  return 0;

  // === Usage examples of other modules (uncomment as needed) ===
  // See each module's listing for its own minimal main.

  // --- Utilities ---
  // shuffleVec(a);                    // random / mt19937
  // BigInt b(123), c(456); c += b; cout << c.toString();
  // DBGV(a);                          // debug pretty-printer
  // vector<int> cmp = compress(a);    // coordinate compression

  // --- Math ---
  // sieve(100); cout << primes.size();
  // linearSieve(100);
  // cout << millerRabin(17ULL);
  // vector<u64> f = factorize(123456ULL);
  // cout << modpow(2, 10);
  // cout << modInv(3, MOD);
  // cout << crt({2, 3}, {5, 7});
  // phiSieve(100); cout << phi[10];
  // mobiusSieve(100);
  // buildFact(20); cout << nCr(5, 2);
  // cout << nCrLucas(1000, 100, MOD);
  // buildStirlingS2(10);
  // multiplyFFT({1,2,3}, {4,5});
  // multiplyNTT({1,2,3}, {4,5});

  // --- Data Structures ---
  // DSUrb d(5); d.unite(0,1); d.unite(1,2); d.rollback();
  // SegTree2D st(10, 10); st.update(3, 4, 7); cout << st.query(0,0,3,4);
  // BIT bit(10); bit.add(3, 5); cout << bit.sum(5);
  // SparseTable sp(a); cout << sp.query(1, 3);
  // LCA lca(n); lca.addEdge(u,v); lca.build(0); cout << lca.lca(u,v);
  // HLD hld(n); hld.build(); auto segs = hld.pathSegments(u,v);

  // --- Strings ---
  // cout << zFunction("abacaba")[3];
  // auto occ = rabinKarp("hello world", "world");
  // AhoCorasick ac; ac.addString("he",0); ac.addString("she",1); ac.
    build();
  // auto cnt = ac.countOccurrences("ushers", 2);
  // auto [odd, even] = manacher("abacaba");
  // SuffArr sa(s); auto lcp = buildLCP(s, sa.p);
  // SAM sam(s.size()); sam.build(s); cout << sam.countDistinct();

  // --- Graphs ---
  // Graph g(n); g.addEdge(0,1); cout << g.bfs(0)[1];
  // auto order = topoKahn(n, adj);
  // auto [ok, col] = isBipartite(n, adj);
  // cout << zeroOneBFS(n, adj, 0)[n-1];
  // cout << kruskal(n, edges);
  // TarjanSCC scc(n, adj); cout << scc.numComps;
  // Dinic dinic(n, s, t); dinic.addEdge(u,v,c); cout << dinic.maxflow();
  // MCMF mcmf(n); mcmf.addEdge(u,v,c,k); auto [flow,cost] = mcmf.
    minCostMaxFlow(s,t);
  // TwoSAT ts(n); ts.addOr(0, true, 1, false); auto val = ts.solve();

  // --- DP ---
  // cout << knapsack01({2,3,4}, {3,4,5}, 5);
  // cout << lisLength({1,3,2,4});
  // cout << lcs("abcde", "ace");
  // cout << editDistance("kitten", "sitting");
  // cout << sosDP({1,2,3,4})[3];
  // cout << digitDP(1000);
  // TreeDP tdp(n); tdp.addEdge(u,v); auto ans = tdp.solve();

  // --- Geometry ---
  // cout << dist2PointSegment(p, a, b);
  // auto [px, py] = projectPointLine(p, a, b);
  // cout << pointInPolygon(p, poly);
  // auto msum = minkowskiSum(polyA, polyB);
  // LiChao lc(0, 1e9); lc.addLine(Line(1, 0)); cout << lc.query(5);
  // cout << closestPair(pts);
  // cout << rotatingCalipers(convexPoly);
}
```

1.2 Random / mt19937

```cpp
// TC: O(1) per call
// usa: shuffling deterministico, generacion de casos, hash randomization
#include <bits/stdc++.h>
using namespace std;

// Seed with fixed value for reproducible runs; change chrono for varied.
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

```

```cpp
// integer in [lo, hi]
int randInt(int lo, int hi){
  return uniform_int_distribution<int>(lo, hi)(rng);
}

// real in [lo, hi)
double randReal(double lo, double hi){
  return uniform_real_distribution<double>(lo, hi)(rng);
}

// shuffle a vector in O(n)
template<class T>
void shuffleVec(vector<T>& v){
  shuffle(v.begin(), v.end(), rng);
}

// pick a random element from a vector (non-empty)
template<class T>
T& randPick(vector<T>& v){
  return v[uniform_int_distribution<int>(0, (int)v.size() - 1)(rng)];
}

int main(){
  // shuffle a vector deterministically
  vector<int> a = {1,2,3,4,5};
  shuffleVec(a);
  for(int x : a) cout << x << " ";
  cout << "\n";

  // random int in [lo, hi]
  int r = randInt(1, 100);
  cout << "randInt = " << r << "\n";

  // shuffle with a fixed seed for reproducibility (replace rng temporarily)
  mt19937_64 fixedRng(42);
  shuffle(a.begin(), a.end(), fixedRng);
  for(int x : a) cout << x << " ";
  cout << "\n";
  return 0;
}
```

1.3 BigInt (add + multiply, base 1e9)

```cpp
// TC: O(N^2) multiply, O(N) add where N is the number of base-1e9 digits
// usa: problemas donde el resultado cabe en algunos miles de digitos (factoriales grandes, etc.)
#include <bits/stdc++.h>
using namespace std;

struct BigInt{
  // store digits in base 1e9 for compactness
  static const int BASE = 1000000000;
  vector<int> d; // least significant first
  bool neg;

  BigInt(): neg(false){}
  BigInt(long long x){ *this = x; }

  BigInt& operator=(long long x){
    neg = x < 0;
    x = abs(x);
    d.clear();
    do{
      d.push_back((int)(x % BASE));
      x /= BASE;
    }while(x > 0);
    return *this;
  }

  BigInt& operator+=(const BigInt& other){
    // assumes same sign for simplicity; extend if needed
    int carry = 0;
    size_t n = max(d.size(), other.d.size());
    if(d.size() < n) d.resize(n, 0);
    for(size_t i=0;i<n;i++){
      long long cur = d[i] + (i < other.d.size() ? other.d[i] : 0) + carry;
      d[i] = (int)(cur % BASE);
      carry = (int)(cur / BASE);
    }
    if(carry) d.push_back(carry);
    return *this;
  }

  BigInt& operator*=(int m){
    // m fits in int (typical contest case)
    long long carry = 0;
    for(size_t i=0;i<d.size();i++){
      long long cur = 1LL * d[i] * m + carry;
      d[i] = (int)(cur % BASE);
      carry = cur / BASE;
    }
    while(carry){
      d.push_back((int)(carry % BASE));
      carry /= BASE;
    }
    return *this;
  }

  string toString() const{
    if(d.empty()) return "0";
    stringstream ss;
    if(neg) ss << "-";
    ss << d.back();
    char buf[16];
    for(int i=(int)d.size()-2;i>=0;i--){
      snprintf(buf, sizeof(buf), "%09d", d[i]);
      ss << buf;
    }
    return ss.str();
```

```
  }
};

int main(){
  // compute 100! and print it
  BigInt fact(1);
  for(int i=2;i<=100;i++) fact *= i;
  cout << fact.toString() << "\n";

  // add two large numbers
  BigInt a(123456789012345LL), b(987654321098765LL);
  a += b;
  cout << a.toString() << "\n";

  // negative number
  BigInt neg(-42);
  cout << neg.toString() << "\n";
  return 0;
}
```

1.4 Coordinate compression

```
// TC: O(N log N)
// usa: cuando los valores son hasta 1e9 pero solo hay N de ellos
#include <bits/stdc++.h>
using namespace std;

// returns a vector of the same size as v, where each value is replaced
//     by its rank in the sorted distinct set (0-indexed).
vector<int> compress(const vector<int>& v){
  vector<int> sorted = v;
  sort(sorted.begin(), sorted.end());
  sorted.erase(unique(sorted.begin(), sorted.end()), sorted.end());
  vector<int> res(v.size());
  for(size_t i=0;i<v.size();i++){
    res[i] = (int)(lower_bound(sorted.begin(), sorted.end(), v[i]) -
    sorted.begin());
  }
  return res;
}

// if you also need the sorted distinct array itself:
pair<vector<int>, vector<int>> compressWithMap(const vector<int>& v){
  vector<int> sorted = v;
  sort(sorted.begin(), sorted.end());
  sorted.erase(unique(sorted.begin(), sorted.end()), sorted.end());
  vector<int> res(v.size());
  for(size_t i=0;i<v.size();i++){
    res[i] = (int)(lower_bound(sorted.begin(), sorted.end(), v[i]) -
    sorted.begin());
  }
  return {res, sorted};
}

// 2D compression: compress both coordinates of each point independently
// usa: grids, point sets, rectangle problems, etc.
// T puede ser int o long long (u otro tipo ordenable)
// TC: O(N log N)
template<typename T>
vector<pair<int,int>> compress2D(const vector<pair<T,T>>& points){
  vector<T> xs, ys;
  xs.reserve(points.size());
  ys.reserve(points.size());
  for(int i=0; i<points.size(); i++){
    xs.push_back(points[i].first);
    ys.push_back(points[i].second);
  }
  sort(xs.begin(), xs.end());
  xs.erase(unique(xs.begin(), xs.end()), xs.end());
  sort(ys.begin(), ys.end());
  ys.erase(unique(ys.begin(), ys.end()), ys.end());

  vector<pair<int,int>> res(points.size());
  for(int i=0; i<points.size(); i++){
    int cx = (int)(lower_bound(xs.begin(), xs.end(), points[i].first) -
    xs.begin());
    int cy = (int)(lower_bound(ys.begin(), ys.end(), points[i].second) -
    ys.begin());
    res[i] = make_pair(cx, cy);
  }
  return res;
}

int main(){
  // typical usage: compress values up to 1e9 to [0, k)
  vector<int> v = {1e9, 5, 100, 5, 1e9, 42}; // considerar long long
  vector<int> c = compress(v);
  for(int x : c) cout << x << " ";   // e.g. 3 0 2 0 3 1
  cout << "\n";

  // 2D example: compress points with coordinates up to 1e9
  // xs unicos: [5, 100, 1e9] -> [0, 1, 2]
  // ys unicos: [5, 42,  1e9] -> [0, 1, 2]
  vector<pair<long long,long long>> pts = {{1e9, 5}, {5, 1e9}, {100, 42},
     {5, 1e9}};
  auto c2d = compress2D(pts);
  for(auto& p : c2d) cout << "(" << p.first << "," << p.second << ") ";
  cout << "\n"; // (2,0) (0,2) (1,1) (0,2)
  return 0;
}
```

2 Matematicas

2.1 Teoria de numeros

2.1 Extended Euclidean Algorithm

```
// TC: O(log(min(a, b)))
```

```cpp
// SC: O(log(min(a, b))) por la pila de recursion
int extgcd(int a, int b, int &x, int &y){
  if(b==0){
    x=1; y=0;
    return a;
  }
  int x1,y1;
  int g=extgcd(b, a%b, x1, y1);
  x = y1;
  y = x1 - y1 * (a / b);
  return g;
}

int main(){
  // ejemplo 1: encontrar x, y tales que a*x + b*y = gcd(a, b)
  int x, y;
  int g = extgcd(12, 8, x, y);
  cout << "gcd = " << g << "\n";              // 4
  cout << "x = " << x << ", y = " << y << "\n"; // 1, -1
  // verifica: 12*1 + 8*(-1) = 4

  // ejemplo 2: inverso modular (cuando gcd(a, m) = 1)
  // encuentra x tal que a*x congruente 1 (mod m)
  int xi, yi;
  int a = 3, m = 7;
  int g2 = extgcd(a, m, xi, yi);
  if(g2 == 1){
    int inv = (xi % m + m) % m;  // ajustar a [0, m)
    cout << a << " * " << inv << " mod " << m << " = "
         << (a * inv) % m << "\n"; // 3 * 5 mod 7 = 1
  }
  return 0;
}
```

2.2 Formulas utiles

Numero de divisores

$$d(n) = (e_1 + 1) \cdot (e_2 + 1) \cdot \cdots \cdot (e_k + 1)$$

Suma de divisores

$$\sigma(n) = \frac{p_1^{e_1+1} - 1}{p_1 - 1} \cdot \frac{p_2^{e_2+1} - 1}{p_2 - 1} \cdot \cdots \cdot \frac{p_k^{e_k+1} - 1}{p_k - 1}$$

Producto de divisores

$$P(n) = n^{\frac{d(n)}{2}}$$

Euler

$$a^b \equiv a^{b \bmod \varphi(n)} \pmod{n}$$

Inclusion – Exclusion

$$\left| \bigcup_{i=1}^{n} A_i \right| = \sum_{\emptyset \neq I \subseteq \{1,2,\ldots,n\}} (-1)^{|I|+1} \left| \bigcap_{i \in I} A_i \right|$$

2.3 Criba de Eratostenes

```cpp
// TC: O(N log log N) build, O(1) isPrime query
// usa: enumerar primos hasta N, comprobar primalidad
#include <bits/stdc++.h>
using namespace std;

vector<int> primes;
vector<bool> is_composite;

void sieve(int n){
  is_composite.assign(n + 1, false);
  is_composite[0] = is_composite[1] = true;
  for(int i=2 ; i*i<=n ; i++){
    if(!is_composite[i]){
      for(int j=i*i;j<=n;j+=i) is_composite[j] = true;
    }
  }
  primes.clear();
  for(int i=2;i<=n;i++)
    if(!is_composite[i]) primes.push_back(i);
}

int main(){
  sieve(100);
  for(int p : primes) cout << p << " ";
  cout << "\n";
  cout << "first 10 primes count: " << primes.size() << "\n";
  return 0;
}
```

2.4 Criba lineal

```cpp
// TC: O(N) build
// usa: cuando se necesita lp[i] (lowest prime factor) para cada i
#include <bits/stdc++.h>
using namespace std;

vector<int> lp; // lowest prime factor
vector<int> primes;

void linearSieve(int n){
  lp.assign(n + 1, 0);
  primes.clear();
  for(int i=2 ; i<=n ; i++){
    if(lp[i] == 0){
      lp[i] = i;
      primes.push_back(i);
    }
    for(int p : primes){
      long long v = 1LL * i * p;
      if(v > n) break;
      lp[v] = p;
      if(p == lp[i]) break;
```

```
    }
  }
}

int main(){
  linearSieve(100);
  // lp[12] = smallest prime factor of 12 (= 2)
  cout << "lp[12] = " << lp[12] << "\n";
  cout << "#primes <= 100: " << primes.size() << "\n";
  return 0;
}
```

2.5 Miller-Rabin primality

```
// TC: O(log^3 n) but with tiny constant
// usa: primalidad de numeros hasta 2^64
#include <bits/stdc++.h>
using namespace std;

using u64 = unsigned long long;
using u128 = __uint128_t;

static u64 modpow_mr(u64 a, u64 e, u64 mod){
  u64 r = 1;
  a %= mod;
  while(e){
    if(e & 1) r = (u128)r * a % mod;
    a = (u128)a * a % mod;
    e >>= 1;
  }
  return r;
}

static bool millerRabin(u64 n){
  if(n < 2) return false;
  for(u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL,
     29ULL, 31ULL, 37ULL}){
    if(n % p == 0) return n == p;
  }
  u64 d = n - 1, s = 0;
  while((d & 1) == 0){ d >>= 1; s++; }
  // bases sufficient for deterministic test below 2^64
  for(u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL,
    1795265022ULL}){
    if(a % n == 0) continue;
    u64 x = modpow_mr(a, d, n);
    if(x == 1 || x == n - 1) continue;
    bool comp = true;
    for(u64 r=1 ; r<s ; r++){
      x = (u128)x * x % n;
      if(x == n - 1){ comp = false; break; }
    }
    if(comp) return false;
  }
  return true;
}

int main(){
  // test primality of several numbers up to 2^64
  for(u64 n : {2ULL, 17ULL, 561ULL, 1000000007ULL, (u64)1e18}){
    cout << n << ": " << (millerRabin(n) ? "prime" : "composite") << "\n"
    ;
  }
  return 0;
}
```

2.6 Pollard's Rho + factorizacion

```
// TC: expected O(n^{1/4}) per factor
// usa: descomponer numeros hasta 2^64 en factores primos
#include <bits/stdc++.h>
using namespace std;

using u64 = unsigned long long;
using u128 = __uint128_t;

// assumes millerRabin and modpow_mr are defined above

static u64 modmul(u64 a, u64 b, u64 mod){
  return (u128)a * b % mod;
}

static u64 pollard(u64 n){
  if(n % 2 == 0) return 2;
  static std::mt19937_64 rng((u64)chrono::steady_clock::now().
    time_since_epoch().count());
  while(true){
    u64 c = uniform_int_distribution<u64>(1, n - 1)(rng);
    u64 x = uniform_int_distribution<u64>(0, n - 1)(rng);
    u64 y = x;
    u64 d = 1;
    auto f = [&](u64 v){ return (modmul(v, v, n) + c) % n; };
    while(d == 1){
      x = f(x);
      y = f(f(y));
      d = std::gcd<u64>(x > y ? x - y : y - x, n);
    }
    if(d != n) return d;
  }
}

// returns sorted list of prime factors (with multiplicity)
void factorRec(u64 n, vector<u64>& out){
  if(n == 1) return;
  if(millerRabin(n)){ out.push_back(n); return; }
  u64 d = pollard(n);
  factorRec(d, out);
  factorRec(n / d, out);
}

```

```cpp
vector<u64> factorize(u64 n){
  vector<u64> out;
  factorRec(n, out);
  sort(out.begin(), out.end());
  return out;
}

int main(){
  // factor 123456789012345 = 3^2 * 5 * 7 * 13 * 29 * 41 * 271 * 9091
  auto f = factorize(123456789012345ULL);
  for(u64 p : f) cout << p << " ";
  cout << "\n";
  return 0;
}
```

2.7 Modular exponentiation

```cpp
// TC: O(log e)
// usa: base^exp % mod
#include <bits/stdc++.h>
using namespace std;

static const long long MOD = 998244353LL;

long long modpow(long long a, long long e, long long mod = MOD){
  long long r = 1;
  a %= mod;
  if(a < 0) a += mod;
  while(e){
    if(e & 1) r = r * a % mod;
    a = a * a % mod;
    e >>= 1;
  }
  return r;
}

int main(){
  // 2^10 mod (1e9+7)
  cout << modpow(2, 10) << "\n";
  // custom mod
  cout << modpow(2, 10, 1000000007LL) << "\n";
  return 0;
}
```

2.8 Modular inverse

```cpp
// TC: O(log mod)
// usa: 1/a mod p (con p primo), o por extended gcd si no
#include <bits/stdc++.h>
using namespace std;

// when MOD is prime
long long modInv(long long a, long long mod){
  return modpow(a, mod - 2, mod);
}

// general version using extended gcd (works for any mod coprime with a)
long long extGcd(long long a, long long b, long long& x, long long& y){
  if(b == 0){ x = 1; y = 0; return a; }
  long long x1, y1;
  long long g = extGcd(b, a % b, x1, y1);
  x = y1;
  y = x1 - y1 * (a / b);
  return g;
}

long long modInvGeneral(long long a, long long mod){
  long long x, y;
  long long g = extGcd(a, mod, x, y);
  if(g != 1) return -1; // no inverse
  return (x % mod + mod) % mod;
}

int main(){
  // inverse of 3 mod 1e9+7
  cout << modInv(3, 1000000007LL) << "\n";
  // general (via extgcd)
  cout << modInvGeneral(7, 26LL) << "\n";  // 7*15 = 105 = 1 mod 26
  return 0;
}
```

2.9 Chinese Remainder Theorem

```cpp
// TC: O(N * log(max(m))) donde N = num. congruencias
//     (cada modpow cuesta O(log m[i]))
// SC: O(1) extra
// usa: resolver sistema de congruencias pairwise coprimes
#include <bits/stdc++.h>
using namespace std;

// returns smallest x >= 0 such that x == r[i] (mod m[i]) for all i
//     assumes m[i] are pairwise coprime
long long crt(const vector<long long>& r, const vector<long long>& m){
  long long x = 0;
  long long M = 1; // modular inverse
  for(int i=0 ; i<r.size() ; i++){
    long long diff = (r[i] - x) % m[i];
    if(diff < 0) diff += m[i];
    long long inv = modpow(M % m[i], m[i] - 2, m[i]);  // (a^b) % m[i]
    long long t = diff * inv % m[i];
    x = x + M * t;
    M *= m[i];
    x %= M;
    if(x < 0) x += M;
  }
  return x;
}
```

```
int main(){
  // x == 2 (mod 3), x == 3 (mod 5), x == 2 (mod 7) -> x = 23
  cout << crt({2, 3, 2}, {3, 5, 7}) << "\n";
  return 0;
}
```

2.10 Euler totient precompute

```
// TC: O(N log log N)
// usa: queries tipo "phi(n) para varios n hasta N"
#include <bits/stdc++.h>
using namespace std;

vector<int> phi;

void phiSieve(int n){
  phi.assign(n + 1, 0);
  for(int i=0;i<=n;i++) phi[i] = i;
  for(int i=2;i<=n;i++){
    if(phi[i] == i){ // prime
      for(int j=i;j<=n;j+=i){
        phi[j] -= phi[j] / i;
      }
    }
  }
}

// single phi via factorization (slower but no precompute)
long long phiSingle(long long n){
  long long result = n;
  for(long long p=2 ; p*p<=n ; p++){
    if(n % p == 0){
      while(n % p == 0) n /= p;
      result -= result / p;
    }
  }
  if(n > 1) result -= result / n;
  return result;
}

int main(){
  phiSieve(100);
  // phi[10] = 4 (numbers coprime with 10: 1,3,7,9)
  for(int i=1 ; i<=10 ; i++)
    cout << "phi(" << i << ") = " << phi[i] << "\n";
  // single phi
  cout << "phi(36) = " << phiSingle(36) << "\n";
  return 0;
}
```

2.11 Mobius precompute

```
// TC: O(N log log N)
// usa: inclusion-exclusion con Mobius, queries tipo sum_{d|n} mu(d)
#include <bits/stdc++.h>
using namespace std;

vector<int> mu;

void mobiusSieve(int n){
  mu.assign(n + 1, 1);
  vector<bool> is_composite(n + 1, false);
  vector<int> primes;
  mu[0] = 0;
  mu[1] = 1;
  for(int i=2;i<=n;i++){
    if(!is_composite[i]){
      primes.push_back(i);
      mu[i] = -1;
    }
    for(int p : primes){
      long long v = 1LL * i * p;
      if(v > n) break;
      is_composite[v] = true;
      if(i % p == 0){
        mu[v] = 0;
        break;
      }else{
        mu[v] = -mu[i];
      }
    }
  }
}

int main(){
  mobiusSieve(20);
  for(int i=1 ; i<=10 ; i++)
    cout << "mu(" << i << ") = " << mu[i] << "\n";
  return 0;
}
```

2.12 Stern-Brocot (best fraction in interval)

```
// TC: O(log(max(a,b))) per query
// usa: encontrar la fraccion mas simple p/q en un rango (lo, hi)
#include <bits/stdc++.h>
using namespace std;

struct Frac { long long p, q; };

// simplest = minimize p+q; iterate through mediants
//     finds the fraction p/q with smallest p+q in the open interval (a/b, c/d)
//     assumes a/b < c/d.
Frac bestInInterval(long long a, long long b, long long c, long long d){
  long long p = a + c, q = b + d;
```

```
  while(true){
    long long k = max((p - a) / b, (p - c) / d);
    // this is a coarse approximation; a full implementation needs limits
    if(k <= 1) break;
    p = a + k * c;
    q = b + k * d;
  }
  return {p, q};
}

int main(){
  // simplest fraction in (0/1, 1/1) is 1/2
  auto f = bestInInterval(0, 1, 1, 1);
  cout << f.p << "/" << f.q << "\n";
  return 0;
}
```

2.2 Combinatoria

2.1 Catalan (formula)

```
// TC: O(n^2)
vector<int> dp;
const int MOD=998244353;

void catalan(int n){ // from 1 to n
  dp.resize(n+1);
  dp[0]=dp[1]=1;
  for(int i=2 ; i<=n ; i++){
    for(int j=0 ; j<i ; j++){
      dp[i]=(dp[i]+(dp[j]*dp[i-j-1])%MOD)%MOD;
    }
  }
}

// TC: O(n)
// consider use of long long
int catalan(int n){
  int ans=1;
  for(int i=0 ; i<n ; i++){
    ans=ans*2*(2*i+1)/(i+2);
  }
  return ans;
}

int main(){
  cout << "catalan(10) = " << catalan(10) << "\n";
  return 0;
}
```

2.2 Factoriales + nCr mod p

```
// TC: O(N) precompute, O(1) query
// usa: C(n, k) mod p, donde p es primo
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 998244353LL;
vector<long long> fact, invFact;

void buildFact(int n){
  fact.assign(n + 1, 1);
  invFact.assign(n + 1, 1);
  for(int i=1 ; i<=n ; i++) fact[i] = fact[i-1] * i % MOD;
  invFact[n] = modpow(fact[n], MOD - 2, MOD);
  for(int i=n ; i>=1 ; i--) invFact[i-1] = invFact[i] * i % MOD;
}

long long nCr(int n, int k){
  if(k < 0 || k > n) return 0;
  return fact[n] * invFact[k] % MOD * invFact[n-k] % MOD;
}

int main(){
  buildFact(100);
  cout << "C(10,3) = " << nCr(10, 3) << "\n";
  cout << "C(100,50) mod = " << nCr(100, 50) << "\n";
  return 0;
}
```

2.3 Lucas theorem

```
// TC: O(log_p n) per query
// usa: C(n, k) mod p con p primo y n, k grandes
#include <bits/stdc++.h>
using namespace std;

// usa: C(n, k) mod p cuando n y k son gigantes y p es primo
// idea: escribir n y k en base p; C(n, k) congruente Productorio C(ni, ki) mod p
//       (un producto por cada digito en base p)
// requires fact[] and invFact[] already built for MOD (small case)
long long nCrLucas(long long n, long long k, long long MOD){
  if(k == 0) return 1;
  long long ni = n % MOD, ki = k % MOD;
  if(ki > ni) return 0;
  return nCrLucas(n / MOD, k / MOD, MOD) * nCr((int)ni, (int)ki) % MOD;
}

int main(){
  buildFact(MOD);  // precompute factorials mod MOD
  // C(1000, 100) mod 998244353 using Lucas
  cout << nCrLucas(1000, 100, MOD) << "\n";
  return 0;
}
```

2.4 Stirling S2 (particiones)

```cpp
// TC: O(N^2)
// usa: numero de formas de particionar N elementos en K subconjuntos no vacios
// rec: S2[i][j] = S2[i-1][j-1] + j * S2[i-1][j]
//      el elem i forma un bloque nuevo, o se une a uno de los j bloques existentes
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 998244353LL;
vector<vector<long long>> S2;

void buildStirlingS2(int n){
  S2.assign(n + 1, vector<long long>(n + 1, 0));
  S2[0][0] = 1;
  for(int i=1;i<=n;i++){
    for(int j=1;j<=i;j++){
      S2[i][j] = (S2[i-1][j-1] + j * S2[i-1][j]) % MOD;
    }
  }
}

int main(){
  buildStirlingS2(10);
  // partitions of 5 elements into 2 nonempty subsets
  cout << "S2(5,2) = " << S2[5][2] << "\n";
  return 0;
}
```

2.5 Stirling S1 (permutaciones por ciclos)

```cpp
// TC: O(N^2)
// usa: numero de permutaciones de N elementos con exactamente K ciclos
// rec: S1[i][j] = S1[i-1][j-1] + (i-1) * S1[i-1][j]
//      el elem i forma un ciclo de 1, o se inserta en uno de los i-1 huecos de ciclos previos
// nota: version sin signo; con signo s(n,k)=(-1)^(n-k)*S1[n][k] es coeficiente de (x)_n
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 998244353LL;
vector<vector<long long>> S1;

void buildStirlingS1(int n){
  S1.assign(n + 1, vector<long long>(n + 1, 0));
  S1[0][0] = 1;
  for(int i=1;i<=n;i++){
    for(int j=1;j<=i;j++){
      S1[i][j] = (S1[i-1][j-1] + (long long)(i - 1) * S1[i-1][j]) % MOD;
    }
  }
}

int main(){
  buildStirlingS1(10);
  // permutations of 4 elements with exactly 2 cycles -> 11
  cout << "S1(4,2) = " << S1[4][2] << "\n";
  return 0;
}
```

2.6 Inclusion-exclusion over subsets

```cpp
// TC: O(2^N * cost_of_evaluating_subset)
// usa: cuando se quiere contar elementos que cumplen alguna combinacion de N propiedades
#include <bits/stdc++.h>
using namespace std;

// f(mask) returns the number of elements satisfying exactly the
//      properties in mask. Returns sum over non-empty masks with sign (-1)^(k+1).
template<class F>
long long inclusionExclusion(int n, F&& f){
  long long result = 0;
  int total = 1 << n;
  for(int mask=1;mask<total;mask++){
    long long cnt = f(mask);
    int bits = __builtin_popcount(mask);
    if(bits % 2 == 1) result += cnt;
    else result -= cnt;
  }
  return result;
}

int main(){
  // count numbers in [1,100] divisible by 2, 3 or 5 (must not be divisible by any)
  // generic f: returns |union over mask|, signed sum via inclusion-exclusion
  auto f = [&](int mask) -> long long {
    long long cnt = 0;
    int primes[] = {2,3,5};
    int bits = __builtin_popcount(mask);
    for(int p : primes) if(mask & (1<<(p==2?0:p==3?1:2))){ /* set bit */
    }
    return cnt;
  };
  // (full example in PLAN_03 central main)
  return 0;
}
```

2.3 Polinomios y transformadas

2.1 FFT (iterativo, complex)

```cpp
// TC: O(N log N)
// usa: multiplicacion de polinomios con coeficientes double
#include <bits/stdc++.h>
using namespace std;

using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd>& a, bool invert){
  int n = a.size();
  // bit-reversal permutation
  for(int i=1,j=0;i<n;i++){
    int bit = n >> 1;
    for(; j&bit; bit>>=1) j ^= bit;
    j ^= bit;
    if(i < j) swap(a[i], a[j]);
  }
  for(int len=2; len<=n; len<<=1){
    double ang = 2 * PI / len * (invert ? -1 : 1);
    cd wlen(cos(ang), sin(ang));
    for(int i=0; i<n; i+=len){
      cd w(1);
      for(int j=0; j<len/2; j++){
        cd u = a[i+j], v = a[i+j+len/2] * w;
        a[i+j] = u + v;
        a[i+j+len/2] = u - v;
        w *= wlen;
      }
    }
  }
  if(invert){
    for(cd& x : a) x /= n;
  }
}

vector<long long> multiplyFFT(const vector<long long>& a, const vector<long long>& b){
  vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
  int n = 1;
  while(n < (int)a.size() + (int)b.size()) n <<= 1;
  fa.resize(n); fb.resize(n);
  fft(fa, false); fft(fb, false);
  for(int i=0;i<n;i++) fa[i] *= fb[i];
  fft(fa, true);
  vector<long long> res(n);
  for(int i=0;i<n;i++) res[i] = (long long)llround(fa[i].real());
  return res;
}

int main(){
  // multiply (1 + x + 2x^2) * (3 + 4x) = 3 + 7x + 10x^2 + 8x^3
  auto res = multiplyFFT({1,1,2}, {3,4});
  for(long long x : res) cout << x << " ";
  cout << "\n";
  return 0;
}
```

2.2 NTT (mod 998244353)

```cpp
// TC: O(N log N)
// usa: multiplicacion exacta de polinomios con coeficientes mod 998244353
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 998244353LL;
const long long PRIMITIVE_ROOT = 3LL;

void ntt(vector<long long>& a, bool invert){
  int n = a.size();
  for(int i=1,j=0;i<n;i++){
    int bit = n >> 1;
    for(; j&bit; bit>>=1) j ^= bit;
    j ^= bit;
    if(i < j) swap(a[i], a[j]);
  }
  for(int len=2; len<=n; len<<=1){
    long long wlen = modpow(PRIMITIVE_ROOT, (MOD - 1) / len, MOD);
    if(invert) wlen = modpow(wlen, MOD - 2, MOD);
    for(int i=0; i<n; i+=len){
      long long w = 1;
      for(int j=0; j<len/2; j++){
        long long u = a[i+j], v = a[i+j+len/2] * w % MOD;
        a[i+j] = (u + v) % MOD;
        a[i+j+len/2] = (u - v + MOD) % MOD;
        w = w * wlen % MOD;
      }
    }
  }
  if(invert){
    long long n_inv = modpow(n, MOD - 2, MOD);
    for(long long& x : a) x = x * n_inv % MOD;
  }
}

vector<long long> multiplyNTT(const vector<long long>& a, const vector<long long>& b){
  vector<long long> fa(a.begin(), a.end()), fb(b.begin(), b.end());
  int n = 1;
  while(n < (int)a.size() + (int)b.size()) n <<= 1;
  fa.resize(n); fb.resize(n);
  ntt(fa, false); ntt(fb, false);
  for(int i=0;i<n;i++) fa[i] = fa[i] * fb[i] % MOD;
  ntt(fa, true);
  return fa;
}

```

```cpp
int main(){
  // multiply (1 + x + 2x^2) * (3 + 4x) mod 998244353
  auto res = multiplyNTT({1,1,2}, {3,4});
  for(long long x : res) cout << x << " ";
  cout << "\n";
  return 0;
}
```

2.3 Multiplicacion de enteros grandes (FFT)

```cpp
// TC: O(N log N) donde N es la cantidad de digitos
// usa: requiere multiplyFFT ya definido. Soporta hasta ~1e6 digitos
#include <bits/stdc++.h>
using namespace std;

string multiplyBigInt(const string& num1, const string& num2){
  if(num1 == "0" || num2 == "0" || num1 == "-0" || num2 == "-0")
    return "0";
  bool neg = (num1[0] == '-') ^ (num2[0] == '-');
  int s1 = (num1[0] == '-' || num1[0] == '+') ? 1 : 0;
  int s2 = (num2[0] == '-' || num2[0] == '+') ? 1 : 0;

  vector<long long> a, b;
  for(int i = (int)num1.size() - 1; i >= s1; i--)
    a.push_back(num1[i]-'0');
  for(int i = (int)num2.size() - 1; i >= s2; i--)
    b.push_back(num2[i]-'0');

  vector<long long> c = multiplyFFT(a, b);

  long long carry = 0;
  string res = "";
  for(size_t i = 0; i < c.size() || carry > 0; i++){
    long long cur = carry + (i < c.size() ? c[i] : 0);
    res.push_back((char)('0' + (cur % 10)));
    carry = cur / 10;
  }
  while(res.size() > 1 && res.back() == '0') res.pop_back();
  if(neg && res != "0") res.push_back('-');
  reverse(res.begin(), res.end());
  return res;
}

int main(){
  // multiplica numeros arbitrariamente grandes en O(N log N)
  cout << multiplyBigInt("12345678901234567890", "98765432109876543210")
    << "\n";
  return 0;
}
```

2.4 Polynomial utilities (mod p)

```cpp
// TC: O(N log N) per operation (using NTT)
// usa: requiere multiplyNTT ya definido
#include <bits/stdc++.h>
using namespace std;

using Poly = vector<long long>;

Poly polyAdd(const Poly& a, const Poly& b){
  Poly c(max(a.size(), b.size()));
  for(size_t i=0;i<a.size();i++) c[i] = (c[i] + a[i]) % MOD;
  for(size_t i=0;i<b.size();i++) c[i] = (c[i] + b[i]) % MOD;
  return c;
}

Poly polySub(const Poly& a, const Poly& b){
  Poly c(max(a.size(), b.size()));
  for(size_t i=0;i<a.size();i++) c[i] = (c[i] + a[i]) % MOD;
  for(size_t i=0;i<b.size();i++) c[i] = (c[i] - b[i] + MOD) % MOD;
  return c;
}

// inverse: 1/B(x) mod x^n
Poly polyInv(const Poly& b, int n){
  Poly r(1, modpow(b[0], MOD - 2, MOD));
  int cur = 1;
  while(cur < n){
    cur <<= 1;
    Poly a(min((int)b.size(), cur));
    for(int i=0;i<(int)a.size();i++) a[i] = b[i];
    Poly nr = multiplyNTT(a, multiplyNTT(r, r));
    r.resize(cur);
    for(int i=0;i<cur;i++) r[i] = (2LL * r[i] - (i < (int)nr.size() ? nr[
    i] : 0) + MOD) % MOD;
  }
  r.resize(n);
  return r;
}

// derivative
Poly polyDeriv(const Poly& a){
  if(a.empty()) return {};
  Poly b((int)a.size() - 1);
  for(size_t i=1;i<a.size();i++) b[i-1] = a[i] * i % MOD;
  return b;
}

// integral
Poly polyInteg(const Poly& a){
  Poly b(a.size() + 1);
  // modular inverse of i+1 for each i
  static vector<long long> inv(1, 1);
  while((int)inv.size() <= (int)a.size()) inv.push_back(modpow(inv.size()
    , MOD - 2, MOD));
  for(size_t i=0;i<a.size();i++) b[i+1] = a[i] * inv[i+1] % MOD;
  return b;
}
```

```

int main(){
  Poly a = {1, 2, 3};
  Poly b = {4, 5};
  Poly c = polyAdd(a, b);
  Poly d = polySub(a, b);
  Poly deriv = polyDeriv(a);  // {2, 6}
  Poly integ = polyInteg(a);  // {0, 1, 1, 1}
  return 0;
}
```

2.4 Algebra lineal

2.1 Gauss-Jordan (mod p)

```
// TC: O(N^3) where N is the matrix dimension
// usa: resolver sistemas lineales, calcular determinante y rango
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 998244353LL;

// returns rank; mutates a in-place to row-echelon form
int gaussJordan(vector<vector<long long>>& a){
  int n = a.size(), m = a[0].size(), rank = 0;
  int row = 0;
  for(int col=0; col<m && row<n; col++){
    int sel = row;
    for(int i=row;i<n;i++) if(a[i][col]){ sel = i; break; }
    if(!a[sel][col]) continue;
    swap(a[sel], a[row]);
    long long inv = modpow(a[row][col], MOD - 2, MOD);
    for(int j=col;j<m;j++) a[row][j] = a[row][j] * inv % MOD;
    for(int i=0;i<n;i++){
      if(i != row && a[i][col]){
        long long factor = a[i][col];
        for(int j=col;j<m;j++){
          a[i][j] = (a[i][j] - factor * a[row][j] % MOD + MOD) % MOD;
        }
      }
    }
    row++; rank++;
  }
  return rank;
}

long long determinant(vector<vector<long long>> a){
  int n = a.size();
  long long det = 1;
  for(int i=0;i<n;i++){
    int sel = i;
    for(int j=i;j<n;j++) if(a[j][i]){ sel = j; break; }
    if(!a[sel][i]) return 0;
    if(sel != i){ swap(a[sel], a[i]); det = (MOD - det) % MOD; }
    det = det * a[i][i] % MOD;
    long long inv = modpow(a[i][i], MOD - 2, MOD);
    for(int j=i+1;j<n;j++){
      long long factor = a[j][i] * inv % MOD;
      for(int k=i;k<n;k++){
        a[j][k] = (a[j][k] - factor * a[i][k] % MOD + MOD) % MOD;
      }
    }
  }
  return det;
}

// --- Resolver Ax = b usando gaussJordan sobre la matriz aumentada ---
// gaussJordan devuelve el RANGO r de la matriz aumentada [A|b].
//   - Si tras la REF existe alguna fila i con todos los coeficientes
//     en cero pero el termino independiente aug[i][n] != 0, el
//     sistema es INCONSISTENTE: no hay solucion.
//   - Si r == n (rango == #incognitas), hay SOLUCION UNICA y vive
//     en la ultima columna de la RREF: x[i] = aug[i][n].
//   - Si r < n, quedan (n - r) variables libres: hay INFINITAS
//     soluciones. La ultima columna da una solucion particular
//     poniendo las libres en 0.
// Retorna:
//   0 -> sin solucion
//   1 -> solucion unica (escrita en 'sol')
//   2 -> infinitas soluciones (escribe una particular en 'sol')
int solveSystem(vector<vector<long long>> A, vector<long long> b,
        vector<long long>& sol){
  int n = (int)A.size();
  // 1) construir la matriz aumentada [A | b] de tamano n x (n+1)
  vector<vector<long long>> aug(n, vector<long long>(n + 1));
  for(int i = 0; i < n; i++){
    for(int j = 0; j < n; j++) aug[i][j] = A[i][j];
    aug[i][n] = b[i];
  }

  // 2) RREF + rango
  int r = gaussJordan(aug);

  // 3) checar inconsistencia: fila nula con termino independiente != 0
  for(int i = r; i < n; i++)
    if(aug[i][n] != 0) return 0;   // sin solucion

  // 4) rango < #incognitas -> variables libres -> infinitas soluciones
  if(r < n) return 2;

  // 5) solucion unica: se lee directo de la ultima columna
  sol.assign(n, 0);
  for(int i = 0; i < n; i++) sol[i] = aug[i][n];
  return 1;
}

int main(){
  vector<vector<long long>> A = {{1,2,3},{4,5,6},{7,8,10}};
  int r = gaussJordan(A);
  cout << "rank = " << r << "\n";
```

```
  cout << "det = " << determinant({{1,2},{3,4}}) << "\n";  // -2 mod p

  // Ejemplo: 2x + y = 5 ,  x - y = 1   ->  x = 2, y = 1
  vector<vector<long long>> M = {{2,1},{1,-1}};
  vector<long long> b = {5,1}, x;
  int status = solveSystem(M, b, x);   // status = 1, x = {2, 1}
  for(long long v : x) cout << v << " ";
  cout << "\n";
  return 0;
}
```

2.2 Matrix exponentiation

```
// TC: O(N^3 log k)
// usa: recurrencias lineales (Fibonacci, contar caminos de longitud k, etc.)
#include <bits/stdc++.h>
using namespace std;

using Matrix = vector<vector<long long>>;

Matrix matMul(const Matrix& a, const Matrix& b, long long MOD){
  int n = a.size();
  Matrix c(n, vector<long long>(n, 0));
  for(int i=0;i<n;i++)
    for(int k=0;k<n;k++) if(a[i][k])
      for(int j=0;j<n;j++)
        c[i][j] = (c[i][j] + a[i][k] * b[k][j]) % MOD;
  return c;
}

Matrix matPow(Matrix base, long long exp, long long MOD){
  int n = base.size();
  Matrix res(n, vector<long long>(n, 0));
  for(int i=0;i<n;i++) res[i][i] = 1;
  while(exp){
    if(exp & 1) res = matMul(res, base, MOD);
    base = matMul(base, base, MOD);
    exp >>= 1;
  }
  return res;
}

int main(){
  // Fibonacci: F(n) = F(n-1) + F(n-2)
  Matrix T = {{1,1},{1,0}};
  Matrix Fn = matPow(T, 10, MOD);
  cout << "F(10) = " << Fn[0][1] << "\n";  // 55
  return 0;
}
```

2.5 Juegos y teoria de juegos

2.1 Sprague-Grundy template

```
// TC: depends on game; usually O(states * transitions)
// usa: juegos impartial, nim-heap decomposition
#include <bits/stdc++.h>
using namespace std;

unordered_map<long long, long long> sgMemo;

long long grundy(long long state){
  if(sgMemo.count(state)) return sgMemo[state];
  set<long long> reachable;
  // populate 'reachable' with grundy of all positions reachable from 'state'
  // ... (game-specific) ...
  long long mex = 0;
  while(reachable.count(mex)) mex++;
  return sgMemo[state] = mex;
}

int main(){
  // Nim with 3 piles of sizes 1, 2, 3
  int g = grundy(1) ^ grundy(2) ^ grundy(3);
  cout << (g ? "first player wins" : "second player wins") << "\n";
  return 0;
}
```

2.2 Small game tables (Kayles SG values)

```
// TC: O(1) per query (table lookup)
// usa: cuando el juego tiene espacio de estados pequeno y conocido
#include <bits/stdc++.h>
using namespace std;

// Kayles grundy values for n=0..71 (precomputed table)
vector<int> kaylesSG = {
  0,1,2,3,1,4,3,2,1,4,2,6,4,5,1,1,7,4,3,2,1,4,2,6,4,5,1,1,7,4,3,2,
  1,4,2,6,4,5,1,1,7,4,3,2,1,4,2,6,4,5,1,1,7,4,3,2,1,4,2,6,4,5,1,1,
  7,4,3,2,1,4,2,6
};

int main(){
  cout << "SG Kayles(7) = " << kaylesSG[7] << "\n";
  return 0;
}
```

3 Estructuras de datos

3.1 Ordenado y contenedores

3.1 Ordered Set (pbds)

```cpp
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/trie_policy.hpp>

using namespace __gnu_pbds;

typedef tree<int, null_type, less<int>, rb_tree_tag,
tree_order_statistics_node_update> pbds;
// find_by_order, order_of_key ; greater<int>, less_equal<int>
// insert -> logn | delete -> | find kth element -> logn
// find # of elements smaller than x -> logn
// up and lb work opposite
// to erase in a multiset s.erase(s.upperbound(x))
```

3.2 Basis (XOR basis)

```cpp
// TC: O(D^2) por insercion (D = numero de bits, aca 64), O(D^2) total
    para n inserciones
// usa: representar conjunto de numeros como base de Gauss sobre GF(2);
    maximo xor subconjunto, pertenencia, k-esimo xor
struct Basis{
  int d=64;
  vector<int> basis;
  int sz=0; // current size of the basis
  Basis(){
    basis.resize(d);
  }
  void insertVector(int mask){
    for(int i=d-1 ; i>=0 ; i--){
      if( (mask & (1LL<<i)) == 0 ) continue;
      if( !basis[i] ){
        basis[i]=mask;
        ++sz;
        return;
      }
      mask ^= basis[i];
    }
  }
};

int main(){
  // ejemplo: dado un conjunto de numeros, encontrar el maximo xor de un
    subconjunto
  vector<int> nums = {1, 2, 3, 5};
  Basis b;
  for(int x : nums) b.insertVector(x);
  int ans = 0;
  for(int i=b.d-1 ; i>=0 ; i--) ans = max(ans, ans ^ b.basis[i]);
  // ans = maximo xor de un subconjunto de nums
  return 0;
}
```

3.3 Trie

```cpp
// TC: O(L) por operacion (L = largo de la palabra) en tiempo, O(N * L)
    en memoria (N palabras)
// usa: insertar, buscar y verificar prefijos sobre un alfabeto (aca 26
    letras)
struct Node{
  Node *links[26];
  bool flag=false;
  bool containsKey(char ch){
    return links[ch-'a'] != NULL;
  }
  void put(char ch, Node* node){
    links[ch-'a'] = node;
  }
  Node* get(char ch){
    return links[ch-'a'];
  }
  void setEnd(){
    flag=true;
  }
  bool isEnd(){
    return flag;
  }
};
class Trie{
  private:
  Node *root;
  public:
  Trie(){
    root = new Node();
  }
  // tc -> O(len)
  void insert(string word){
    Node* node = root;
    for(int i=0 ; i<word.size() ; i++){
      if(!node->containsKey(word[i])){
        node->put(word[i], new Node());
      }
      // moves to reference trie
      node = node->get(word[i]);
    }
    node->setEnd();
  }
  // tc -> O(len)
  bool search(string word){
    Node* node = root;
    for(int i=0 ; i<word.size() ; i++){
      if(!node->containsKey(word[i])){
        return false;
      }
      node = node->get(word[i]);
```

```
    }
    return node->isEnd();
  }
  // tc -> O(len)
  bool startsWith(string prefix){
    Node* node = root;
    for(int i=0 ; i<prefix.size() ; i++){
      if(!node->containsKey(prefix[i])){
        return false;
      }
      node = node->get(prefix[i]);
    }
    return true;
  }
};
int main(){
  Trie trie;
  trie.insert("hello");
  trie.insert("world");
  cout << trie.search("hello") << endl; // 1
  cout << trie.search("hell") << endl;  // 0
  cout << trie.startsWith("hell") << endl; // 1
  cout << trie.startsWith("wor") << endl;  // 1
  return 0;
}
```

3.2 Union-Find

3.1 DSU classico

```
class DisjointSet{
  vector<int> parent, size;
  public:
  DisjointSet(int n){
    parent.resize(n+1);
    size.resize(n+1);
    for(int i=0 ; i<=n ; i++){
      parent[i]=i;
      size[i]=1;
    }
  }
  int findUPar(int node){
    if(node == parent[node]) return node;
    return parent[node]=findUPar(parent[node]);
  }
  void UnionBySize(int u, int v){
    int ulp_u=findUPar(u);
    int ulp_v=findUPar(v);
    if(ulp_u == ulp_v) return;
    if(size[ulp_u] < size[ulp_v]){
      parent[ulp_u]=ulp_v;
      size[ulp_v]+=size[ulp_u];
    }else{
      parent[ulp_v]=ulp_u;
      size[ulp_u]+=size[ulp_v];
    }
  }
};
```

3.2 DSU con rollback

```
// TC: O(log N) amortized per union/find; rollback is O(1) undo of last
    union
// usa: divide & conquer offline, Mo con updates, undoable DSU
#include <bits/stdc++.h>
using namespace std;

struct DSUrb{
  vector<int> parent, sz;
  vector<pair<int,int>> history; // stack of (node, old_size)
  int components;

  DSUrb(int n){
    parent.resize(n);
    sz.assign(n, 1);
    iota(parent.begin(), parent.end(), 0);
    components = n;
  }

  int find(int v){
    // no path compression to keep rollback easy
    while(v != parent[v]) v = parent[v];
    return v;
  }

  bool unite(int a, int b){
    a = find(a); b = find(b);
    if(a == b) return false;
    if(sz[a] > sz[b]) swap(a, b);
    history.push_back({a, sz[a]});
    history.push_back({b, sz[b]});
    parent[a] = b;
    sz[b] += sz[a];
    components--;
    return true;
  }

  void snapshot(){
    history.push_back({-1, -1}); // marker
  }

  void rollback(){
    // undo until last marker (or to empty if no markers)
    while(!history.empty() && history.back().first != -1){
      auto [v, sOld] = history.back(); history.pop_back();
      auto [w, sO]   = history.back(); history.pop_back();
      parent[v] = v;
      sz[w] = sO;
      components++;
```

```
    }
    if(!history.empty()) history.pop_back(); // pop marker
  }
};

int main(){
  DSUrb d(5);
  d.unite(0,1); d.unite(1,2);   // {0,1,2} merged
  cout << d.components << "\n";  // 3
  d.rollback();
  cout << d.components << "\n";  // 5 (everything undone)
  return 0;
}
```

3.3 Segment Trees

3.1 Segment Tree Add + Sum

```
// TC: O(log n) por operacion, O(n) en memoria
// usa: sumar un valor a todos los elementos de un rango y consultar la
    suma en un rango
struct SegmentTree {
  struct Node {
    int val = 0, lazy = 0;
  };

  int n;
  vector<Node> st;

  SegmentTree(int size) {
    n = 1;
    while(__builtin_popcount(n) != 1) n++;
    st.resize(2*n);
  }

  void push(int node, int l, int r) {
    if(st[node].lazy == 0) return;
    st[node].val += (r-l+1) * st[node].lazy;
    if(l != r) {
      st[node*2].lazy += st[node].lazy;
      st[node*2+1].lazy += st[node].lazy;
    }
    st[node].lazy = 0;
  }

  void add_range(int node, int l, int r, int ql, int qr, int val) {
    push(node, l, r);
    if(ql <= l && r <= qr) {
      st[node].lazy += val;
      push(node, l, r);
      return;
    }
    if(r < ql || qr < l) return;
    int mid = l + (r-l)/2;
    add_range(node*2, l, mid, ql, qr, val);
    add_range(node*2+1, mid+1, r, ql, qr, val);
    st[node].val = st[node*2].val + st[node*2+1].val;
  }

  int query(int node, int l, int r, int ql, int qr) {
    push(node, l, r);
    if(ql <= l && r <= qr) return st[node].val;
    if(r < ql || qr < l) return 0;
    int mid = l + (r-l)/2;
    return query(node*2, l, mid, ql, qr)
    + query(node*2+1, mid+1, r, ql, qr);
  }

  void add(int l, int r, int val) {
    add_range(1, 0, n-1, l, r, val);
  }
  int sum(int l, int r) {
    return query(1, 0, n-1, l, r);
  }
};
int main(){
  SegmentTree st(5);
  st.add(0, 4, 1);              // [1,1,1,1,1]
  st.add(1, 3, 2);              // [1,3,3,3,1]
  cout << st.sum(0, 4) << endl; // 11
  cout << st.sum(1, 3) << endl; // 9
  st.add(0, 4, -1);             // [0,2,2,2,0]
  cout << st.sum(0, 4) << endl; // 6
  return 0;
}
```

3.2 Segment Tree Lazy Assign + Sum

```
// TC: O(log n) por operacion, O(n) en memoria
// usa: asignar un valor a un rango y consultar la suma en un rango
struct SegmentTree {
  struct Node {
    int val = 0;
    int lazy = LLONG_MAX; // valor neutro
  };

  int n;
  vector<Node> st;

  SegmentTree(int size) {
    n = size;
    while(__builtin_popcount(n) != 1) n++;
    st.resize(2*n);
  }

  void push(int node, int l, int r) {
    if(st[node].lazy == LLONG_MAX) return;
    st[node].val = (r-l+1) * st[node].lazy;
    if(l != r) {
```

```
      st[node*2].lazy = st[node].lazy;
      st[node*2+1].lazy = st[node].lazy;
    }
    st[node].lazy = LLONG_MAX;
  }

  void assign_range(int node, int l, int r, int ql, int qr, int val) {
    push(node, l, r);
    if(ql <= l && r <= qr) {
      st[node].lazy = val;
      push(node, l, r);
      return;
    }
    if(r < ql || qr < l) return;
    int mid = l + (r-l)/2;
    assign_range(node*2, l, mid, ql, qr, val);
    assign_range(node*2+1, mid+1, r, ql, qr, val);
    st[node].val = st[node*2].val + st[node*2+1].val;
  }

  int query(int node, int l, int r, int ql, int qr) {
    push(node, l, r);
    if(ql <= l && r <= qr) return st[node].val;
    if(r < ql || qr < l) return 0;
    int mid = l + (r-l)/2;
    return query(node*2, l, mid, ql, qr)
    + query(node*2+1, mid+1, r, ql, qr);
  }

  void assign(int l, int r, int val) {
    assign_range(1, 0, n-1, l, r, val);
  }
  int sum(int l, int r) {
    return query(1, 0, n-1, l, r);
  }
};
int main(){
  SegmentTree st(5);
  st.assign(0, 4, 1);           // [1,1,1,1,1]
  st.assign(1, 3, 2);           // [1,2,2,2,1]
  cout << st.sum(0, 4) << endl; // 8
  cout << st.sum(1, 3) << endl; // 6
  st.assign(0, 4, 5);           // [5,5,5,5,5]
  cout << st.sum(0, 4) << endl; // 25
  return 0;
}
```

3.3 Segment Tree Add + lower bound index

```
struct SegmentTree {
  struct Node {
    int val = 0, lazy = 0;
  };

  int n;
  vector<Node> st;

  SegmentTree(int size) {
    n = size;
    while(__builtin_popcount(n) != 1) n++;
    st.resize(2*n);
  }

  void push(int node, int l, int r) {
    if(st[node].lazy == 0) return;
    st[node].val += st[node].lazy;
    if(l != r) {
      st[node*2].lazy += st[node].lazy;
      st[node*2+1].lazy += st[node].lazy;
    }
    st[node].lazy = 0;
  }

  void add_range(int node, int l, int r, int ql, int qr, int val) {
    push(node, l, r);
    if(ql <= l && r <= qr) {
      st[node].lazy = val;
      push(node, l, r);
      return;
    }
    if(r < ql || qr < l) return;
    int mid = l + (r-l)/2;
    add_range(node*2, l, mid, ql, qr, val);
    add_range(node*2+1, mid+1, r, ql, qr, val);
    st[node].val = max(st[node*2].val, st[node*2+1].val);
  }

  int query(int node, int l, int r, int val, int bound) {
    push(node, l, r);
    if(r < bound || st[node].val < val) return -1;
    if(l == r) return l;
    int mid = l + (r-l)/2;
    int left = query(node*2, l, mid, val, bound);
    if(left == -1) return query(node*2+1, mid+1, r, val, bound);
    return left;
  }

  void add(int l, int r, int val) {
    add_range(1, 0, n-1, l, r, val);
  }
  int lower_bound_index(int val, int bound) {
    return query(1, 0, n-1, val, bound);
  }
};
```

3.4 Segment Tree Lazy Add + Set + Sum

```
// TC: O(log n) por operacion, O(n) en memoria
// usa: sumar o asignar valores en rangos y consultar sumas
```

```cpp
// lazy_set = LLONG_MAX indica "no hay set pendiente" (sirve para set a 0)
struct SegmentTree {
  struct Node {
    int val = 0, lazy_add = 0;
    int lazy_set = LLONG_MAX;
    Node(int x=0) { val = x; lazy_add = 0; lazy_set = LLONG_MAX; }
  };

  int n;
  vector<Node> st;

  SegmentTree(const vector<int>& a) {
    n = a.size();
    while(__builtin_popcount(n) != 1) n++;
    st.resize(2*n);
    for(int i=0 ; i<(int)a.size() ; i++)
    st[n+i] = Node(a[i]);
    for(int i=n-1 ; i>=1; i--)
    st[i] = merge(st[2*i], st[2*i+1]);
  }

  Node merge(Node &left, Node &right) {
    Node ans;
    ans.val = left.val + right.val;
    return ans;
  }

  void push(int node, int l, int r) {
    if(st[node].lazy_set != LLONG_MAX) {
      st[node].val = st[node].lazy_set * (r-l+1);
      if(l != r) {
        st[node*2].lazy_set = st[node].lazy_set;
        st[node*2].lazy_add = 0;
        st[node*2+1].lazy_set = st[node].lazy_set;
        st[node*2+1].lazy_add = 0;
      }
      st[node].lazy_set = LLONG_MAX;
    }
    if(st[node].lazy_add != 0) {
      st[node].val += st[node].lazy_add * (r-l+1);
      if(l != r) {
        st[node*2].lazy_add += st[node].lazy_add;
        st[node*2+1].lazy_add += st[node].lazy_add;
      }
      st[node].lazy_add = 0;
    }
  }

  void update(int node, int l, int r, int ql, int qr, int val,
  int type) {
    push(node, l, r);
    if(ql <= l && r <= qr) {
      if(type == 1) st[node].lazy_add += val;
      else st[node].lazy_set = val;
      push(node, l, r);
      return;
    }
    if(r < ql || qr < l) return;
    int mid = l + (r-l)/2;
    update(node*2, l, mid, ql, qr, val, type);
    update(node*2+1, mid+1, r, ql, qr, val, type);
    st[node].val = st[node*2].val + st[node*2+1].val;
  }

  int query(int node, int l, int r, int ql, int qr) {
    push(node, l, r);
    if(ql <= l && r <= qr) return st[node].val;
    if(r < ql || qr < l) return 0;
    int mid = l + (r-l)/2;
    return query(node*2, l, mid, ql, qr)
    + query(node*2+1, mid+1, r, ql, qr);
  }

  // Funciones publicas
  void add_range(int l, int r, int val) {
    update(1, 0, n-1, l, r, val, 1);
  }
  void set_range(int l, int r, int val) {
    update(1, 0, n-1, l, r, val, 2);
  }
  int sum(int l, int r) {
    return query(1, 0, n-1, l, r);
  }
};
int main(){
  SegmentTree st({1, 2, 3, 4, 5});
  st.add_range(0, 4, 2);        // [3,4,5,6,7]
  cout << st.sum(0, 4) << endl; // 25
  st.set_range(1, 3, 0);        // [3,0,0,0,7]
  st.add_range(0, 4, 1);        // [4,1,1,1,8]
  cout << st.sum(0, 4) << endl; // 15
  st.set_range(2, 4, 5);        // [4,1,5,5,5]
  cout << st.sum(2, 4) << endl; // 15
  return 0;
}
```

3.5 MO

```cpp
// TC: O((n+q) * sqrt(n)) offline. Ordena queries por bloque de l, y dentro del bloque por r alternado.
// usa: responder muchas queries sobre subarreglos cuando se puede agregar/quitar un elemento en O(1)
//      (ej: contar distintos, suma en rango, frecuencia, xor, etc).
struct Q{
  int l,r,idx;
};

int main(){
  int n,m; cin >> n >> m;
  vector<int> a(n);
```

```
  for(int i=0 ; i<n ; i++) cin >> a[i];

  vector<Q> q(m);
  for(int i=0 ; i<m ; i++){
    int l,r; cin >> l >> r;
    q[i].l = l-1;
    q[i].r = r-1;
    q[i].idx = i;
  }

  int rootN = sqrtl(n);
  sort(q.begin(), q.end(), [&](Q x, Q y){
    int bx = x.l / rootN, by = y.l / rootN;
    if(bx != by) return bx < by;
    return (bx & 1) ? (x.r > y.r) : (x.r < y.r);
  });

  vector<int> freq(1000001, 0);
  long long curr = 0;
  vector<long long> ans(m);

  auto add = [&](int pos){
    int x = a[pos];
    if(freq[x] == 0) curr++;
    freq[x]++;
  };
  auto del = [&](int pos){
    int x = a[pos];
    freq[x]--;
    if(freq[x] == 0) curr--;
  };

  int l=0, r=-1;
  for(int i=0 ; i<m ; i++){
    int ql=q[i].l, qr=q[i].r;
    while(r < qr){ add(++r); }
    while(r > qr){ del(r--); }
    while(l < ql){ del(l++); }
    while(l > ql){ add(--l); }
    ans[q[i].idx] = curr; // cuenta los distintos en [ql, qr]
  }

  for(int i=0 ; i<m ; i++) cout << ans[i] << "\n";
  return 0;
}
```

3.6 Segment Tree 2D (con BASE)

```
// TC: O(log N * log M) por operacion
// usa: sin struct, BASE_X y BASE_Y potencias de 2, queries en rectangulo
#include <bits/stdc++.h>
using namespace std;

int BASE_X = 1, BASE_Y = 1;
vector<vector<int>> st;

void build(int n, int m, const vector<vector<int>>& a = {}){
  BASE_X = 1; while(BASE_X < n) BASE_X <<= 1;
  BASE_Y = 1; while(BASE_Y < m) BASE_Y <<= 1;
  st.assign(2 * BASE_X, vector<int>(2 * BASE_Y, 0));

  if(!a.empty()){
    for(int i = 0; i < n; i++)
      for(int j = 0; j < m; j++)
        st[BASE_X + i][BASE_Y + j] = a[i][j];

    // Construir arboles en dimension Y
    for(int i = BASE_X; i < 2 * BASE_X; i++)
      for(int j = BASE_Y - 1; j > 0; j--)
        st[i][j] = st[i][2 * j] + st[i][2 * j + 1];

    // Construir arboles en dimension X
    for(int i = BASE_X - 1; i > 0; i--)
      for(int j = 1; j < 2 * BASE_Y; j++)
        st[i][j] = st[2 * i][j] + st[2 * i + 1][j];
  }
}

// Update puntual en celda (x, y) con val
void update(int x, int y, int val){
  int px = BASE_X + x, py = BASE_Y + y;
  st[px][py] = val;
  for(int j = py / 2; j > 0; j >>= 1){
    st[px][j] = st[px][2 * j] + st[px][2 * j + 1];
  }
  for(int i = px / 2; i > 0; i >>= 1){
    for(int j = py; j > 0; j >>= 1){
      st[i][j] = st[2 * i][j] + st[2 * i + 1][j];
    }
  }
}

// Consultas recursivas con ly, ry, qly, qry y lx, rx, qlx, qrx
int _query_y(int nodex, int nodey, int ly, int ry, int qly, int qry){
  if(ry < qly || qry < ly) return 0;
  if(qly <= ly && ry <= qry) return st[nodex][nodey];
  int mid = ly + (ry - ly) / 2;
  return _query_y(nodex, 2 * nodey, ly, mid, qly, qry) +
         _query_y(nodex, 2 * nodey + 1, mid + 1, ry, qly, qry);
}

int _query_x(int nodex, int lx, int rx, int qlx, int qrx, int qly, int
    qry){
  if(rx < qlx || qrx < lx) return 0;
  if(qlx <= lx && rx <= qrx){
    return _query_y(nodex, 1, 0, BASE_Y - 1, qly, qry);
  }
  int mid = lx + (rx - lx) / 2;
  return _query_x(2 * nodex, lx, mid, qlx, qrx, qly, qry) +
         _query_x(2 * nodex + 1, mid + 1, rx, qlx, qrx, qly, qry);
}
```

```
int query(int qlx, int qrx, int qly, int qry){
  return _query_x(1, 0, BASE_X - 1, qlx, qrx, qly, qry);
}

int main(){
  build(4, 4);
  update(1, 1, 5);
  update(2, 2, 7);
  cout << query(0, 3, 0, 3) << "\n"; // 12
  cout << query(1, 1, 1, 1) << "\n"; // 5
  return 0;
}
```

3.7 Segment Tree persistente

```
// TC: O(log N) per query, O(log N) per update (creates new root)
// usa: queries en versiones historicas del arreglo
#include <bits/stdc++.h>
using namespace std;

struct PNode{
  long long val;
  PNode *l, *r;
  PNode(long long v = 0, PNode* l = nullptr, PNode* r = nullptr): val(v),
     l(l), r(r){}
};

PNode* pBuild(int l, int r, const vector<long long>& a){
  if(l == r) return new PNode(a[l]);
  int m = (l + r) / 2;
  return new PNode(0, pBuild(l, m, a), pBuild(m+1, r, a));
}

PNode* pUpdate(PNode* node, int l, int r, int pos, long long val){
  if(l == r) return new PNode(val);
  int m = (l + r) / 2;
  if(pos <= m) return new PNode(0, pUpdate(node->l, l, m, pos, val), node
    ->r);
  else return new PNode(0, node->l, pUpdate(node->r, m+1, r, pos, val));
}

long long pQuery(PNode* node, int l, int r, int ql, int qr){
  if(!node || ql > r || qr < l) return 0;
  if(ql <= l && r <= qr) return node->val;
  int m = (l + r) / 2;
  return pQuery(node->l, l, m, ql, qr) + pQuery(node->r, m+1, r, ql, qr);
}

int main(){
  vector<long long> a = {1, 2, 3, 4, 5};
  PNode* root = pBuild(0, 4, a);
  PNode* v2 = pUpdate(root, 0, 4, 2, 10);  // set a[2] = 10
  cout << pQuery(root, 0, 4, 0, 4) << "\n";  // 15 (v1)
  cout << pQuery(v2, 0, 4, 0, 4) << "\n";    // 22 (v2)
  return 0;
}
```

3.8 Segment Tree (con BASE)

```
// TC: O(log N) por operacion, O(N) espacio
// usa: sin struct, BASE potencia de 2, update punto y query rango
#include <bits/stdc++.h>
using namespace std;

int BASE = 1;
vector<long long> st;

void build(int n, const vector<long long>& a = {}){
  BASE = 1;
  while(BASE < n) BASE <<= 1;
  st.assign(2 * BASE, 0); // o -INF para max
  for(int i = 0; i < (int)a.size(); i++) st[BASE + i] = a[i];
  for(int i = BASE - 1; i > 0; i--) st[i] = st[2 * i] + st[2 * i + 1];
}

// Update puntual: asigna val en posicion u
void update(int u, long long val){
  st[BASE + u] = val;
  for(int i = (BASE + u) / 2; i > 0; i >>= 1){
    st[i] = st[2 * i] + st[2 * i + 1];
  }
}

// 1. Consulta iterativa en rango [ql, qr]:
long long query_it(int ql, int qr){
  long long res = 0;
  for(int l = BASE + ql, r = BASE + qr + 1; l < r; l >>= 1, r >>= 1){
    if(l & 1) res += st[l++];
    if(r & 1) res += st[--r];
  }
  return res;
}

// 2. Consulta recursiva con l, r, ql, qr:
long long _get(int node, int l, int r, int ql, int qr){
  if(r < ql || qr < l) return 0;
  if(ql <= l && r <= qr) return st[node];
  int mid = l + (r - l) / 2;
  return _get(2 * node, l, mid, ql, qr) +
         _get(2 * node + 1, mid + 1, r, ql, qr);
}

long long get(int ql, int qr){
  return _get(1, 0, BASE - 1, ql, qr);
}

int main(){
  int n = 5;
  vector<long long> a = {1, 2, 3, 4, 5};
```

```
  build(n, a);
  cout << get(1, 3) << "\n";       // 9 (2 + 3 + 4)
  update(2, 10);
  cout << query_it(1, 3) << "\n"; // 16 (2 + 10 + 4)
  return 0;
}
```

3.4 Fenwick / BIT

3.1 BIT point update + prefix sum

```
// TC: O(log N) per operation
// usa: queries de suma en prefijo
#include <bits/stdc++.h>
using namespace std;

struct BIT{
  int n;
  vector<long long> t;
  BIT(int n): n(n), t(n+1, 0){}
  void add(int i, long long delta){
    for(; i <= n; i += i & -i) t[i] += delta;
  }
  long long sum(int i){
    long long r = 0;
    for(; i > 0; i -= i & -i) r += t[i];
    return r;
  }
  long long rangeSum(int l, int r){
    return sum(r) - sum(l - 1);
  }
};

int main(){
  BIT bit(10);
  bit.add(3, 5);
  bit.add(7, 2);
  cout << bit.sum(5) << "\n";      // 5
  cout << bit.rangeSum(1, 10) << "\n";  // 7
  return 0;
}
```

3.2 BIT range add + point query

```
// TC: O(log N) per operation
// usa: sumar delta a [l, r] y consultar un punto
#include <bits/stdc++.h>
using namespace std;

struct BITRangeAdd{
  int n;
  vector<long long> t;
  BITRangeAdd(int n): n(n), t(n+1, 0){}
  void add(int i, long long delta){
    for(; i <= n; i += i & -i) t[i] += delta;
  }
  void rangeAdd(int l, int r, long long delta){
    add(l, delta);
    if(r + 1 <= n) add(r + 1, -delta);
  }
  long long pointQuery(int i){
    long long r = 0;
    for(; i > 0; i -= i & -i) r += t[i];
    return r;
  }
};

int main(){
  BITRangeAdd bit(10);
  bit.rangeAdd(2, 5, 3);
  cout << bit.pointQuery(1) << "\n";  // 0
  cout << bit.pointQuery(3) << "\n";  // 3
  cout << bit.pointQuery(6) << "\n";  // 0
  return 0;
}
```

3.3 BIT range add + range sum

```
// TC: O(log N) per operation
// usa: sumar delta a [l, r] y consultar suma de prefijo
#include <bits/stdc++.h>
using namespace std;

struct BIT2{
  int n;
  vector<long long> t1, t2;
  BIT2(int n): n(n), t1(n+1, 0), t2(n+1, 0){}
  void add(vector<long long>& t, int i, long long delta){
    for(; i <= n; i += i & -i) t[i] += delta;
  }
  long long sum(const vector<long long>& t, int i){
    long long r = 0;
    for(; i > 0; i -= i & -i) r += t[i];
    return r;
  }
  void rangeAdd(int l, int r, long long delta){
    add(t1, l, delta);
    add(t1, r + 1, -delta);
    add(t2, l, delta * (l - 1));
    add(t2, r + 1, -delta * r);
  }
  long long prefixSum(int i){
    return sum(t1, i) * i - sum(t2, i);
  }
  long long rangeSum(int l, int r){
    return prefixSum(r) - prefixSum(l - 1);
  }
```

```
};

int main(){
  BIT2 bit(10);
  bit.rangeAdd(2, 5, 3);
  cout << bit.rangeSum(1, 10) << "\n";  // 12
  cout << bit.rangeSum(3, 5) << "\n";   // 9
  return 0;
}
```

3.5 RMQ y sparse table

3.1 Sparse Table

```
// TC: O(N log N) build, O(1) query
// usa: min/max/gcd en rango estatico (sin updates)
#include <bits/stdc++.h>
using namespace std;

struct SparseTable{
  int n, LOG;
  vector<vector<int>> st;
  vector<int> lg;
  SparseTable(const vector<int>& a){
    n = a.size();
    LOG = 32 - __builtin_clz(n);
    st.assign(LOG, vector<int>(n));
    st[0] = a;
    for(int k=1;k<LOG;k++){
      for(int i=0;i + (1<<k) <= n;i++){
        st[k][i] = min(st[k-1][i], st[k-1][i + (1<<(k-1))]);
      }
    }
    lg.assign(n + 1, 0);
    for(int i=2;i<=n;i++) lg[i] = lg[i/2] + 1;
  }
  int query(int l, int r){
    int k = lg[r - l + 1];
    return min(st[k][l], st[k][r - (1<<k) + 1]);
  }
};

int main(){
  SparseTable st({3, 1, 4, 1, 5, 9, 2, 6});
  cout << st.query(0, 4) << "\n";  // 1 (min in [3,1,4,1,5])
  cout << st.query(2, 7) << "\n";  // 2
  return 0;
}
```

3.2 Disjoint Sparse Table

```
// TC: O(N log N) build, O(1) query
// usa: queries idempotentes en arreglos estaticos (alternativa a Sparse Table)
#include <bits/stdc++.h>
using namespace std;

struct DisjointSparseTable{
  int n, LOG;
  vector<vector<int>> st;
  DisjointSparseTable(const vector<int>& a){
    n = a.size();
    LOG = (n <= 1 ? 1 : 32 - __builtin_clz(n - 1) + 1);
    st.assign(LOG, a);
    for(int k=1; k<LOG; ++k){
      int half = 1 << k;
      for(int mid = half; mid < n; mid += 2 * half){
        for(int i = mid - 2; i >= mid - half; --i)
          st[k][i] = min(a[i], st[k][i + 1]);
        int lim = min(n, mid + half);
        for(int i = mid + 1; i < lim; ++i)
          st[k][i] = min(st[k][i - 1], a[i]);
      }
    }
  }
  int query(int l, int r){
    if(l == r) return st[0][l];
    int k = 31 - __builtin_clz(l ^ r);
    return min(st[k][l], st[k][r]);
  }
};

int main(){
  DisjointSparseTable dst({3, 1, 4, 1, 5, 9, 2, 6});
  cout << dst.query(0, 4) << "\n";  // 1
  cout << dst.query(2, 7) << "\n";  // 2
  return 0;
}
```

3.6 Trees y DP en arbol

3.1 Binary Lifting / LCA

```
// TC: O(N log N) build, O(log N) por query
// usa: lowest common ancestor y k-esimo ancestro en arbol
#include <bits/stdc++.h>
using namespace std;

const int LOG = 20; // 2^20 > 1e6
vector<int> depth;
vector<vector<int>> up;

void dfs(int node, int par, int d, vector<int> adj[]){
  depth[node] = d;
  up[0][node] = par;
  for(auto adjN : adj[node]) if(adjN != par){
```

```
    dfs(adjN, node, d + 1, adj);
  }
}

// Camina 'step' pasos hacia arriba desde 'node' (k-th ancestor)
int walk(int node, int step){
  for(int i = 0; i <= LOG; i++) if((step >> i) & 1){
    node = up[i][node];
  }
  return node;
}

int get_lca(int u, int v){
  if(depth[u] > depth[v]) swap(u, v);
  v = walk(v, depth[v] - depth[u]);
  if(u == v) return u;
  for(int i = LOG; i >= 0; i--){
    if(up[i][u] != up[i][v]){
      u = up[i][u];
      v = up[i][v];
    }
  }
  return up[0][u];
}

int dist(int u, int v){
  return depth[u] + depth[v] - 2 * depth[get_lca(u, v)];
}

void init_lca(int root, int n, vector<int> adj[]){
  depth.assign(n, 0);
  up.assign(LOG + 1, vector<int>(n, 0));
  dfs(root, root, 0, adj);
  for(int i = 1; i <= LOG; i++){
    for(int j = 0; j < n; j++){
      up[i][j] = up[i - 1][up[i - 1][j]];
    }
  }
}
```

3.2 Heavy-Light Decomposition (HLD)

```
// TC: O(N) build, O(log^2 N) por query en camino entre u y v
// usa: descomponer arbol en cadenas pesadas para queries de camino con SegTree
#include <bits/stdc++.h>
using namespace std;

vector<int> heavy, sz, parent, depth, pos, head;
int t = -1;

void dfs1(int node, int par, int d, vector<int> adj[]){
  parent[node] = par;
  depth[node] = d;
  sz[node] = 1;
  int mx = 0;
  for(auto adjN : adj[node]) if(adjN != par){
    dfs1(adjN, node, d + 1, adj);
    sz[node] += sz[adjN];
    if(sz[adjN] > mx){
      mx = sz[adjN];
      heavy[node] = adjN;
    }
  }
}

void dfs2(int node, int h, vector<int> adj[]){
  head[node] = h;
  pos[node] = ++t;
  if(heavy[node] != -1) dfs2(heavy[node], h, adj);
  for(auto adjN : adj[node]){
    if(adjN == parent[node] || adjN == heavy[node]) continue;
    dfs2(adjN, adjN, adj);
  }
}

void init_hld(int root, int n, vector<int> adj[]){
  parent.assign(n, -1);
  heavy.assign(n, -1);
  sz.assign(n, 1);
  depth.assign(n, 0);
  pos.assign(n, 0);
  head.assign(n, 0);
  t = -1;
  dfs1(root, -1, 0, adj);
  dfs2(root, root, adj);
}

// Procesa los rangos [pos[head[u]], pos[u]] en el SegTree
// Ejemplo para queries en camino u <-> v (suma, max, etc.):
int query_path(int u, int v){
  int res = 0; // elemento neutro de la operacion
  while(head[u] != head[v]){
    if(depth[head[u]] < depth[head[v]]) swap(u, v);
    // res = combine(res, query_segtree(pos[head[u]], pos[u]));
    u = parent[head[u]];
  }
  if(depth[u] > depth[v]) swap(u, v);
  // res = combine(res, query_segtree(pos[u], pos[v]));
  return res;
}
```

3.3 Centroid Decomposition

```
// TC: O(N log N) descomposicion completa
// usa: D&C en arboles; ejemplo: contar caminos de longitud exacta k
#include <bits/stdc++.h>
using namespace std;

vector<int> sz;
```

```cpp
vector<bool> removed;
vector<int> cnt; // cnt[d] = cantidad de nodos a distancia d
int n, k;
long long ans = 0;

int dfs_size(int node, int par, vector<int> adj[]){
  sz[node] = 1;
  for(auto adjN : adj[node]) if(adjN != par && !removed[adjN]){
    dfs_size(adjN, node, adj);
    sz[node] += sz[adjN];
  }
  return sz[node];
}

int get_centroid(int node, int par, int total, vector<int> adj[]){
  for(auto adjN : adj[node]){
    if(adjN != par && !removed[adjN] && sz[adjN] > total / 2){
      return get_centroid(adjN, node, total, adj);
    }
  }
  return node;
}

void dfs_depth(int node, int par, int d, vector<int> &depth, vector<int>
    adj[]){
  if(d > k) return;
  depth.push_back(d);
  for(auto adjN : adj[node]) if(adjN != par && !removed[adjN]){
    dfs_depth(adjN, node, d + 1, depth, adj);
  }
}

// Cuenta caminos de longitud k que pasan por el centroide c
void solve_centroid(int c, vector<int> adj[]){
  cnt[0] = 1;
  int mx = 0;
  vector<int> depth;
  for(auto adjN : adj[c]) if(!removed[adjN]){
    depth.clear();
    dfs_depth(adjN, c, 1, depth, adj);
    for(auto &d : depth) if(k >= d) ans += cnt[k - d];
    for(auto &d : depth) { cnt[d]++; mx = max(mx, d); }
  }
  // Limpiar cnt solo hasta la maxima profundidad vista (O(mx))
  for(int i = 0; i <= mx; i++) cnt[i] = 0;
}

void decompose(int node, vector<int> adj[]){
  int total = dfs_size(node, -1, adj);
  int c = get_centroid(node, -1, total, adj);

  solve_centroid(c, adj);

  removed[c] = true;
  for(auto adjN : adj[c]) if(!removed[adjN]){
    decompose(adjN, adj);
  }
}

int main(){
  cin >> n >> k;
  vector<int> adj[n];
  for(int i = 0; i < n - 1; i++){
    int u, v; cin >> u >> v; u--; v--;
    adj[u].push_back(v);
    adj[v].push_back(u);
  }
  sz.resize(n);
  removed.resize(n, false);
  cnt.resize(n + 1, 0);
  decompose(0, adj);
  cout << ans << "\n";
}
```

3.4 Euler Tour Technique (2 Variantes)

```cpp
// TC: O(N) aplanar el arbol, O(log N) por query con SegTree / BIT
// Aplana el arbol para convertir problemas de arboles en consultas de
    rangos
#include <bits/stdc++.h>
using namespace std;

// ============================================================
// VARIANTE 1: Subtree Queries (Subarbol -> Rango contiguo)
// El subarbol de u corresponde al rango [tin[u], tout[u]] (tamano N)
// ============================================================
vector<int> tin, tout;
int t = -1;

void dfs_subtree(int node, int par, vector<int> adj[]){
  tin[node] = ++t;
  for(auto adjN : adj[node]) if(adjN != par){
    dfs_subtree(adjN, node, adj);
  }
  tout[node] = t; // al salir, 't' actual cubre todo el subarbol
}
// En el SegTree/BIT de tamano N:
//   pos tin[u] guarda el valor a[u]
// Query subarbol de u: query_segtree(tin[u], tout[u])
// Update valor de u:   update_segtree(tin[u], val)

// ============================================================
// VARIANTE 2: Path Queries (Camino Raiz -> Nodo u con + / -)
// tin tiene +val[u] y tout tiene -val[u] (tamano 2N)
// Camino de raiz a u = suma en rango [tin[root], tin[u]]
// ============================================================
void dfs_path(int node, int par, vector<int> adj[]){
  tin[node] = ++t;
  for(auto adjN : adj[node]) if(adjN != par){
    dfs_path(adjN, node, adj);
  }
```

```
  tout[node] = ++t; // tin y tout reciben tiempos distintos
}
// En el SegTree/BIT de tamano 2N:
//   pos tin[u]  guarda +a[u]
//   pos tout[u] guarda -a[u]
// Query camino raiz a u: query_segtree(tin[0], tin[u])
// Update valor de u:     sumar delta en tin[u], restar delta en tout[u]
```

3.7 Otras

3.1 Mo con updates

```
// TC: O(N^{5/3}) per query
// usa: queries offline con updates mezclados
#include <bits/stdc++.h>
using namespace std;

struct MoWithUpdates{
  struct Query{
    int l, r, t, idx;
  };
  struct Update{
    int pos, val;
  };
  int n, q;
  vector<Query> queries;
  vector<Update> updates;
  vector<int> a;
  long long curAns = 0;
  vector<long long> ans;

  void addPos(int x){
    // update curAns given that position x is added to current range
  }
  void removePos(int x){
    // update curAns given that position x is removed from current range
  }
  void applyUpdate(const Update& u){
    // apply update; if u.pos is in [curL, curR], update state
  }
  void revertUpdate(const Update& u){
    // revert update; mirror of applyUpdate
  }
  void solve(){
    int blockSize = pow(n, 2.0/3.0);
    sort(queries.begin(), queries.end(), [&](const Query& A, const Query&
     B){
      int bA = A.l / blockSize, bB = B.l / blockSize;
      if(bA != bB) return bA < bB;
      int bAr = A.r / blockSize, bBr = B.r / blockSize;
      if(bAr != bBr) return bAr < bBr;
      return A.t < B.t;
    });
    ans.assign(queries.size(), 0);
    int curL = 0, curR = -1, curT = -1;
    for(const auto& qry : queries){
      while(curL > qry.l) addPos(--curL);
      while(curR < qry.r) addPos(++curR);
      while(curL < qry.l) removePos(curL++);
      while(curR > qry.r) removePos(curR--);
      while(curT < qry.t) applyUpdate(updates[++curT]);
      while(curT > qry.t) revertUpdate(updates[curT--]);
      ans[qry.idx] = curAns;
    }
  }
};

int main(){
  // skeleton: addPos/removePos/applyUpdate/revertUpdate must be filled
  // per problem; see standard Mo's algorithm tutorial.
  MoWithUpdates mwu;
  // mwu.addPos, mwu.removePos, mwu.applyUpdate, mwu.revertUpdate need
    impl
  // mwu.solve();
  return 0;
}
```

3.2 sqrt-decomposition generico

```
// TC: O(sqrt(N)) per operation
// usa: cuando quieres una EDD simple y lazy
#include <bits/stdc++.h>
using namespace std;

template<class T>
struct SqrtDecomp{
  int n, B;
  vector<T> a, bucket;
  SqrtDecomp(const vector<T>& v): a(v){
    n = v.size();
    B = (int)sqrt(n) + 1;
    bucket.assign(B, 0);
    for(int i=0;i<n;i++) bucket[i / B] += v[i];
  }
  void pointUpdate(int p, T val){
    bucket[p / B] += val - a[p];
    a[p] = val;
  }
  // range sum query
  T rangeSum(int l, int r){
    T res = 0;
    int bl = l / B, br = r / B;
    if(bl == br){
      for(int i=l;i<=r;i++) res += a[i];
    }else{
      for(int i=l;i<(bl+1)*B;i++) res += a[i];
      for(int i=bl+1;i<br;i++) res += bucket[i];
      for(int i=br*B;i<=r;i++) res += a[i];
    }
```

```
    return res;
  }
};

int main(){
  SqrtDecomp<int> sd({1, 2, 3, 4, 5, 6, 7, 8, 9, 10});
  cout << sd.rangeSum(2, 7) << "\n";  // 3+4+5+6+7+8 = 33
  sd.pointUpdate(4, 100);
  cout << sd.rangeSum(2, 7) << "\n";  // 128
  return 0;
}
```

4 Strings

4.1 Hashing

4.1 Rolling Hash (doble modulo)

```
// TC: precompute O(N), getHash O(1)  (N = |s|)
// usar: comparar substrings, palindromos, longest common substring,
//       contar substrings distintos, hashing de paths en trees, etc.
// cuidado: doble hash reduce colisiones a casi cero pero no las elimina
struct pair_hash {
  size_t operator()(const pair<int,int>& p) const {
    return ((uint64_t)p.first << 32) ^ (uint64_t)p.second;
  }
};
struct H{
  vector<pair<int,int>> h,p;
  const int MOD1=998244353;
  const int MOD2=1e9+7;
  const int A=31;
  string s;
  H(string &a){
    s=a;
    int n=s.size();
    h.resize(n+1); p.resize(n+1,{1,1});
    for(int i=1 ; i<=n ; i++){
      h[i].F=((h[i-1].F*A)%MOD1 + s[i-1])%MOD1;
      h[i].S=((h[i-1].S*A)%MOD2 + s[i-1])%MOD2;
      p[i].F=(p[i-1].F*A)%MOD1;
      p[i].S=(p[i-1].S*A)%MOD2;
    }
  }
  pair<int,int> getHash(int l, int r){
    pair<int,int> ans;
    ans.F=(h[r].F-(h[l-1].F*p[r-l+1].F)%MOD1)%MOD1;
    ans.S=(h[r].S-(h[l-1].S*p[r-l+1].S)%MOD2)%MOD2;
    ans.F=(ans.F+MOD1)%MOD1;
    ans.S=(ans.S+MOD2)%MOD2;
    return ans;
  }
};
```

```
// unordered_set<pair<int,int>, pair_hash> st;

int main(){
  string s = "banana";
  H h(s);
  // comparar substrings en O(1) por hash (1-indexed, inclusivo)
  auto a = h.getHash(1, 3); // "ban"
  auto b = h.getHash(4, 6); // "ana"
  cout << (a == b ? "iguales" : "distintos") << "\n";
  return 0;
}
```

4.2 Patrones

4.1 KMP + LPS

```
// TC: O(N + M)  (N = |s|, M = |p|)
vector<int> create_lps(string &needle){
  int n=(int)needle.size();
  vector<int> lps(n); // longest prefix suffix
  int i=1 , prevLPS=0;
  while(i < n){
    if(needle[i] == needle[prevLPS]){
      lps[i]=prevLPS+1;
      i++; prevLPS++;
    }else if(prevLPS == 0){
      lps[i]=0; i++;
    }else{
      prevLPS=lps[prevLPS-1];
    }
  }
  return lps;
}

void solve(){
  string s,p; cin >> s >> p;

  int sz_text=(int)s.size() , sz_pattern=(int)p.size();
  vector<int> lps=create_lps(p);
  int cnt=0;
  int i=0; // ptr text
  int j=0; // ptr pattern
  while(i<sz_text){
    if(s[i] == p[j]){
      i++; j++;
    }else if(j == 0) i++;
    else j=lps[j-1];
    if(j == sz_pattern){
      cnt++;
      j=lps[j-1];
    }
  }
  cout<< cnt <<"\n";
}
```

4.2 Z-algorithm

```cpp
// TC: O(N) build
// usa: encontrar todas las ocurrencias de un patron en un texto concatenado
#include <bits/stdc++.h>
using namespace std;

vector<int> zFunction(const string& s){
  int n = s.size();
  vector<int> z(n, 0);
  int l = 0, r = 0;
  for(int i=1;i<n;i++){
    if(i <= r) z[i] = min(r - i + 1, z[i - l]);
    while(i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if(i + z[i] - 1 > r){ l = i; r = i + z[i] - 1; }
  }
  return z;
}

int main(){
  string s = "abacaba";
  int n = s.size();
  auto z = zFunction(s);
  for(int x : z) cout << x << " ";  // 7 0 3 0 1 0 1
  cout << "\n";

  // (1) LCP entre s y cualquier sufijo s[i..]: max(z[i+1..n-1])
  int best = 0;
  for(int i = 1; i < n; i++) best = max(best, z[i]);
  cout << "max LCP(s, s[i..]) = " << best << "\n";  // 3 ("aba")

  // (2) ocurrencias de cada prefijo de largo k como substring (diff array)
  vector<int> prefCnt(n + 2, 0);
  for(int i = 1; i < n; i++){ prefCnt[1]++; prefCnt[z[i] + 1]--; }
  for(int k = 1; k <= n; k++) prefCnt[k] += prefCnt[k - 1];
  for(int k = 1; k <= n; k++)
    cout << "prefijo largo " << k << " aparece " << prefCnt[k] << " veces\n";

  // (3) palindromos impares: z sobre s + "#" + reverse(s)
  //     palindromo centrado en c tiene largo zp[n+1+(n-1-2*c)]
  string rs(s.rbegin(), s.rend());
  auto zp = zFunction(s + "#" + rs);
  cout << "palindromos impares por centro:\n";
  for(int c = 0; c < n; c++){
    int idx = n + 1 + (n - 1 - 2*c);
    if(idx >= 0 && idx < (int)zp.size())
      cout << "  centro " << c << " -> largo " << zp[idx] << "\n";
  }

  // (4) pattern matching: z de (patron + '#' + texto)
  string p = "aba", t = "abacabacaba";
  auto z2 = zFunction(p + "#" + t);
  for(int i = 0; i < (int)z2.size(); i++)
    if(z2[i] == (int)p.size()) cout << "match at " << i - p.size() - 1 << "\n";
  return 0;
}
```

4.3 Rabin-Karp (single hash)

```cpp
// TC: O(N + M) expected
// usa: encontrar patron con hash rodante; mejor con doble hash
#include <bits/stdc++.h>
using namespace std;

const long long BASE = 91138233LL;
const long long MOD = 1000000007LL;

// returns all starting positions of pattern in text
vector<int> rabinKarp(const string& text, const string& pattern){
  int n = text.size(), m = pattern.size();
  if(m > n) return {};
  long long h = 1;
  for(int i=0;i<m-1;i++) h = h * BASE % MOD;
  long long pHash = 0, tHash = 0;
  for(int i=0;i<m;i++){
    pHash = (pHash * BASE + pattern[i]) % MOD;
    tHash = (tHash * BASE + text[i]) % MOD;
  }
  vector<int> occ;
  for(int i=0;i<=n-m;i++){
    if(pHash == tHash){
      bool match = true;
      for(int j=0;j<m;j++) if(text[i+j] != pattern[j]){ match = false; break; }
      if(match) occ.push_back(i);
    }
    if(i < n - m){
      tHash = ((tHash - text[i] * h % MOD + MOD) * BASE + text[i+m]) % MOD;
    }
  }
  return occ;
}

int main(){
  auto occ = rabinKarp("hello world", "world");
  for(int pos : occ) cout << "found at " << pos << "\n";
  return 0;
}
```

4.4 Aho-Corasick (basic)

```cpp
// TC: O(sum |patterns| + |text|)
// usa: encontrar todos los patterns en un texto
```

```cpp
#include <bits/stdc++.h>
using namespace std;

struct AhoCorasick{
  struct Node{
    int next[26];
    int link;
    vector<int> out; // indices of patterns ending here
    Node(){
      fill(begin(next), end(next), -1);
      link = -1;
    }
  };
  vector<Node> t;
  AhoCorasick(){ t.push_back(Node()); }
  void addString(const string& s, int id){
    int v = 0;
    for(char c : s){
      int k = c - 'a';
      if(t[v].next[k] == -1){
        t[v].next[k] = (int)t.size();
        t.push_back(Node());
      }
      v = t[v].next[k];
    }
    t[v].out.push_back(id);
  }
  void build(){
    queue<int> q;
    t[0].link = 0;
    for(int k=0;k<26;k++){
      if(t[0].next[k] != -1){
        t[t[0].next[k]].link = 0;
        q.push(t[0].next[k]);
      }else{
        t[0].next[k] = 0;
      }
    }
    while(!q.empty()){
      int v = q.front(); q.pop();
      for(int k=0;k<26;k++){
        int u = t[v].next[k];
        if(u != -1){
          t[u].link = t[t[v].link].next[k];
          // propagate outputs from link
          for(int pid : t[t[u].link].out) t[u].out.push_back(pid);
          q.push(u);
        }else{
          t[v].next[k] = t[t[v].link].next[k];
        }
      }
    }
  }
  // returns count of occurrences per pattern id
  vector<int> countOccurrences(const string& text, int numPatterns){
    vector<int> cnt(numPatterns, 0);
    int v = 0;
    for(char c : text){
      v = t[v].next[c - 'a'];
      for(int pid : t[v].out) cnt[pid]++;
    }
    return cnt;
  }
};

int main(){
  AhoCorasick ac;
  ac.addString("he", 0);
  ac.addString("she", 1);
  ac.addString("his", 2);
  ac.build();
  auto cnt = ac.countOccurrences("ushers", 3);
  for(int c : cnt) cout << c << " ";
  cout << "\n";
  return 0;
}
```

4.5 Aho-Corasick DP for forbidden patterns

```cpp
// TC: O(L * states * alphabet) per DP step
// usa: contar strings de longitud L evitando ciertos patterns
#include <bits/stdc++.h>
using namespace std;

const long long AC_MOD = 998244353LL;

// assumes AhoCorasick is built; transition automaton is already computed.
long long countAvoiding(const AhoCorasick& ac, int L){
  int S = (int)ac.t.size();
  vector<long long> dp(S, 0), ndp(S, 0);
  if(ac.t[0].out.empty()) dp[0] = 1;
  for(int i=0;i<L;i++){
    fill(ndp.begin(), ndp.end(), 0);
    for(int v=0;v<S;v++){
      if(dp[v] == 0) continue;
      for(int k=0;k<26;k++){
        int u = ac.t[v].next[k];
        if(ac.t[u].out.empty()){
          ndp[u] = (ndp[u] + dp[v]) % AC_MOD;
        }
      }
    }
    dp = ndp;
  }
  long long total = 0;
  for(long long x : dp) total = (total + x) % AC_MOD;
  return total;
}

int main(){
```

```
  // count strings of length L over [a-z] avoiding all patterns in ac
  AhoCorasick ac;
  ac.addString("ab", 0);
  ac.build();
  cout << countAvoiding(ac, 3) << "\n";  // 26^3 - count with "ab"
  return 0;
}
```

4.3 Estructuras avanzadas

4.1 Suffix Array

```
// TC: O(N log N) build, O(N) memoria
// usa: comparar substrings en O(1) (con LCP), contar substrings distintos,
//      longest repeated substring, longest common substring entre 2 strings,
//      pattern matching O(|p| log N) con binary search sobre SA,
//      encontrar el k-esimo substring lex, concatenar strings y ordenar sufijos
// sa.p[i]  = posicion en s del i-esimo sufijo en orden lexicografico
// sa.c[i]  = clase de equivalencia (rank) del sufijo que empieza en i
struct SuffArr{
  vector<int> p,c;
  int n;
  string s;
  SuffArr(string &a){
    s=a;
    n=s.size();
    p.resize(n) ; c.resize(n);
    build();
  }
  void count_sort(vector<int> &p, vector<int> &c){
    int n=p.size();
    vector<int> cnt(n);
    for(auto x : c ) cnt[x]++;
    vector<int> p_new(n) , pos(n);
    pos[0]=0;
    for(int i=1 ; i<n ; i++){
      pos[i]=pos[i-1]+cnt[i-1];
    }
    for(auto x : p){
      int i=c[x];
      p_new[pos[i]]=x;
      pos[i]++;
    }
    p=p_new;
  }
  void build(){
    {
      vector<pair<int,int>> tmp(n);
      for(int i=0 ; i<n ; i++){
        tmp[i]={s[i],i};
      }
      sort(all(tmp));
      for(int i=0 ; i<n ; i++){
        p[i]=tmp[i].S;
      }
      c[p[0]]=0;
      for(int i=1 ; i<n ; i++){
        if(tmp[i].F == tmp[i-1].F){
          c[p[i]]=c[p[i-1]];
        }else{
          c[p[i]]=c[p[i-1]]+1;
        }
      }
    }
    int k=0;
    while((1<<k) < n){
      // k -> k+1
      for(int i=0; i<n ; i++){
        p[i]=(p[i]-(1<<k) + n)%n;
      }

      count_sort(p,c);

      vector<int> c_new(n);
      c_new[p[0]]=0;
      for(int i=1 ; i<n ; i++){
        pair<int,int> prev={c[p[i-1]],c[(p[i-1]+(1<<k))%n]};
        pair<int,int> now={c[p[i]],c[(p[i]+(1<<k))%n]};
        if(now == prev){
          c_new[p[i]]=c_new[p[i-1]];
        }else{
          c_new[p[i]]=c_new[p[i-1]]+1;
        }
      }
      c=c_new;
      k++;
    }
  }
};

int main(){
  string s = "banana";
  SuffArr sa(s);

  // (1) imprimir SA y sufijos en orden lex
  cout << "SA: ";
  for(int i = 0; i < sa.n; i++) cout << sa.p[i] << " ";  // 5 3 1 0 4 2
  cout << "\nsufijos ordenados:\n";
  for(int i = 0; i < sa.n; i++)
    cout << "  " << sa.s.substr(sa.p[i]) << "\n";

  // (2) contar ocurrencias de un patron en O(|p| log N) con binary search
  string p = "ana";
  auto cmpLess = [&](int suf){
    return sa.s.compare(suf, (int)p.size(), p) < 0;
  };
```

```
  int lo = 0, hi = sa.n;
  while(lo < hi){ int m = (lo+hi)/2; if(cmpLess(sa.p[m])) lo = m+1; else
    hi = m; }
  int first = lo;
  // upper bound: lower_bound(p + '{') (cualquier char > los del alfabeto
    )
  string pNext = p + "{";
  auto cmpLess2 = [&](int suf){
    return sa.s.compare(suf, (int)pNext.size(), pNext) < 0;
  };
  lo = first; hi = sa.n;
  while(lo < hi){ int m = (lo+hi)/2; if(cmpLess2(sa.p[m])) lo = m+1; else
     hi = m; }
  int last = lo;
  cout << "ocurrencias de \"" << p << "\": " << (last - first) << "\n";
    // 2

  // (3) longest repeated substring: max(lcp[i]) sobre todo el SA
  //     (combinado con el LCP de Kasai - ver archivo lcp.tex)
  return 0;
}
```

4.2 LCP Array (Kasai)

```
// TC: O(N)
// usa: longest common prefix entre sufijos adyacentes (necesita SA
    precomputado)
#include <bits/stdc++.h>
using namespace std;

vector<int> buildLCP(const string& s, const vector<int>& sa){
  int n = s.size();
  vector<int> rank_(n);
  for(int i=0;i<n;i++) rank_[sa[i]] = i;
  vector<int> lcp(max(0, n - 1), 0);
  int h = 0;
  for(int i=0;i<n;i++){
    if(rank_[i] > 0){
      int j = sa[rank_[i] - 1];
      while(i + h < n && j + h < n && s[i+h] == s[j+h]) h++;
      lcp[rank_[i] - 1] = h;
      if(h > 0) h--;
    }
  }
  return lcp;
}

int main(){
  string s = "banana";
  vector<int> sa = {5,3,1,0,4,2};  // assume precomputed SA
  auto lcp = buildLCP(s, sa);
  for(int x : lcp) cout << x << " ";
  cout << "\n";
  return 0;
}
```

4.3 Suffix Automaton

```
// TC: O(N) build
// usa: contar substrings distintas, longest common substring, etc.
#include <bits/stdc++.h>
using namespace std;

struct SAM{
  struct State{
    int next[26];
    int link, len;
    long long occ; // number of end positions (for counting)
    State(){
      fill(begin(next), end(next), -1);
      link = -1;
      len = 0;
      occ = 0;
    }
  };
  vector<State> st;
  int last;
  SAM(int maxLen){
    st.reserve(2 * maxLen);
    st.push_back(State());
    st[0].link = -1;
    last = 0;
  }
  void extend(char c){
    int k = c - 'a';
    int cur = (int)st.size();
    st.push_back(State());
    st[cur].len = st[last].len + 1;
    st[cur].occ = 1;
    int p = last;
    while(p != -1 && st[p].next[k] == -1){
      st[p].next[k] = cur;
      p = st[p].link;
    }
    if(p == -1){
      st[cur].link = 0;
    }else{
      int q = st[p].next[k];
      if(st[p].len + 1 == st[q].len){
        st[cur].link = q;
      }else{
        int clone = (int)st.size();
        st.push_back(st[q]);
        st[clone].len = st[p].len + 1;
        st[clone].occ = 0; // clone doesn't count as new end
        while(p != -1 && st[p].next[k] == q){
          st[p].next[k] = clone;
          p = st[p].link;
        }
```

```
        st[q].link = st[cur].link = clone;
      }
    }
    last = cur;
  }
  void build(const string& s){
    for(char c : s) extend(c);
    // propagate occ counts in reverse order of len
    vector<int> order(st.size());
    iota(order.begin(), order.end(), 0);
    sort(order.begin(), order.end(), [&](int a, int b){ return st[a].len
    > st[b].len; });
    for(int v : order){
      if(st[v].link != -1) st[st[v].link].occ += st[v].occ;
    }
  }
  // returns number of distinct substrings
  long long countDistinct(){
    long long ans = 0;
    for(size_t i=1;i<st.size();i++){
      ans += st[i].len - st[st[i].link].len;
    }
    return ans;
  }
  // longest common substring of s with the SAM's string
  int longestCommonSubstring(const string& s){
    int v = 0, l = 0, best = 0;
    for(char c : s){
      int k = c - 'a';
      while(v != 0 && st[v].next[k] == -1){
        v = st[v].link;
        l = st[v].len;
      }
      if(st[v].next[k] != -1){
        v = st[v].next[k];
        l++;
      }
      best = max(best, l);
    }
    return best;
  }
};

int main(){
  SAM sam(100);
  sam.build("ababa");
  cout << "distinct substrings: " << sam.countDistinct() << "\n";
  cout << "LCS with 'baba': " << sam.longestCommonSubstring("baba") << "\
    n";
  return 0;
}
```

4.4 Palindromos

4.1 Manacher

```
// TC: O(N) para construir odd/even; cada query O(1) con los helpers
// usa: longest palindromic substring, contar palindromos, queries O(1) "
    es s[l..r] palindromo?"
#include <bits/stdc++.h>
using namespace std;

// Convencion de ESTE codigo (distinta a la "clasica de radio"):
//   odd[i]  = k (k >= 1): el palindromo IMPAR mas largo centrado en i
//             es s[i - k + 1 .. i + k - 1] (longitud 2k - 1).
//             -> cuenta ademas cuantos palindromos impares hay centrados
     en i:
//                k palindromos (los de longitud 1, 3, 5, ..., 2k-1).
//   even[i] = k (k >= 0): el palindromo PAR mas largo "centrado entre i
    -1 e i"
//             es s[i - k .. i + k - 1] (longitud 2k). Si k == 0 no hay
    palindromo par ahi.
//             -> cuenta cuantos palindromos pares hay "centrados en i":
    k.
//      Equivalencias utiles:
//        palindromo impar centrado en i de longitud L impar -> odd[i] >=
    (L+1)/2
//        palindromo par   "centrado entre i-1 e i" de long 2k -> even[i]
    >= k
pair<vector<int>, vector<int>> manacher(const string& s){
  int n = (int)s.size();
  vector<int> odd(n, 0);
  int l = 0, r = -1;
  for(int i = 0; i < n; i++){
    int k = (i > r) ? 1 : min(odd[l + r - i], r - i + 1);
    while(i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
    odd[i] = k;
    if(i + k - 1 > r){ l = i - k + 1; r = i + k - 1; }
  }
  vector<int> even(n, 0);
  l = 0; r = -1;
  for(int i = 0; i < n; i++){
    int k = (i > r) ? 0 : min(even[l + r - i + 1], r - i + 1);
    while(i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) k++;
    even[i] = k;
    if(i + k - 1 > r){ l = i - k; r = i + k - 1; }
  }
  return {odd, even};
}

// LONGEST PALINDROMIC SUBSTRING
string longestPalindrome(const string& s,
             const vector<int>& odd,
             const vector<int>& even){
  int n = (int)s.size(), bestL = 0, bestLen = 1;
  for(int i = 0; i < n; i++){
    // impar centrado en i: longitud 2*odd[i] - 1
    int len = 2 * odd[i] - 1;
```

```
    if(len > bestLen){ bestLen = len; bestL = i - odd[i] + 1; }
    // par centrado entre i-1 e i: longitud 2*even[i]
    len = 2 * even[i];
    if(len > bestLen){ bestLen = len; bestL = i - even[i]; }
  }
  return s.substr(bestL, bestLen);
}

// COUNT PALINDROMES
long long countPalindromes(const vector<int>& odd,
               const vector<int>& even){
  long long total = 0;
  for(int x : odd)  total += x;
  for(int x : even) total += x;
  return total;
}

// PALINDROMIC SUBSTRING QUERIES
//   Devuelve true sii s[l..r] (0-indexed, inclusivo) es palindromo.
bool isPalindrome(int l, int r,
          const vector<int>& odd,
          const vector<int>& even){
  int len = r - l + 1;
  if(len <= 1) return true;
  if(len % 2 == 1){
    int c = (l + r) / 2;          // centro del palindromo impar
    return odd[c] >= (len + 1) / 2;
  } else {
    int c = (l + r + 1) / 2;      // borde derecho del centro par
    return even[c] >= len / 2;
  }
}

int main(){
  string s = "abacaba";
  auto [odd, even] = manacher(s);

  // imprime los radios
  for(int x : odd)  cout << x << " ";  cout << "\n";
  for(int x : even) cout << x << " ";  cout << "\n";

  // 1) longest palindromic substring -> "abacaba"
  cout << "longest = " << longestPalindrome(s, odd, even) << "\n";

  // 2) count palindromes -> 12 para "abacaba"
  cout << "count   = " << countPalindromes(odd, even) << "\n";

  // 3) queries O(1) "es s[l..r] palindromo?"
  //    (0-indexed, inclusivo)
  auto Q = [&](int l, int r){
    return isPalindrome(l, r, odd, even);
  };
  cout << "s[0..6] aba...? " << Q(0, 6) << "\n";   // 1 -> "abacaba"
  cout << "s[0..5] aba...? " << Q(0, 5) << "\n";   // 0
  cout << "s[0..2] aba?   " << Q(0, 2) << "\n";    // 1 -> "aba"
  cout << "s[1..3] bac?   " << Q(1, 3) << "\n";    // 0
  cout << "s[2..4] aca?   " << Q(2, 4) << "\n";    // 1 -> "aca"
  cout << "s[0..1] ab?    " << Q(0, 1) << "\n";    // 0
  cout << "s[0..0] a?     " << Q(0, 0) << "\n";    // 1
  return 0;
}
```

4.5 Otros

4.1 Lyndon factorization (Duval)

```
// TC: O(N)
// usa: descomponer un string en factores de Lyndon
#include <bits/stdc++.h>
using namespace std;

vector<int> lyndonFactorization(const string& s){
  // returns ending positions of each Lyndon factor (0-indexed)
  int n = s.size();
  vector<int> cuts;
  int i = 0;
  while(i < n){
    int j = i, k = i + 1;
    while(k < n && s[j] <= s[k]){
      if(s[j] < s[k]) j = i;
      else j++;
      k++;
    }
    int len = k - j;
    while(i <= j){
      cuts.push_back(i + len - 1);
      i += len;
    }
  }
  return cuts;
}

int main(){
  string s = "ababa";
  auto cuts = lyndonFactorization(s);
  for(int c : cuts) cout << c << " ";  // 0 2 4
  cout << "\n";
  return 0;
}
```

5 Grafos

5.1 Caminos minimos

5.1 Dijkstra

```cpp
// TC: O((N + M) log M)
// usa: camino minimo desde s con pesos no negativos; inserta duplicados y descarta entradas viejas al sacar
vector<int> dijkstraPQ(int n, vector<vector<int>> adj[], int s){
  priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
  vector<int> dist(n,1e18);
  dist[s]=0;
  pq.push({0,s});
  while(!pq.empty()){
    auto [dis,node]=pq.top(); pq.pop();
    if(dis>dist[node]) continue;
    for(auto [adjN,adjW] : adj[node]){
      if(dis+adjW < dist[adjN]){
        dist[adjN]=dis+adjW;
        pq.push({dist[adjN],adjN});
      }
    }
  }
  return dist;
}
```

5.2 Bellman-Ford

```cpp
// TC: O(N * M)
// 1. Camino minimo desde s y deteccion de ciclo negativo alcanzable
vector<long long> bellman_ford(int n, vector<array<int,3>>& edges, int s, bool& has_cycle){
  const long long INF = 1e18;
  vector<long long> dist(n, INF);
  dist[s] = 0;
  for(int i = 0; i < n - 1; i++){
    for(auto [u, v, w] : edges){
      if(dist[u] < INF && dist[u] + w < dist[v]){
        dist[v] = max(-INF, dist[u] + w);
      }
    }
  }
  has_cycle = false;
  for(auto [u, v, w] : edges){
    if(dist[u] < INF && dist[u] + w < dist[v]){
      has_cycle = true; break;
    }
  }
  return dist;
}

// 2. Reconstruir ciclo negativo (ej. CSES - Cycle Finding)
// Retorna vertices del ciclo {v1, v2, ..., v1} en orden dirigido, o vacio si no hay.
// s = -1: detecta CUALQUIER ciclo en el grafo (dist=0 para todo nodo).
// s >= 0: detecta ciclo negativo alcanzable desde s.
vector<int> get_negative_cycle(int n, vector<array<int,3>>& edges, int s = -1){
  const long long INF = 1e18;
  vector<long long> dist(n, s == -1 ? 0 : INF);
  vector<int> parent(n, -1);
  if(s != -1) dist[s] = 0;
  int last_relaxed = -1;

  // N relajaciones: si en la vuelta N se relaja algo, hay ciclo negativo
  for(int rep = 0; rep < n; rep++){
    last_relaxed = -1;
    for(auto [u, v, w] : edges){
      if(dist[u] < INF && dist[u] + w < dist[v]){
        dist[v] = max(-INF, dist[u] + w);
        parent[v] = u;
        last_relaxed = v;
      }
    }
  }
  if(last_relaxed == -1) return {}; // No hay ciclo negativo

  // Retroceder N veces: el ultimo vertice relajado podria estar fuera del ciclo
  // (en una rama alcanzable). Al retroceder N veces por parent se entra seguro al ciclo.
  for(int i = 0; i < n; i++) last_relaxed = parent[last_relaxed];

  vector<int> cycle;
  for(int cur = last_relaxed; ; cur = parent[cur]){
    cycle.push_back(cur);
    if(cur == last_relaxed && cycle.size() > 1) break;
  }
  reverse(cycle.begin(), cycle.end());
  return cycle;
}
```

5.3 Floyd-Warshall

```cpp
// TC: O(N^3)
// usa: camino minimo entre todo par de nodos; permite pesos negativos; detecta ciclo negativo si ans[i][i] < 0
vector<vector<int>> floyd(int n, vector<vector<int>> adj[]){
  vector<vector<int>> ans(n,vector<int>(n,1e18));
  for(int i=0 ; i<n ; i++) ans[i][i]=0;
  for(int i=0 ; i<n ; i++){
    for(auto x : adj[i]){
      int adjN=x[0] , adjW=x[1];
      ans[i][adjN]=adjW;
      ans[adjN][i]=adjW;
    }
  }

  for(int via=0 ; via<n ; via++){
    for(int i=0 ; i<n ; i++){
      for(int j=0 ; j<n ; j++){
        ans[i][j]=min(ans[i][j],
        ans[i][via]+ans[via][j]);
```

```
      }
    }
  }

  // Detect negative cycle: if any ans[i][i] < 0 after the algorithm,
  // then there is a negative cycle reachable from i.
  for(int i=0 ; i<n ; i++){
    if(ans[i][i] < 0){
      // negative cycle reachable from source i
      }
    }
  return ans;
}
```

5.2 Componentes y cortes

5.1 Kosaraju SCC

```
// TC: O(N + M)
// usa: encuentra componentes fuertemente conexas (SCC) usando dos DFS:
    una por orden de salida y otra sobre el grafo transpuesto
void dfs(int node, vector<pair<int,int>> adj[], vector<bool> &vis,
stack<int> &st){
  vis[node]=1;
  for(auto [adjN,adjW] : adj[node]){
    if(!vis[adjN]){
      dfs(adjN , adj , vis , st);
    }
  }
  st.push(node);
}

void dfs2(int node, vector<pair<int,int>> adj[], vector<bool> &vis,
vector<int> &scc, int scc_number){
  vis[node]=1;
  scc[node]=scc_number;
  for(auto [adjN,adjW] : adj[node]){
    if(!vis[adjN]){
      dfs2(adjN , adj , vis , scc , scc_number);
    }
  }
}

void solve(){
  int n,m; cin >> n >> m;
  vector< pair<int,int> > adj[n];
  for(int i=0 ; i<m ; i++){
    int u,v,w; cin >> u >> v >> w;
    u--; v--;
    adj[u].emplace_back(v,w);
  }

  vector<bool> vis(n);
  stack<int> st;
  for(int i=0 ; i<n ; i++){
    if(!vis[i]){
      dfs(i , adj , vis , st);
    }
  }

  vector<pair<int,int>> adjT[n];
  for(int i=0 ; i<n ; i++){
    vis[i]=0;
    for(auto [adjN,adjW] : adj[i]){
      adjT[adjN].push_back({i,adjW});
    }
  }

  vector<int> scc(n);
  int scc_number=0;
  while(!st.empty()){
    int node=st.top();
    st.pop();
    if(!vis[node]){
      scc_number++;
      dfs2(node, adjT, vis, scc, scc_number);
    }
  }

  vector<pair<int,int>> adjCond[scc_number];
  for(int i=0 ; i<n ; i++){
    for(auto [adjN,adjW] : adj[i]){
      if(scc[i] != scc[adjN]){
        adjCond[scc[i]-1].emplace_back(scc[adjN]-1,adjW);
      }
    }
  }
}
```

5.2 Tarjan (puentes)

```
// TC: O(N + M)
// usa: encuentra puentes (aristas que al remover desconectan el grafo)
    usando tiempos de entrada y lowlink
void dfs(int node, int parent, vector<int> adj[], vector<int> &time,
vector<int> &low, vector<array<int,2>> &ans, int &t){
  time[node] = low[node] = t;
  t++;
  for(auto adjN : adj[node]){
    if(adjN == parent) continue;
    if(!time[adjN]){
      dfs(adjN, node, adj, time, low, ans, t);
      low[node]=min(low[node],low[adjN]);
      if(time[node] < low[adjN]){
        int mn=min(node,adjN) , mx=max(node,adjN);
        ans.push_back({mn,mx});
      }
    }else{
      low[node]=min(low[node],low[adjN]);
```

```
    }
  }
}

void solve(){
  int n,m; cin >> n >> m;
  vector<int> adj[n+1];
  for(int i=0 ; i<m ; i++){
    int u,v; cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);
  }

  vector<int> time(n+1) , low(n+1);
  vector<array<int,2>> ans;
  int t=1;
  dfs(1, -1, adj, time, low, ans, t);
}
```

5.3 Tarjan (puntos de articulacion)

```
// TC: O(N + M)
// usa: encuentra puntos de articulacion (nodos que al remover
    desconectan el grafo) usando tiempos de entrada y lowlink
void dfs(int node, int parent, vector<int> adj[], vector<int> &tin,
vector<int> &low, vector<bool> &artPoint, int &t){
  tin[node]=low[node]=++t;
  int child=0;
  for(auto adjN : adj[node]){
    if(adjN == parent) continue;
    if(!tin[adjN]){
      dfs(adjN, node, adj, tin, low, artPoint, t);
      low[node]=min(low[node],low[adjN]);
      if(low[adjN] >= tin[node] && parent != -1){
        artPoint[node]=true;
      }
      child++;
    }else{
      low[node]=min(low[node],tin[adjN]);
    }
  }
  if(child>1 && parent==-1) artPoint[node]=true;
}

void solve(){
  int n,m; cin >> n >> m;
  vector<int> adj[n+1];
  for(int i=0 ; i<m ; i++){
    int u,v; cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);
  }

  vector<int> tin(n+1) , low(n+1);
  vector<bool> artPoint(n+1);
  int t=0;
  for(int i=1 ; i<=n ; i++){
    if(tin[i] == 0){
      dfs(i , -1 , adj , tin , low , artPoint, t);
    }
  }
}
```

5.4 Tarjan SCC

```
// TC: O(N + M)
// usa: encontrar componentes fuertemente conexos
#include <bits/stdc++.h>
using namespace std;

struct TarjanSCC{
  int n, timer, numComps;
  vector<int> comp, low, disc, st;
  vector<bool> inStack;
  const vector<vector<int>>& adj;
  TarjanSCC(int n, const vector<vector<int>>& adj): n(n), adj(adj){
    comp.assign(n, -1);
    low.assign(n, 0);
    disc.assign(n, 0);
    inStack.assign(n, false);
    timer = 0;
    numComps = 0;
    for(int i=0;i<n;i++) if(disc[i] == 0) dfs(i);
  }
  void dfs(int node){
    disc[node] = low[node] = ++timer;
    st.push_back(node);
    inStack[node] = true;
    for(auto adjN : adj[node]){
      if(disc[adjN] == 0){
        dfs(adjN);
        low[node] = min(low[node], low[adjN]);
      }else if(inStack[adjN]){
        low[node] = min(low[node], disc[adjN]);
      }
    }
    if(low[node] == disc[node]){
      while(true){
        int cur = st.back(); st.pop_back();
        inStack[cur] = false;
        comp[cur] = numComps;
        if(cur == node) break;
      }
      numComps++;
    }
  }
};

int main(){
  vector<vector<int>> adj = {{1}, {2,3}, {0}, {4}, {}};
```

```
  TarjanSCC scc(5, adj);
  cout << "#SCCs = " << scc.numComps << "\n";  // 3
  return 0;
}
```

5.3 Matching

5.1 Kuhn (bipartite matching)

```
// TC: O(V * E) donde V = nodos lado izquierdo, E = aristas
// usa: matching maximo en grafo bipartito; usa DFS con vis para encontrar augmenting paths
// nota: para N, M <= 1e3 anda bien; para mas usar Hopcroft-Karp (O(sqrt(V) * E))
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<int> adj[1005];

bool try_match(int node, vector<int> adj[], vector<int>& matchR, vector<bool>& vis){
  for(auto adjN : adj[node]){
    if(vis[adjN]) continue;
    vis[adjN] = true;
    if(matchR[adjN] == -1 || try_match(matchR[adjN], adj, matchR, vis)){
      matchR[adjN] = node;
      return true;
    }
  }
  return false;
}

int main(){
  // entrada: n nodos izquierda, m nodos derecha, m aristas (u, v) con u en [1..n], v en [1..m]
  cin >> n >> m;
  for(int i=0 ; i<m ; i++){
    int u, v; cin >> u >> v;
    adj[u].push_back(v);
  }
  vector<int> matchR(m + 1, -1);
  int matching = 0;
  for(int node = 1; node <= n; node++){
    vector<bool> vis(m + 1, false);
    if(try_match(node, adj, matchR, vis)) matching++;
  }
  cout << matching << "\n";
  return 0;
}
```

5.2 Hopcroft-Karp

```
// TC: O(sqrt(V) * E) donde V = nodos lado izquierdo, E = aristas
// usa: matching maximo en grafo bipartito; usa BFS + DFS por capas para encontrar varios augmenting paths por fase
// nota: para N, M <= 1e5; para N, M <= 1e3 alcanza Kuhn (O(V*E))
struct HopcroftKarp {
  int n, m; // n->#left_nodes  m->#right_nodes
  vector<vector<int>> adj;
  vector<int> dist, matchU, matchV;

  HopcroftKarp(int n, int m): n(n), m(m) {
    adj.assign(n + 1, {});
    matchU.assign(n + 1, 0);
    matchV.assign(m + 1, 0);
    dist.assign(n + 1, 0);
  }

  void addEdge(int u, int v) {
    adj[u].push_back(v);
  }

  bool bfs() {
    queue<int> q;
    for (int u = 1; u <= n; u++) {
      if (matchU[u] == 0) {
        dist[u] = 0;
        q.push(u);
      } else {
        dist[u] = -1;
      }
    }
    bool found = false;
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (int v : adj[u]) {
        if (matchV[v] != 0 && dist[matchV[v]] == -1) {
          dist[matchV[v]] = dist[u] + 1;
          q.push(matchV[v]);
        }
        if (matchV[v] == 0) {
          found = true;
        }
      }
    }
    return found;
  }

  bool dfs(int u) {
    for (int v : adj[u]) {
      if (matchV[v] == 0 ||
      (dist[matchV[v]] == dist[u] + 1 && dfs(matchV[v]))) {
        matchU[u] = v;
        matchV[v] = u;
        return true;
      }
    }
    dist[u] = -1;
```

```
    return false;
  }

  int maxMatching() {
    int matching = 0;
    while (bfs()) {
      for (int u = 1; u <= n; u++) {
        if (matchU[u] == 0 && dfs(u)) {
          matching++;
        }
      }
    }
    return matching;
  }
};

int main(){
  // entrada: n nodos izquierda, m nodos derecha, k aristas (u, v) con u
    en [1..n], v en [1..m]
  int n, m, k;
  cin >> n >> m >> k;
  HopcroftKarp hk(n, m);
  for(int i=0;i<k;i++){
    int u, v; cin >> u >> v;
    hk.addEdge(u, v);
  }
  cout << hk.maxMatching() << "\n";
  return 0;
}
```

5.4 MST

5.1 Kruskal

```
// TC: O(M log M)
// usa: MST con aristas ordenadas por peso
#include <bits/stdc++.h>
using namespace std;

struct DSUk{
  vector<int> p, r;
  DSUk(int n): p(n), r(n, 0){ iota(p.begin(), p.end(), 0); }
  int find(int v){ return p[v] == v ? v : p[v] = find(p[v]); }
  bool unite(int a, int b){
    a = find(a); b = find(b);
    if(a == b) return false;
    if(r[a] < r[b]) swap(a, b);
    p[b] = a;
    if(r[a] == r[b]) r[a]++;
    return true;
  }
};

long long kruskal(int n, vector<array<int,3>>& edges){
  sort(edges.begin(), edges.end(), [](auto& A, auto& B){
    return A[2] < B[2];
  });
  DSUk dsu(n);
  long long total = 0;
  int cnt = 0;
  for(auto [u, v, w] : edges){
    if(dsu.unite(u, v)){
      total += w;
      if(++cnt == n - 1) break;
    }
  }
  return total;
}

int main(){
  vector<array<int,3>> edges = {{0,1,4},{1,2,1},{2,3,2},{0,3,5}};
  cout << "MST weight = " << kruskal(4, edges) << "\n";  // 7
  return 0;
}
```

5.2 Prim (priority queue)

```
// TC: O((N + M) log N)
// usa: MST alternativo, mejor en grafos densos
#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

long long prim(int n, const vector<vector<pair<int,int>>>& adj){
  vector<long long> dist(n, INF);
  vector<bool> vis(n, false);
  priority_queue<pair<long long,int>, vector<pair<long long,int>>,
    greater<pair<long long,int>>> pq;
  dist[0] = 0;
  pq.push({0, 0});
  long long total = 0;
  while(!pq.empty()){
    auto [dis, node] = pq.top(); pq.pop();
    if(vis[node]) continue;
    vis[node] = true;
    total += dis;
    for(auto [adjN, adjW] : adj[node]){
      if(!vis[adjN] && adjW < dist[adjN]){
        dist[adjN] = adjW;
        pq.push({adjW, adjN});
      }
    }
  }
  return total;
}

int main(){
  vector<vector<pair<int,int>>> adj(4);
```

```
  adj[0] = {{1,4},{3,5}}; adj[1] = {{0,4},{2,1},{3,2}};
  adj[2] = {{1,1},{3,2}}; adj[3] = {{0,5},{1,2},{2,2}};
  cout << "MST weight = " << prim(4, adj) << "\n";  // 7
  return 0;
}
```

5.5 Basicos

5.1 Topological sort (Kahn + DFS)

```
// TC: O(N + M)
// usa: orden lineal de un DAG
#include <bits/stdc++.h>
using namespace std;

vector<int> topoKahn(int n, const vector<vector<int>>& adj){
  vector<int> indegree(n, 0);
  for(int i=0 ; i<n ; i++) for(auto adjN : adj[i]) indegree[adjN]++;
  queue<int> q;
  for(int i=0 ; i<n ; i++) if(indegree[i] == 0) q.push(i);
  vector<int> order;
  while(!q.empty()){
    int node = q.front(); q.pop();
    order.push_back(node);
    for(auto adjN : adj[node]){
      if(--indegree[adjN] == 0) q.push(adjN);
    }
  }
  return order; // size < n if cycle
}

void dfsVisit(int node, const vector<vector<int>>& adj, vector<int>&
    state, vector<int>& order){
  state[node] = 1;
  for(auto adjN : adj[node]){
    // if(state[adjN] == 1) -> ciclo: back edge a ancestro en la pila de
    recursion
    if(state[adjN] == 0)
      dfsVisit(adjN, adj, state, order);
  }
  state[node] = 2;
  order.push_back(node);
}

vector<int> topoDFS(int n, const vector<vector<int>>& adj){
  vector<int> order;
  vector<int> state(n, 0); // 0=white, 1=gray, 2=black
  for(int i=0 ; i<n ; i++)
    if(state[i] == 0)
      dfsVisit(i, adj, state, order);
  reverse(order.begin(), order.end());
  return order;
}

```

```
int main(){
  vector<vector<int>> adj = {{1,2}, {3}, {3}, {}};
  auto order = topoKahn(4, adj);
  for(int node : order) cout << node << " ";  // 0 1 2 3
  cout << "\n";
  return 0;
}
```

5.2 DP en DAG (camino mas largo / corto con relajacion)

```
// TC: O(N + M)
// usa: Procesar en orden topologico relaja cada arista EXACTAMENTE 1 vez
     (Bellman-Ford lineal).
// Sirve para camino mas largo/corto con pesos arbitrarios (positivos o
    negativos) y conteo de caminos.
const long long INF = 1e18;

// Retorna {dist, parent}. dist[i] = -INF si i es inalcanzable desde s.
pair<vector<long long>, vector<int>> dagLongestPath(int n, const vector<
    vector<pair<int,long long>>>& adj, const vector<int>& topo, int s){
  vector<long long> dist(n, -INF);
  vector<int> parent(n, -1);
  dist[s] = 0; // o 1 si se cuentan ciudades/vertices

  for(auto node : topo){
    if(dist[node] == -INF) continue; // Inalcanzable desde s
    for(auto [adjN, adjW] : adj[node]){
      if(dist[node] + adjW > dist[adjN]){
        dist[adjN] = dist[node] + adjW;
        parent[adjN] = node; // Para reconstruir la ruta
      }
    }
  }
  return {dist, parent};
}

// Reconstruir camino de s a t (retorna vacio si t es inalcanzable)
vector<int> getPath(int s, int t, const vector<int>& parent){
  if(s != t && parent[t] == -1) return {};
  vector<int> path;
  for(int cur = t; cur != -1; cur = parent[cur]) path.push_back(cur);
  reverse(path.begin(), path.end());
  if(path.empty() || path[0] != s) return {};
  return path;
}

// Conteo de caminos s -> t (modulo MOD):
// vector<long long> ways(n, 0); ways[s] = 1;
// for(auto node : topo) if(ways[node]) for(auto [adjN, adjW] : adj[node
    ]) ways[adjN] = (ways[adjN] + ways[node]) % MOD;
```

5.6 Flujos

5.1 Ford-Fulkerson (DFS max flow)

```cpp
// TC: O(M * max_flow) con DFS
// usa: max flow clasico; busca caminos aumentantes con DFS
// diferencia con Edmonds-Karp: FF usa DFS (depende del flujo), EK usa BFS O(N * M^2)
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n, m;
int flow = 1e18;

bool dfs(int node, vector<int> &parent, vector<bool> &vis, vector<pair<int,int>> adj[]){
  vis[node] = true;
  if(node == n - 1) return true;
  for(auto [adjN, w] : adj[node]){
    if(!vis[adjN] && w > 0){
      parent[adjN] = node;
      bool tmp = dfs(adjN, parent, vis, adj);
      if(tmp){
        flow = min(flow, w);
        return true;
      }
    }
  }
  return false;
}

signed main(){
  ios_base::sync_with_stdio(0);
  cin.tie(0); cout.tie(0);

  cin >> n >> m;
  vector<pair<int,int>> adj[n];
  for(int i = 0; i < m; i++){
    int u, v, w; cin >> u >> v >> w; u--; v--;
    adj[u].emplace_back(v, w);
    adj[v].emplace_back(u, 0);
  }

  int mx = 0;
  while(true){
    vector<int> parent(n, -1);
    vector<bool> vis(n);
    flow = 1e18;
    dfs(0, parent, vis, adj);

    if(flow != 1e18) mx += flow;
    else break;

    for(int cur = n - 1; cur != 0; cur = parent[cur]){
      int par = -1;
      for(auto &[adjN, w] : adj[cur]){
        if(adjN == parent[cur]){
          w += flow;
          par = adjN;
          break;
        }
      }
      for(auto &[adjN, w] : adj[par]){
        if(adjN == cur && w >= flow){
          w -= flow;
          break;
        }
      }
    }
  }

  cout << mx << "\n";
}
```

5.2 Edmonds-Karp (max flow)

```cpp
// TC: O(N * M^2)
// usa: max flow simple; busca caminos aumentantes mas cortos con BFS
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n, m;
int flow = 1e18;

int bfs(int s, vector<int> &parent, vector<pair<int,int>> adj[]){
  vector<bool> vis(n);
  vis[s] = true;
  queue<pair<int,int>> q;
  q.push(make_pair(s, 1e18));
  while(!q.empty()){
    auto [node, mn] = q.front();
    q.pop();
    for(auto [adjN, w] : adj[node]){
      if(!vis[adjN] && w > 0){
        q.push(make_pair(adjN, min(mn, w)));
        vis[adjN] = true;
        parent[adjN] = node;
        if(adjN == n - 1){
          return min(mn, w);
        }
      }
    }
  }
  return 1e18;
}

signed main(){
  ios_base::sync_with_stdio(0);
  cin.tie(0); cout.tie(0);
```

```
  cin >> n >> m;
  vector<pair<int,int>> adj[n];
  for(int i = 0; i < m; i++){
    int u, v, w; cin >> u >> v >> w; u--; v--;
    adj[u].emplace_back(v, w);
    adj[v].emplace_back(u, 0);
  }

  int mx = 0;
  while(true){
    vector<int> parent(n, -1);
    flow = bfs(0, parent, adj);
    if(flow != 1e18) mx += flow;
    else break;

    for(int cur = n - 1; cur != 0; cur = parent[cur]){
      int par = -1;
      for(auto &[adjN, w] : adj[cur]){
        if(adjN == parent[cur]){
          w += flow;
          par = adjN;
          break;
        }
      }
      for(auto &[adjN, w] : adj[parent[cur]]){
        if(adjN == cur && w >= flow){
          w -= flow;
          break;
        }
      }
    }
  }

  cout << mx << "\n";
}
```

5.3 Dinic (max flow)

```
// TC: O(N^2 * M) general, O(sqrt(N) * M) para redes unitarias / matching
// usa: max flow eficiente con BFS (niveles) y DFS (blocking flow con ptr)
#include <bits/stdc++.h>
using namespace std;
#define int long long

struct Edge{
  int v;
  int cap;
  int rev;
};

int n, m;
vector<vector<Edge>> adj;

void bfs(int s, vector<int> &level, vector<vector<Edge>> &adj){
  level[s] = 1;
  queue<pair<int,int>> q;
  q.push(make_pair(s, 1));
  while(!q.empty()){
    auto [node, l] = q.front();
    q.pop();
    for(auto adjN : adj[node]){
      if(level[adjN.v] == 0 && adjN.cap > 0){
        level[adjN.v] = l + 1;
        q.push(make_pair(adjN.v, l + 1));
      }
    }
  }
}

int dfs(int node, int pushed, vector<int> &ptr, vector<int> &level,
    vector<vector<Edge>> &adj){
  if(node == n - 1 || pushed == 0){
    return pushed;
  }
  for(int &cid = ptr[node]; cid < adj[node].size(); cid++){
    Edge &adjN = adj[node][cid];
    if(level[adjN.v] == level[node] + 1 && adjN.cap > 0){
      int tmp = dfs(adjN.v, min(pushed, adjN.cap), ptr, level, adj);
      if(tmp > 0){
        adjN.cap -= tmp;
        adj[adjN.v][adjN.rev].cap += tmp;
        return tmp;
      }
    }
  }
  return 0;
}

signed main(){
  ios_base::sync_with_stdio(0);
  cin.tie(0); cout.tie(0);

  cin >> n >> m;
  adj.resize(n);
  for(int i = 0; i < m; i++){
    int u, v, w; cin >> u >> v >> w; u--; v--;
    adj[u].push_back(Edge{v, w, (int)adj[v].size()});
    adj[v].push_back(Edge{u, 0, (int)adj[u].size() - 1});
  }

  int mx = 0;
  int source = 0, sink = n - 1;

  while(true){
    vector<int> level(n);
    vector<int> ptr(n);
    bfs(source, level, adj);
    if(level[sink] == 0) break;

```

```
    while(int flow = dfs(source, 1e18, ptr, level, adj)){
      mx += flow;
    }
  }

  cout << mx << "\n";
}
```

5.4 MCMF con SPFA (Simple / Costos Negativos)

```
// TC: O(F * V * E) peor caso. En la practica suele ser rapido.
// CUANDO USAR:
// - Grafos pequenos o medianos (V <= 200, E <= 1000).
// - Grafos con costos iniciales negativos (sin ciclos neg).
// - Facil y rapido de tipear en contest (sin potenciales).
#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

struct MCMF_SPFA {
  struct Edge { int to, rev; long long cap, cost; };
  int n;
  vector<vector<Edge>> g;
  MCMF_SPFA(int n): n(n), g(n) {}

  void add_edge(int u, int v, long long cap, long long cost) {
    g[u].push_back({v, (int)g[v].size(), cap, cost});
    g[v].push_back({u, (int)g[u].size() - 1, 0, -cost});
  }

  // Retorna {max_flow, min_cost}
  pair<long long, long long> min_cost_flow(int s, int t) {
    long long flow = 0, cost = 0;
    vector<long long> dist(n);
    vector<int> pv(n), pe(n);
    vector<bool> inq(n);
    queue<int> q;

    while (true) {
      fill(dist.begin(), dist.end(), INF);
      fill(inq.begin(), inq.end(), false);
      dist[s] = 0;
      q.push(s);
      inq[s] = true;

      while (!q.empty()) {
        int u = q.front(); q.pop();
        inq[u] = false;
        for (int i = 0; i < (int)g[u].size(); i++) {
          auto &e = g[u][i];
          if (e.cap > 0 && dist[u] + e.cost < dist[e.to]) {
            dist[e.to] = dist[u] + e.cost;
            pv[e.to] = u;
            pe[e.to] = i;
            if (!inq[e.to]) {
              inq[e.to] = true;
              q.push(e.to);
            }
          }
        }
      }

      if (dist[t] == INF) break; // no hay mas caminos

      long long push = INF;
      for (int v = t; v != s; v = pv[v]) {
        push = min(push, g[pv[v]][pe[v]].cap);
      }
      for (int v = t; v != s; v = pv[v]) {
        auto &e = g[pv[v]][pe[v]];
        e.cap -= push;
        g[v][e.rev].cap += push;
        cost += push * e.cost;
      }
      flow += push;
    }
    return {flow, cost};
  }
};
```

5.5 MCMF con Potenciales de Johnson (Dijkstra / Rapido)

```
// TC: O(F * (V + E) log V). Con SPFA inicial: +O(V * E).
// CUANDO USAR:
// - Grafos grandes (V hasta 2000-5000, E hasta 20000+).
// - Flujo grande F o limites de tiempo estrictos (evita TLE).
// - Garantia contra grafos "SPFA-killer".
// NOTA: Si hay costos iniciales < 0, se corre SPFA una sola
// vez al inicio para fijar los potenciales h[].
#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

struct MCMF_Pot {
  struct Edge { int to, rev; long long cap, cost; };
  int n;
  vector<vector<Edge>> g;
  MCMF_Pot(int n): n(n), g(n) {}

  void add_edge(int u, int v, long long cap, long long cost) {
    g[u].push_back({v, (int)g[v].size(), cap, cost});
    g[v].push_back({u, (int)g[u].size() - 1, 0, -cost});
  }

  // Opcional: solo si hay aristas con costo inicial < 0
  void init_potentials_spfa(int s, vector<long long> &h) {
    fill(h.begin(), h.end(), INF);
    vector<bool> inq(n, false);
```

```
    queue<int> q;
    h[s] = 0; q.push(s); inq[s] = true;
    while (!q.empty()) {
      int u = q.front(); q.pop();
      inq[u] = false;
      for (auto &e : g[u]) {
        if (e.cap > 0 && h[u] != INF &&
            h[u] + e.cost < h[e.to]) {
          h[e.to] = h[u] + e.cost;
          if (!inq[e.to]) {
            inq[e.to] = true;
            q.push(e.to);
          }
        }
      }
    }
    for (int i = 0; i < n; i++) if (h[i] == INF) h[i] = 0;
  }

  // Retorna {max_flow, min_cost}
  pair<long long, long long> min_cost_flow(int s, int t) {
    long long flow = 0, cost = 0;
    vector<long long> h(n, 0); // potenciales
    // Descomentar si hay costos iniciales negativos:
    // init_potentials_spfa(s, h);

    vector<long long> dist(n);
    vector<int> pv(n), pe(n);
    using P = pair<long long, int>;

    while (true) {
      priority_queue<P, vector<P>, greater<P>> pq;
      fill(dist.begin(), dist.end(), INF);
      dist[s] = 0;
      pq.push({0, s});

      while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (int i = 0; i < (int)g[u].size(); i++) {
          auto &e = g[u][i];
          // Costo reducido no-negativo: cost + h[u] - h[e.to]
          long long red = e.cost + h[u] - h[e.to];
          if (e.cap > 0 && dist[u] + red < dist[e.to]) {
            dist[e.to] = dist[u] + red;
            pv[e.to] = u;
            pe[e.to] = i;
            pq.push({dist[e.to], e.to});
          }
        }
      }

      if (dist[t] == INF) break; // no hay mas caminos

      // Actualizar potenciales (clamping a dist[t])
      for (int i = 0; i < n; i++) {
        h[i] += min(dist[i], dist[t]);
      }

      long long push = INF;
      for (int v = t; v != s; v = pv[v]) {
        push = min(push, g[pv[v]][pe[v]].cap);
      }
      for (int v = t; v != s; v = pv[v]) {
        auto &e = g[pv[v]][pe[v]];
        e.cap -= push;
        g[v][e.rev].cap += push;
        cost += push * e.cost;
      }
      flow += push;
    }
    return {flow, cost};
  }
};
```

5.6 Algoritmo Hungaro (Assignment Problem)

```
// TC: O(N^2 * M), si N = M => O(N^3)
// usa: matching bipartito con pesos / problema de asignacion (min cost)
// 1-indexed: n filas (trabajadores), m columnas (tareas), requiere n <= m
// Para MAX cost: pasar -cost[i][j] y el costo maximo es -res.first
#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

// cost[1..n][1..m] es el costo de asignar trabajador i a tarea j
// Retorna {costo_minimo, asignacion}: ans[i] = tarea asignada al trabajador i
pair<long long, vector<int>> hungarian(int n, int m, const vector<vector<long long>>& cost){
  vector<long long> u(n + 1, 0), v(m + 1, 0);
  vector<int> p(m + 1, 0), way(m + 1, 0);

  for(int i = 1; i <= n; i++){
    p[0] = i;
    int j0 = 0;
    vector<long long> minv(m + 1, INF);
    vector<bool> used(m + 1, false);

    do {
      used[j0] = true;
      int i0 = p[j0], j1 = 0;
      long long delta = INF;

      for(int j = 1; j <= m; j++){
        if(!used[j]){
          long long cur = cost[i0][j] - u[i0] - v[j];
          if(cur < minv[j]){
            minv[j] = cur;
```

```
            way[j] = j0;
          }
          if(minv[j] < delta){
            delta = minv[j];
            j1 = j;
          }
        }
      }

      for(int j = 0; j <= m; j++){
        if(used[j]){
          u[p[j]] += delta;
          v[j] -= delta;
        } else {
          minv[j] -= delta;
        }
      }
      j0 = j1;
    } while(p[j0] != 0);

    do {
      int j1 = way[j0];
      p[j0] = p[j1];
      j0 = j1;
    } while(j0);
  }

  long long min_cost = -v[0];
  vector<int> ans(n + 1, 0);
  for(int j = 1; j <= m; j++){
    if(p[j] > 0) ans[p[j]] = j; // trabajador p[j] asignado a tarea j
  }
  return {min_cost, ans};
}

int main(){
  int n = 3, m = 3;
  // Matriz de costos 1-indexed de tamano (n+1) x (m+1)
  vector<vector<long long>> cost = {
    {0, 0, 0, 0},
    {0, 4, 1, 3},
    {0, 2, 0, 5},
    {0, 3, 2, 2}
  };

  auto [min_cost, match] = hungarian(n, m, cost);
  cout << "Costo minimo = " << min_cost << "\n"; // 5
  for(int i = 1; i <= n; i++){
    cout << "Trabajador " << i << " -> Tarea " << match[i] << "\n";
  }
  return 0;
}
```

5.7 Especiales

5.1 2-SAT (implication graph + SCC)

```
// TC: O(N + M)
// usa: formulas booleanas en CNF con clausulas de 2 literales
// Clausulas comunes (not x = x^1):
// - A or B   : adj[a^1].push_back(b),   adj[b^1].push_back(a)
// - A xor B  : (A v B) ^ (~A v ~B)  [solo 1 es true, A != B]
//              adj[a^1].push_back(b),   adj[b^1].push_back(a)
//              adj[a].push_back(b^1),   adj[b].push_back(a^1)
// - A <=> B  : (A => B) ^ (B => A)    [ambos iguales, A == B]
//              adj[a].push_back(b),     adj[b^1].push_back(a^1)
//              adj[b].push_back(a),     adj[a^1].push_back(b^1)
// - Forzar A : adj[a^1].push_back(a)  (~A => A obliga a que A sea true)
// - Forzar ~A: adj[a].push_back(a^1)  (A => ~A obliga a que A sea false)
#include <bits/stdc++.h>
using namespace std;

vector<bool> vis;
vector<int> scc;
stack<int> st;
int scc_number=-1;

int main(){
  int n,m; cin >> n >> m;
  vector<int> adj[2*m]; // doble de aristas, par true, impar false
  for(int i=0 ; i<m ; i++){
    char t1,t2; int v1,v2; cin >> t1 >> v1 >> t2 >> v2;
    v1--; v2--;
    int a=2*v1 + (t1=='-');
    int b=2*v2 + (t2=='-');
    adj[a^1].push_back(b);
    adj[b^1].push_back(a);
  }

  vis.resize(2*m);
  for(int i=0 ; i<2*m ; i++) if(!vis[i]) dfs1(i,adj);

  vector<int> adjT[2*m];
  for(int i=0 ; i<2*m ; i++){
    vis[i]=false;
    for(int adjN : adj[i]) adjT[adjN].push_back(i);
  }

  scc.resize(2*m);
  while(!st.empty()){
    int node = st.top();
    st.pop();
    if(!vis[node]){
      scc_number++;
      dfs2(node,adjT);
    }
  }

  vector<char> ans(m);
```

```
  for(int i=0 ; i<m ; i++){
    if(scc[2*i] == scc[2*i+1]){
      // contradiccion
      return 0;
    }
    if(scc[2*i] > scc[2*i+1]) ans[i]='+';
    else ans[i]='-';
  }

  for(auto &x : ans) cout<< x <<" ";
  cout<<"\n";

  return 0;
}
```

5.2 Euler tour / Hierholzer (camino y circuito)

```
// TC: O(N + M) - Algoritmo de Hierholzer
// usa: existencia y construccion de camino / circuito euleriano
//      (visita cada ARISTA exactamente una vez; admite multigrafos)
#include <bits/stdc++.h>
using namespace std;

// ---------- Condiciones necesarias y suficientes ----------
//   NO DIRIGIDO, conexo:
//     - circuito euleriano: todos los vertices tienen grado par
//     - camino  euleriano: exactamente 0 o 2 vertices con grado impar
//                          (si son 2, el camino va de uno al otro)
//   DIRIGIDO, debilmente conexo sobre el subgrafo con aristas:
//     - circuito euleriano: indeg(v) == outdeg(v) para todo v
//     - camino  euleriano: existe s con outdeg(s) -indeg(s) = 1,
//                          existe t con indeg(t) - outdeg(t) = 1,
//                          y el resto cumple indeg == outdeg

int main(){
  int n,m; cin >> n >> m;
  vector<pair<int,int>> adj[n]; // no dirigido necesitamos par para
    guardar id
  // vector<int> indeg(n) , outdeg(n);
  vector<int> deg(n);
  for(int i=0 ; i<m ; i++){
    int u,v; cin >> u >> v; u--; v--;
    // si es dirigido solo usar uno y ya no seria necesario guardar id
    adj[u].emplace_back(v,i);
    adj[v].emplace_back(u,i);
    deg[u]++; deg[v]++; // si es dirigido llenar indeg y outdeg
  }
  int start=0 , end=-1;
  int odd=0;
  for(int i=0 ; i<n ; i++){
    // contar odd, asignar start, end , revisar condiciones de existencia
    if(deg[i]&1){
      start=i; odd++;
    }
  }
  if(odd!=0 && odd!=2){
    // no hay camino ni circuito
    return 0;
  }

  stack<int> st;
  // solo si es no dirigido
  vector<bool> vis_edges(m);
  vector<int> path; // o circuit
  st.push(start);
  while(!st.empty()){
    int node = st.top();
    // si es no dirigido lo necesitamos
    while(!adj[node].empty() && vis_edges[adj[node].back().second]){
      adj[node].pop_back();
    }
    if(!adj[node].empty()){
      auto [adjN,id] = adj[node].back();
      adj[node].pop_back();
      st.push(adjN);
      vis_edges[id]=true;
    }else{
      path.push_back(node);
      st.pop();
    }
  }
  reverse(path.begin() , path.end());

  // check si va por todos los edges
  if(path.size() != m+1){
    // no pasa por todo
  }

  for(auto &x : path) cout<< x <<" ";
  cout<<"\n";
}
```

5.3 Hamiltonian path / cycle (bitmask DP + backtracking)

Teoria de Grafos Hamiltonianos (NP-completo): Recorre cada **vertice** exactamente una vez.

- **Ciclo Hamiltoniano**: Empieza y termina en el **mismo** vertice ($A \to \cdots \to A$), visitando cada vertice intermedio exactamente una vez.
- **Camino Hamiltoniano**: Empieza en A y termina en B con $B \neq A$ ($A \to \cdots \to B$), visitando cada vertice del grafo exactamente una vez.
- **Teorema de Dirac** (suficiente): Si $n \geq 3$ y para todo v, $\deg(v) \geq \lceil n/2 \rceil$, existe ciclo hamiltoniano.
- **Teorema de Ore** (suficiente): Si para todo par no adyacente (u, v), $\deg(u) + \deg(v) \geq n$, existe ciclo hamiltoniano.

- **Grafos Torneo**: En cualquier torneo (grafo dirigido completo), **siempre** existe al menos un camino hamiltoniano.
- **Enfoque CP** ($n \leq 20$): Bitmask DP en $O(2^n \cdot n^2)$ o $O(2^n \cdot n)$. Estado dp[mask][v] = 1 (camino sobre subconjunto mask terminando en v).
- **Cuadriculas / Caballo**: Heuristica de Warnsdorff (greedy hacia el vecino con menor grado de salida).

```cpp
// TC: O(N^2 * 2^N)  (bitmask DP), O(N!) peor caso (backtracking)
// usa: existencia y construccion de camino / ciclo hamiltoniano;
//      (visita cada VERTICE exactamente una vez; NP-completo)
//      TSP (min costo de ciclo hamiltoniano) si las aristas tienen peso
#include <bits/stdc++.h>
using namespace std;

// -- Teorema de Dirac (condicion SUFICIENTE) --
//   Si G es simple, |V| = n >= 3, y todo vertice tiene grado >= n/2,
//   entonces G tiene un ciclo hamiltoniano. (No es necesario.)
//   Ore: si para todo par no adyacente (u,v) se cumple
//        deg(u) + deg(v) >= n, entonces G es hamiltoniano.

bool satisfiesDirac(int n, const vector<vector<int>>& adj){
  if(n < 3) return false;
  for(int v=0;v<n;v++) if((int)adj[v].size() < (n+1)/2) return false;
  return true;
}
bool satisfiesOre(int n, const vector<vector<int>>& adj){
  if(n < 3) return false;
  // construye conjunto de adyacencia
  vector<vector<char>> A(n, vector<char>(n, 0));
  for(int u=0;u<n;u++) for(int v:adj[u]) A[u][v] = 1;
  for(int u=0;u<n;u++)
    for(int v=u+1;v<n;v++)
      if(!A[u][v] && ((int)adj[u].size() + (int)adj[v].size() < n))
        return false;
  return true;
}

// -- Bitmask DP: Hamiltonian path starting at 'src' --
// dp[mask][v] = true si hay un camino que visita exactamente 'mask' y
    termina en v
// memoria: O(N * 2^N) bool; tiempo: O(N^2 * 2^N)
bool hasHamiltonianPathFrom(int n, const vector<vector<int>>& adj, int
    src){
  vector<vector<char>> dp(1<<n, vector<char>(n, 0));
  dp[1<<src][src] = 1;
  for(int mask=1; mask<(1<<n); mask++){
    for(int node=0 ; node<n ; node++) if(dp[mask][node]){
      for(auto adjN : adj[node]){
        if(!(mask & (1<<adjN))){
          dp[mask | (1<<adjN)][adjN] = 1;
        }
      }
    }
  }
  for(int mask=1; mask<(1<<n); mask++){
    if(__builtin_popcount(mask) != n) continue;
    for(int node=0 ; node<n ; node++) if(dp[mask][node]) return true;
  }
  return false;
}

// -- Bitmask DP: Hamiltonian cycle --
// dp[mask][node] = true si hay un camino de 'src' que visita 'mask' y
    termina en node
bool hasHamiltonianCycle(int n, const vector<vector<int>>& adj, int src =
     0){
  if(n < 3) return false;
  vector<vector<char>> dp(1<<n, vector<char>(n, 0));
  dp[1<<src][src] = 1;
  for(int mask=1; mask<(1<<n); mask++){
    for(int node=0 ; node<n ; node++) if(dp[mask][node]){
      for(auto adjN : adj[node]){
        if(!(mask & (1<<adjN))){
          dp[mask | (1<<adjN)][adjN] = 1;
        }
      }
    }
  }
  int full = (1<<n) - 1;
  for(int node=0 ; node<n ; node++){
    if(dp[full][node]){
      // cierra el ciclo si node es adyacente a src
      for(auto adjN : adj[node]) if(adjN == src) return true;
    }
  }
  return false;
}

// -- Reconstruccion de un camino hamiltoniano (bitmask DP) --
// devuelve la secuencia de vertices o vacio si no existe
vector<int> findHamiltonianPath(int n, const vector<vector<int>>& adj,
    int src){
  vector<vector<int>> par(1<<n, vector<int>(n, -1));
  vector<vector<char>> dp(1<<n, vector<char>(n, 0));
  dp[1<<src][src] = 1;
  for(int mask=1; mask<(1<<n); mask++){
    for(int node=0 ; node<n ; node++) if(dp[mask][node]){
      for(auto adjN : adj[node]){
        if(!(mask & (1<<adjN))){
          dp[mask | (1<<adjN)][adjN] = 1;
          par[mask | (1<<adjN)][adjN] = node;
        }
      }
    }
  }
  int endV = -1, full = -1;
  for(int mask=1; mask<(1<<n); mask++){
    if(__builtin_popcount(mask) != n) continue;
    for(int v=0; v<n; v++) if(dp[mask][v]){ endV = v; full = mask; break;
     }
```

```
    if(endV != -1) break;
  }
  if(endV == -1) return {};
  vector<int> path;
  int cur = endV, m = full;
  while(cur != -1){
    path.push_back(cur);
    int p = par[m][cur];
    m &= ~(1<<cur);
    cur = p;
  }
  reverse(path.begin(), path.end());
  return path;
}

// -- TSP minimo: ciclo hamiltoniano de costo minimo --
// dp[mask][v] = min costo de camino src -> ... -> v visitando 'mask'
// respuesta = min_v dp[full][v] + w(v, src) si el cierre es valido
const long long LINF = (1LL<<60);
long long tspMinCycle(int n, const vector<vector<long long>>& w, int src
    = 0){
  if(n == 1) return 0;
  vector<vector<long long>> dp(1<<n, vector<long long>(n, LINF));
  dp[1<<src][src] = 0;
  for(int mask=1; mask<(1<<n); mask++){
    for(int v=0; v<n; v++){
      if(dp[mask][v] == LINF) continue;
      for(int u=0; u<n; u++){
        if(!(mask & (1<<u)) && w[v][u] < LINF){
          dp[mask | (1<<u)][u] = min(dp[mask | (1<<u)][u],
          dp[mask][v] + w[v][u]);
        }
      }
    }
  }
  int full = (1<<n) - 1;
  long long ans = LINF;
  for(int v=0; v<n; v++){
    if(dp[full][v] < LINF && w[v][src] < LINF){
      ans = min(ans, dp[full][v] + w[v][src]);
    }
  }
  return ans;
}

int main(){
  // ejemplo: ciclo de 4 vertices (0-1-2-3-0) -> claramente hamiltoniano
  int n = 4;
  vector<vector<int>> adj(n);
  auto add = [&](int u, int v){
    adj[u].push_back(v); adj[v].push_back(u);
  };
  add(0,1); add(1,2); add(2,3); add(3,0);

  cout << "Dirac: " << satisfiesDirac(n, adj) << " (n/2 = 2, todos grado
    2 -> true)\n";
  cout << "Ore:   " << satisfiesOre(n, adj) << "\n";
  cout << "cycle? " << hasHamiltonianCycle(n, adj) << "\n";
  cout << "path?  " << hasHamiltonianPathFrom(n, adj, 0) << "\n";

  auto path = findHamiltonianPath(n, adj, 0);
  for(int v : path) cout << v << " ";
  cout << "\n";

  // TSP: mismo grafo con pesos -> ciclo minimo
  vector<vector<long long>> w(n, vector<long long>(n, LINF));
  for(int i=0;i<n;i++) w[i][i] = 0;
  w[0][1] = w[1][0] = 10;
  w[1][2] = w[2][1] = 20;
  w[2][3] = w[3][2] = 30;
  w[3][0] = w[0][3] = 40;
  cout << "TSP min = " << tspMinCycle(n, w) << "\n"; // 10+20+30+40 = 100
  return 0;
}
```

6 DP

6.1 LIS O(n log n)

```
// TC: O(N log N)
// usa: longitud de la LIS estrictamente creciente
#include <bits/stdc++.h>
using namespace std;

int lisLength(const vector<int>& a){
  vector<int> dp;
  for(int x : a){
    auto it = lower_bound(dp.begin(), dp.end(), x);
    if(it == dp.end()) dp.push_back(x);
    else *it = x;
  }
  return (int)dp.size();
}

// TC: O(N log N),   memoria O(N)
// usa: reconstruye una LIS (la que el algoritmo encontro); una valida
    cualquiera
//   no decreciente (LNDS): cambiar lower_bound -> upper_bound
//   reconstruye via `parent[i]` (forward) + `dpIdx` (posicion final de
    la cola)
vector<int> lisSequence(const vector<int>& a){
  int n = (int)a.size();
  if(n == 0) return {};
  vector<int> dp;              // dp[k] = menor cola posible de una IS de
     long k+1
  vector<int> dpIdx;           // dpIdx[k] = indice en a de esa cola
  vector<int> parent(n, -1);   // parent[i] = indice previo en la LIS que
     termina en i
  for(int i = 0; i < n; i++){
```

```
    int pos = (int)(lower_bound(dp.begin(), dp.end(), a[i]) - dp.begin())
    ;
    if(pos == (int)dp.size()){
      dp.push_back(a[i]);
      dpIdx.push_back(i);
    }else{
      dp[pos] = a[i];
      dpIdx[pos] = i;
    }
    if(pos > 0) parent[i] = dpIdx[pos - 1];
  }
  // reconstruir desde la cola de la LIS
  vector<int> res;
  for(int i = dpIdx.back(); i != -1; i = parent[i])
    res.push_back(a[i]);
  reverse(res.begin(), res.end());
  return res;
}

// TC: O(N log N),   memoria O(N) (solo un vector extra: n enteros)
// usa: variante compacta. `insertedAt[i]` = pos en dp donde cayo a[i].
//   Reconstruye recorriendo de atras hacia adelante y agarrando
//   el ultimo i con insertedAt[i] == need (need baja de L-1 a 0).
vector<int> lisSequenceCompact(const vector<int>& a){
  int n = (int)a.size();
  if(n == 0) return {};
  vector<int> dp;
  vector<int> insertedAt(n);
  for(int i = 0; i < n; i++){
    int pos = (int)(lower_bound(dp.begin(), dp.end(), a[i]) - dp.begin())
    ;
    if(pos == (int)dp.size()) dp.push_back(a[i]);
    else dp[pos] = a[i];
    insertedAt[i] = pos;
  }
  vector<int> res;
  for(int i = n - 1, need = (int)dp.size() - 1; i >= 0 && need >= 0; i--)
    if(insertedAt[i] == need)
      res.push_back(a[i]), need--;
  reverse(res.begin(), res.end());
  return res;
}

// Diferencia entre las 2:
//   lisSequence       usa 2 vectores extra (parent, dpIdx); reconstruye
    forward.
//   lisSequenceCompact usa 1 vector extra (insertedAt); reconstruye
    backward.
// Compacta = menos memoria y menos codigo; basta para reconstruir una
    LIS.
// Si necesitas contar/todas las LIS, o generalizar el "anterior", usa la
     primera.

int main(){
  vector<int> a = {1, 3, 2, 4, 5, 2};
  cout << lisLength(a) << "\n";            // 4
  auto seq = lisSequence(a);
  for(int x : seq) cout << x << " ";       // p.ej. 1 2 4 5  (alguna LIS
    valida)
  cout << "\n";
  auto seq2 = lisSequenceCompact(a);
  for(int x : seq2) cout << x << " ";      // p.ej. 1 2 4 5  (alguna LIS
    valida)
  cout << "\n";
  return 0;
}
```

6.2 LCS / Edit distance

```
// TC: O(N * M)
// usa: longest common subsequence y distancia de edicion (Levenshtein)
#include <bits/stdc++.h>
using namespace std;

int lcs(const string& a, const string& b){
  int n = a.size(), m = b.size();
  vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
  for(int i=1;i<=n;i++){
    for(int j=1;j<=m;j++){
      if(a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
      else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
    }
  }
  return dp[n][m];
}

int editDistance(const string& a, const string& b){
  int n = a.size(), m = b.size();
  vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
  for(int i=0;i<=n;i++) dp[i][0] = i;
  for(int j=0;j<=m;j++) dp[0][j] = j;
  for(int i=1;i<=n;i++){
    for(int j=1;j<=m;j++){
      if(a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1];
      else dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
    }
  }
  return dp[n][m];
}

int main(){
  cout << lcs("abcde", "ace") << "\n";       // 3
  cout << editDistance("kitten", "sitting") << "\n";  // 3
  return 0;
}
```

6.3 Bitmask DP over subsets

```cpp
// TC: O(N * 2^N)
// usa: DP donde el estado es un subconjunto de N elementos
#include <bits/stdc++.h>
using namespace std;

// iterate subsets in increasing order
// for(int mask=0; mask<(1<<N); mask++){ ... }

// iterate submasks of mask
// for(int sub=mask; sub; sub=(sub-1)&mask){ ... }

// iterate over supersets of mask (within universe U)
// for(int sup=mask; sup<U; sup=(sup+1)|mask){ ... }

int main(){
  // iterate submasks of a mask
  int mask = 0b1011;
  for(int sub = mask; sub; sub = (sub-1) & mask){
    // process sub
  }
  return 0;
}
```

6.4 SOS DP

```cpp
// TC: O(N * 2^N),   N = cantidad de bits (usar N <= 20)
//
// Las 4 variantes combinan 2 ejes:
//   - Forward vs Backward: forward usa +=  (zeta / suma)
//                          backward usa -= (mobius / inversa)
//   - Sufijo 1 vs 2:      1 opera sobre subsets   (mask ^ (1<<i))
//                         2 opera sobre supersets (mask | (1<<i))
// Backward n es la inversa de Forward n.

#include <bits/stdc++.h>
using namespace std;

int bitsNeeded(int sz){
    int n = 0;
    while((1 << n) < sz) n++;
    return n;
}

// dp[mask] = sum_{sub pertenece mask} f[sub]
// Caso clasico: para cada mask, sumar f sobre todas sus sub-masks.
vector<long long> sosForward1(const vector<long long>& f){
    int n = bitsNeeded((int)f.size());
    vector<long long> dp = f;
    dp.resize(1 << n);
    for(int i = 0; i < n; i++)
        for(int mask = 0; mask < (1 << n); mask++)
            if(mask & (1 << i))
                dp[mask] += dp[mask ^ (1 << i)];
    return dp;
}

// dp[mask] = sum_{sup contiene mask} f[sup]
// Para cada mask, sumar f sobre las masks que la contienen.
vector<long long> sosForward2(const vector<long long>& f){
    int n = bitsNeeded((int)f.size());
    vector<long long> dp = f;
    dp.resize(1 << n);
    for(int i = 0; i < n; i++)
        for(int mask = 0; mask < (1 << n); mask++)
            if(!(mask & (1 << i)))
                dp[mask] += dp[mask | (1 << i)];
    return dp;
}

// Inversa de sosForward1 (mobius en el lattice de subsets)
vector<long long> sosBackward1(const vector<long long>& g){
    int n = bitsNeeded((int)g.size());
    vector<long long> dp = g;
    dp.resize(1 << n);
    for(int i = 0; i < n; i++)
        for(int mask = 0; mask < (1 << n); mask++)
            if(mask & (1 << i))
                dp[mask] -= dp[mask ^ (1 << i)];
    return dp;
}

// Inversa de sosForward2 (mobius en el lattice de supersets)
vector<long long> sosBackward2(const vector<long long>& g){
    int n = bitsNeeded((int)g.size());
    vector<long long> dp = g;
    dp.resize(1 << n);
    for(int i = 0; i < n; i++)
        for(int mask = 0; mask < (1 << n); mask++)
            if(!(mask & (1 << i)))
                dp[mask] -= dp[mask | (1 << i)];
    return dp;
}

int main(){
    vector<long long> f = {1, 2, 3, 4};  // n = 2

    auto f1 = sosForward1(f);   // {1, 3, 4, 10}
    auto f2 = sosForward2(f);   // {10, 6, 7, 4}
    auto b1 = sosBackward1(f1); // {1, 2, 3, 4}  (recupera f)
    auto b2 = sosBackward2(f2); // {1, 2, 3, 4}  (recupera f)
    return 0;
}
```

Guía rápida.

- **sosForward1**: Zeta sobre subconjuntos. Acumula hacia arriba: dp[mask] = $\sum_{S \subseteq \text{mask}} f[S]$.
- **sosForward2**: Zeta sobre superconjuntos. Acumula hacia abajo: dp[mask] = $\sum_{\text{mask} \subseteq S} f[S]$.

- **sosBackward1 / sosBackward2**: Inversión de Möbius (deshacen el forward respectivo cambiando `+=` por `-=`).

Cómo usarlo y aplicaciones comunes:

1. **Consultas de bits sobre un arreglo (ej. CSES Bit Problem):** Sea $f[\text{val}]$ la frecuencia de cada número en el arreglo de N elementos:
   - $x \mid y = x \iff y \subseteq x$ (contar y que sean submáscaras de x):
     Se resuelve directamente con sosForward1$(f)[x]$.
   - $x \,\&\, y = x \iff x \subseteq y$ (contar y que sean supermáscaras de x):
     Se resuelve directamente con sosForward2$(f)[x]$.
   - $x \,\&\, y \neq 0$ (contar y que comparten al menos un bit con x):
     Se calcula por complemento: $N - \text{sosForward1}(f)[((1 \ll n) - 1) \oplus x]$.
2. **Convoluciones Bitwise (OR y AND) tipo FFT en $O(N \cdot 2^N)$:**
   - **Convolución OR** ($C[k] = \sum_{i|j=k} A[i] \cdot B[j]$):
     1. $A' = \text{sosForward1}(A)$, $B' = \text{sosForward1}(B)$
     2. Multiplicar punto a punto: $C'[k] = A'[k] \cdot B'[k]$
     3. Recuperar con $C = \text{sosBackward1}(C')$
   - **Convolución AND** ($C[k] = \sum_{i \,\&\, j=k} A[i] \cdot B[j]$):
     1. $A' = \text{sosForward2}(A)$, $B' = \text{sosForward2}(B)$
     2. Multiplicar punto a punto: $C'[k] = A'[k] \cdot B'[k]$
     3. Recuperar con $C = \text{sosBackward2}(C')$
3. **Principio de Inclusión-Exclusión automático:** Al contar combinaciones donde es fácil calcular "al menos *mask*" ($mask \subseteq S$), aplicar `sosBackward2` recupera cuántos cumplen "exactamente *mask*" en $O(N \cdot 2^N)$, realizando automáticamente toda la inclusión-exclusión con signos alternados $(-1)^{|S \setminus mask|}$.
4. **Problema clásico: CF 449D (Jzzhu and Numbers) — AND de subsecuencias $= 0$:**
   - *Enunciado*: Dado un arreglo de N enteros ($N, a_i \leq 10^6 < 2^{20}$), contar cuántas subsecuencias no vacías tienen un AND bitwise igual a 0: $a_{i_1} \,\&\, a_{i_2} \,\&\, \ldots \,\&\, a_{i_k} = 0 \pmod{10^9+7}$.
   - *Idea*: Para que el AND de una subsecuencia tenga todos los bits de una máscara S encendidos ($S \subseteq$ AND), **cada** elemento elegido debe cumplir $S \subseteq a_j$.
   - *Paso 1 (SOS DP)*: Calculamos $cnt[S] = \sum_{S \subseteq x} \text{freq}[x]$ con `sosForward2`.
   - *Paso 2 (Subsecuencias)*: Hay $2^{cnt[S]} - 1$ subsecuencias cuyo AND contiene a S.
   - *Paso 3 (Inclusión-Exclusión)*: Para que el AND tenga **exactamente 0** bits encendidos:

$$\text{ans} = \sum_{S=0}^{2^{20}-1} (-1)^{|S|} \cdot 2^{cnt[S]} \pmod{10^9+7}$$

   donde $|S| = \text{popcount}(S)$ (los -1 de $2^{cnt[S]} - 1$ se cancelan pues $\sum (-1)^{|S|} = 0$). Complejidad: $O(N + 20 \cdot 2^{20})$.

6.5 Digit DP template

```cpp
// TC: O(D * 10 * state_size) where D is number of digits
// usa: contar numeros en [0, N] que cumplen una propiedad sobre sus digitos
#include <bits/stdc++.h>
using namespace std;

// skeleton only; adapt state, transition, and base case to your problem
string digitStr;
long long memo[20][2][200]; // dimensions depend on problem (pos, tight, state)

long long updateState(long long state, int d){
  // problem-specific; return new state after appending digit d
  return state;
}

long long digitDFS(int pos, bool tight, long long state){
  if(pos == (int)digitStr.size()) return 1; // adapt per problem (e.g., state matches)
  if(!tight && memo[pos][0][state] != -1) return memo[pos][0][state];
  int limit = tight ? digitStr[pos] - '0' : 9;
  long long res = 0;
  for(int d=0; d<=limit; d++){
    long long ns = updateState(state, d);
    res += digitDFS(pos + 1, tight && (d == limit), ns);
  }
  if(!tight) memo[pos][0][state] = res;
  return res;
}

long long digitDP(long long N){
  digitStr = to_string(N);
  memset(memo, -1, sizeof(memo));
  return digitDFS(0, true, 0);
}

int main(){
  // count numbers in [0, 999] whose digit sum equals 10
  digitStr = to_string(999);
  memset(memo, -1, sizeof(memo));
  cout << digitDFS(0, true, 0) << "\n";  // (skeleton; fill updateState)
  return 0;
}
```

Versión iterativa (forward). Tabla `dp[pos][smaller][state]` llenada de `pos=0` a `pos=D`. `smaller=0` ⇒ tight (prox dígito ≤ `N[pos]`); `smaller=1` ⇒ libre (`0..9`).

```cpp
// Cuenta cuantos x en [0, N) (= [0, N-1]) cumplen una propiedad sobre sus digitos.
// Para rango [a, b] INCLUSIVO:    digitDP(b+1) - digitDP(a).
// TC: O(D * 10 * (STATE_MAX + 1)).   Memoria: O(D * 2 * (STATE_MAX + 1)).

// Ejemplo de abajo: contar x sin dos digitos adyacentes iguales.
//   state = "ultimo digito colocado" (o NOT_STARTED al principio).
long long digitDP(long long N){
  // 1) digitos de N, mas significativo primero.
  vector<int> dig;
  if(N == 0){
    dig = {0};
  }else{
    long long t = N;
    while(t > 0){ dig.push_back((int)(t % 10)); t /= 10; }
    reverse(dig.begin(), dig.end());
  }
  int D = (int)dig.size();

  // 2) cota + sentinel. Para "no adyacentes iguales" alcanza 10.
  // adaptar, ahorita esta del 0 al 9 y el 10 de flag
  const int STATE_MAX   = 10;
  const int NOT_STARTED = STATE_MAX;

  // 3) dp[pos][smaller][state], state en [0, STATE_MAX].
  vector<vector<vector<long long>>> dp(D + 1,
    vector<vector<long long>>(2, vector<long long>(STATE_MAX + 1, 0)));
  dp[0][0][NOT_STARTED] = 1;

  // 4) DP.
  for(int pos = 0; pos < D; pos++){
    for(int smaller = 0; smaller <= 1; smaller++){
      for(int digit = 0; digit <= STATE_MAX; digit++){
        long long ways = dp[pos][smaller][digit];
        // poda
        if(ways == 0) continue;
        for(int nxt = 0; nxt <= (smaller ? 9 : dig[pos]); nxt++){
          // en este caso no puede haber 2 digitos consecutivos iguales
          // se puede modificar para se salten transiciones no validas
          if(nxt == digit) continue;

          int new_digit = nxt;
          if(digit == NOT_STARTED && nxt == 0){
            new_digit = NOT_STARTED;
          }
          dp[pos + 1][smaller || (nxt < dig[pos])][new_digit] += ways;
        }
      }
    }
  }

  // 5) Finales validos. dp[D][1][*] = "< N"; dp[D][0][*] = N exacto (no se suma).
  //    Para "no adyacentes": cualquier final vale (incluye NOT_STARTED -> el numero 0).
  long long ans = 0;
  for(int digit = 0; digit <= STATE_MAX; digit++)
    ans += dp[D][1][digit];   // considera menores a N, [0,N)
  return ans;
}

int main(){
  // Ejemplo: contar x en [0, N) sin dos digitos adyacentes iguales.
  cout << digitDP(11)  << "\n";   // -> 11 (0..10 todos validos)
  cout << digitDP(12)  << "\n";   // -> 11 (sin el 11)
  cout << digitDP(100) << "\n";   // -> 91
  return 0;
}
```

6.6 Tree DP (Reroot aditivo)

```cpp
// TC: O(N)
// usa: para cada nodo, calcular alguna propiedad asumiendo que ese nodo es la raiz
#include <bits/stdc++.h>
using namespace std;

struct TreeDP{
  int n;
  vector<vector<int>> adj;
  vector<long long> dp, up, ans;
  vector<int> sz;
  TreeDP(int n): n(n){
    adj.assign(n, {});
    dp.assign(n, 0);
    up.assign(n, 0);
    ans.assign(n, 0);
    sz.assign(n, 0);
  }
  void addEdge(int u, int v){
    adj[u].push_back(v);
    adj[v].push_back(u);
  }
  // 1st pass: compute dp[v] as the answer for subtree of v (rooted at 0)
  void dfs1(int v, int p){
    sz[v] = 1;
    for(int u : adj[v]){
      if(u == p) continue;
      dfs1(u, v);
      sz[v] += sz[u];
      dp[v] += dp[u] + sz[u]; // example - sum of distances from u
    }
  }
  // 2nd pass: reroot - propagate contributions
  void dfs2(int v, int p){
    ans[v] = dp[v] + up[v];
    for(int u : adj[v]){
      if(u == p) continue;
```

```
      // when rerooting at u, u's contribution comes out, v's grows
      up[u] = up[v] + dp[v] - (dp[u] + sz[u]) + (n - sz[u]);
      dfs2(u, v);
    }
  }
  vector<long long> solve(int root = 0){
    dfs1(root, -1);
    dfs2(root, -1);
    return ans;
  }
};
```

6.7 Tree DP (Reroot prefijos y sufijos)

```
// TC: O(N)
// Usa pref/suf cuando la operacion NO es invertible
// (ej. mod compuesto, division por 0, max, gcd)
const int MOD = 1e9 + 7;
vector<vector<int>> adj;
vector<long long> dp, up, ans;

void dfs(int node, int par){
  dp[node] = 1;
  for(auto adjN : adj[node]) if(adjN != par){
    dfs(adjN, node);
    dp[node] = dp[node] * ((dp[adjN] + 1) % MOD) % MOD;
  }
}

void dfs2(int node, int par){
  vector<int> children;
  for(auto adjN : adj[node]) if(adjN != par){
    children.push_back(adjN);
  }
  int sz = children.size();
  vector<long long> pref(sz + 1, 1), suf(sz + 1, 1);
  for(int i = 0; i < sz; i++){
    pref[i + 1] = pref[i] * ((dp[children[i]] + 1) % MOD) % MOD;
  }
  for(int i = sz - 1; i >= 0; i--){
    suf[i] = suf[i + 1] * ((dp[children[i]] + 1) % MOD) % MOD;
  }
  for(int i = 0; i < sz; i++){
    int child = children[i];
    long long sib = pref[i] * suf[i + 1] % MOD;
    up[child] = sib * ((up[node] + 1) % MOD) % MOD;
    ans[child] = dp[child] * ((up[child] + 1) % MOD) % MOD;
    dfs2(child, node);
  }
}

void solve(int root = 0){
  dfs(root, -1);
  up[root] = 0;
  ans[root] = dp[root];
  dfs2(root, -1);
}
```

7 Geometria

7.1 Puntos y segmentos

7.1 Conceptos y formulas de geometria 2D

1. Vectores, Dot Product y Cross Product. Un punto o vector en 2D es $P = (x, y)$. La resta $\vec{u} = B - A = (x_B - x_A, y_B - y_A)$ representa el vector que va de A a B.

- **Magnitud**: $|\vec{u}| = \sqrt{x^2 + y^2}$, con $|\vec{u}|^2 = x^2 + y^2$. En CP, **siempre que sea posible compara distancias al cuadrado** para trabajar con enteros y evitar perdida de precision por punto flotante.
- **Producto Punto (Dot Product)**:

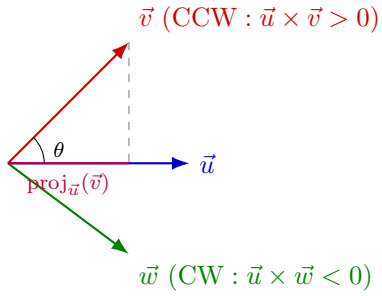
$$\vec{u} \cdot \vec{v} = x_1 x_2 + y_1 y_2 = |\vec{u}||\vec{v}| \cos\theta$$

  - $\vec{u} \cdot \vec{v} > 0 \iff \theta < 90°$ (angulo agudo).
  - $\vec{u} \cdot \vec{v} = 0 \iff \theta = 90°$ (**perpendiculares / ortogonales**).
  - $\vec{u} \cdot \vec{v} < 0 \iff \theta > 90°$ (angulo obtuso).
  - Proyeccion escalar de $\vec{u}$ sobre $\vec{v}$: $\frac{\vec{u} \cdot \vec{v}}{|\vec{v}|}$. Proyeccion vectorial: $\text{proj}_{\vec{v}}(\vec{u}) = \frac{\vec{u} \cdot \vec{v}}{|\vec{v}|^2} \vec{v}$.

- **Producto Cruz 2D (Cross Product / Pseudocruz)**:

$$\vec{u} \times \vec{v} = x_1 y_2 - y_1 x_2 = |\vec{u}||\vec{v}| \sin\theta$$

  - $|\vec{u} \times \vec{v}|$ = **Area del paralelogramo** formado por $\vec{u}$ y $\vec{v}$.
  - Area con signo del triangulo $\triangle ABC$: $\frac{1}{2}((B - A) \times (C - A))$.
  - $\vec{u} \times \vec{v} > 0 \iff \vec{v}$ esta a la **izquierda** de $\vec{u}$ (giro antihorario / CCW).
  - $\vec{u} \times \vec{v} < 0 \iff \vec{v}$ esta a la **derecha** de $\vec{u}$ (giro horario / CW).
  - $\vec{u} \times \vec{v} = 0 \iff \vec{u}$ y $\vec{v}$ son **colineales / paralelos**.



2. Criterios de Paralelismo y Perpendicularidad. Dados dos vectores $\vec{u} = (x_1, y_1)$ y $\vec{v} = (x_2, y_2)$:

- **Paralelos** ($\vec{u} \parallel \vec{v}$):
$$\vec{u} \times \vec{v} = x_1 y_2 - y_1 x_2 = 0$$
Misma direccion y sentido si $\vec{u} \cdot \vec{v} > 0$; sentidos opuestos si $\vec{u} \cdot \vec{v} < 0$.
- **Perpendiculares** ($\vec{u} \perp \vec{v}$):
$$\vec{u} \cdot \vec{v} = x_1 x_2 + y_1 y_2 = 0$$
- **Vector perpendicular rotado**: Rotar $\vec{u} = (x, y)$ en $90°$ antihorario da $\vec{u}^{\perp} = (-y, x)$. Rotar $-90°$ (horario) da $(y, -x)$.

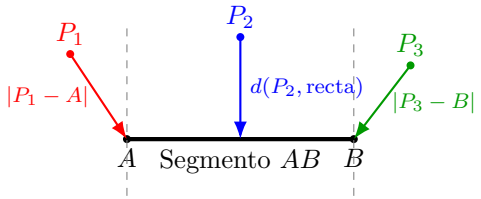
3. Orientacion de 3 puntos (Prueba CCW). Dados los puntos A, B, C, evaluar:

$$\text{orient}(A, B, C) = (B - A) \times (C - A) = (x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A)$$

- > 0: C queda a la izquierda de la recta dirigida $A \to B$ (vuelta a la izquierda / CCW).
- < 0: C queda a la derecha de la recta dirigida $A \to B$ (vuelta a la derecha / CW).
- $= 0$: A, B, C son colineales.

4. Lineas y Distancias.

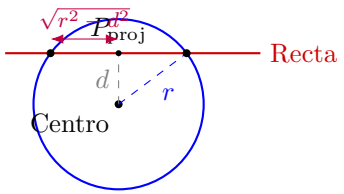
- **Ecuacion general de la recta**: $Ax + By + C = 0$. Dados dos puntos P y Q: $A = y_Q - y_P$, $B = x_P - x_Q$, $C = -(Ax_P + By_P)$.
- **Distancia punto a recta**: Distancia del punto P a la recta que pasa por A y B:
$$d(P, \text{recta}(AB)) = \frac{|(B - A) \times (P - A)|}{|B - A|} = \frac{|Ax_P + By_P + C|}{\sqrt{A^2 + B^2}}$$
- **Distancia punto a segmento** AB:
  - Si $(B - A) \cdot (P - A) \leq 0$, el punto mas cercano del segmento es $A \implies d = |P - A|$.
  - Si $(A - B) \cdot (P - B) \leq 0$, el punto mas cercano del segmento es $B \implies d = |P - B|$.
  - En otro caso, la proyeccion cae dentro del segmento $\implies d = d(P, \text{recta}(AB))$.



- **Interseccion de dos rectas**: $A_1x + B_1y + C_1 = 0$ y $A_2x + B_2y + C_2 = 0$. Determinante $D = A_1B_2 - A_2B_1$.
  - Si $D = 0$: las rectas son paralelas ($C_1A_2 \neq C_2A_1$) o coincidentes ($C_1A_2 = C_2A_1$).
  - Si $D \neq 0$: interseccion unica en:
$$x = \frac{B_1C_2 - B_2C_1}{D}, \quad y = \frac{C_1A_2 - C_2A_1}{D}$$

5. Circunferencias (Circulos).

- **Ecuacion**: $(x - h)^2 + (y - k)^2 = r^2$, con centro (h, k) y radio r.
- **Interseccion recta-circunferencia**: Calcular $d = d(\text{centro}, \text{recta})$.
  - $d > r$: 0 intersecciones (exterior).
  - $d = r$: 1 interseccion (tangente en la proyeccion P_{proj} del centro sobre la recta).
  - $d < r$: 2 intersecciones. Los puntos estan a distancia $h = \sqrt{r^2 - d^2}$ a ambos lados de P_{proj} a lo largo del vector unitario director de la recta.
- **Interseccion entre dos circunferencias**: Centros C_1, C_2 con radios r_1, r_2 y distancia $d = |C_1 - C_2|$:
  - $d > r_1 + r_2$ o $d < |r_1 - r_2|$: no se intersectan (disjuntos externos o uno contenido en otro).
  - $d = 0$ y $r_1 = r_2$: infinitos puntos (circunferencias identicas).
  - $d = r_1 + r_2$ o $d = |r_1 - r_2|$: 1 punto (tangencia exterior o interior).
  - En otro caso: 2 puntos de corte simetricos respecto a la linea de centros.
- **Circuncentro de un triangulo** $\triangle ABC$: Punto de interseccion de las tres mediatrices. Radio de la circunferencia circunscrita:
$$R = \frac{a \cdot b \cdot c}{4 \cdot \text{Area}} = \frac{|B - A|\,|C - B|\,|A - C|}{2\,|(B - A) \times (C - A)|}$$



6. Tips criticos de precision y tipos en CP.

- **Evitar flotantes en orientacion**: Al hacer cross product de coordenadas $\leq 10^9$, el resultado puede llegar a $2 \cdot 10^{18}$, cabiendo en `long long`. Si se multiplica con otras distancias, usar `__int128_t`.
- **Comparacion con EPS**: Para flotantes (`double`), usar constante $\varepsilon = 10^{-9}$:
  - Igualdad: `abs(a - b) < EPS`
  - Menor estricto: `a < b - EPS`
  - Menor o igual: `a < b + EPS`
- **Angulo de un vector**: Usar `atan2(y, x)`, que devuelve el angulo exacto en $[-\pi, \pi]$ considerando signos de ambos cuadrantes sin division por cero.

7.2 Point + cross + dot

```cpp
struct Point{
  int x,y;
  void read(){cin >> x >> y;}
  Point operator -(const Point &b) const {
    return Point{x-b.x , y-b.y};
  }

  // Respecto al origen
  int operator *(const Point &b) const {
    return x*b.y - y*b.x;
  }

  // Respecto a this
  int cross(const Point &b, const Point &c) const {
    return (b-*this) * (c-*this);
  }
  int dot(const Point &b, const Point &c) const {
    return (b.x-x)*(c.x-x) + (b.y-y)*(c.y-y);
  }
  bool operator <(const Point &b) const {
    return make_pair(x,y)<make_pair(b.x,b.y);
  }
};
```

7.3 Interseccion de segmentos

```cpp
// TC: O(1)
// usa: checa si los segmentos [a,b] y [c,d] se intersectan (incluye
    colineales solapados)
// nota: usa pruebas de orientacion (cross products) + bounding boxes; no
     retorna punto de cruce
// linear segments intersection
bool intersect(Point a, Point b, Point c, Point d){
  // check parallel or collinear
  if( (b-a) * (d-c) == 0 ){
    // check if parallel
    if(a.cross(b,c) != 0){
      return false;
    }
    // check if collinear, bounding box
    for(int i=0 ; i<2 ; i++){
      if(max(a.x,b.x)<min(c.x,d.x) || max(a.y,b.y)<min(c.y,d.y)){
        return false;
      }
      swap(a,c);
      swap(b,d);
    }
    return true;
  }
  for(int i=0 ; i<2 ; i++){
    int x=a.cross(b,c) , y=a.cross(b,d);
    if( (x<0 && y<0) || (x>0 && y>0)){
      return false;
    }
    swap(a,c);
    swap(b,d);
  }
  return true;
}
```

7.4 Contains colineal

```cpp
// TC: O(1)
// usa: checa si el punto c esta en el segmento [a,b] (colineal y dentro
    del bounding box)
bool contains(Point a, Point b, Point c){
  if( a.cross(b,c) == 0){
    return (min(a.x,b.x)<=c.x) && (c.x<=max(a.x,b.x))
    && (min(a.y,b.y)<=c.y) && (c.y<=max(a.y,b.y));
  }
  return false;
}
```

7.5 Distance point-segment / point-line

```cpp
// TC: O(1)
// usa: distancia minima de un punto a un segmento o a la recta que lo
    contiene
#include <bits/stdc++.h>
using namespace std;

// returns squared distance from p to line (a, b)
long long dist2PointLine(const Point& p, const Point& a, const Point& b){
  Point ab = b - a;
  long long area2 = abs((long long)ab * (p - a));
```

```
  long long len2 = 1LL * ab.x * ab.x + 1LL * ab.y * ab.y;
  if(len2 == 0) return 1LL * (p.x - a.x) * (p.x - a.x) + 1LL * (p.y - a.y
    ) * (p.y - a.y);
  return (area2 * area2) / len2;
}

long long dist2PointSegment(const Point& p, const Point& a, const Point&
    b){
  Point ap = p - a;
  Point ab = b - a;
  long long dot1 = 1LL * ab.x * ap.x + 1LL * ab.y * ap.y;
  if(dot1 <= 0) return 1LL * ap.x * ap.x + 1LL * ap.y * ap.y;
  Point bp = p - b;
  long long dot2 = -1LL * ab.x * bp.x - 1LL * ab.y * bp.y;
  if(dot2 <= 0) return 1LL * bp.x * bp.x + 1LL * bp.y * bp.y;
  long long len2 = 1LL * ab.x * ab.x + 1LL * ab.y * ab.y;
  long long area2 = abs((long long)ab * ap);
  return (area2 * area2) / len2;
}

int main(){
  Point a{0,0}, b{4,0}, p{2,3};
  cout << "dist to segment: " << dist2PointSegment(p,a,b) << "\n";    // 9
  cout << "dist to line:    " << dist2PointLine(p,a,b) << "\n";       // 9
  return 0;
}
```

7.6 Projection of point onto line

```
// TC: O(1)
// usa: la proyeccion ortogonal de p sobre la recta que pasa por a y b (
    devuelve double)
#include <bits/stdc++.h>
using namespace std;

pair<double,double> projectPointLine(const Point& p, const Point& a,
    const Point& b){
  double ax = a.x, ay = a.y, bx = b.x, by = b.y, px = p.x, py = p.y;
  double dx = bx - ax, dy = by - ay;
  double len2 = dx*dx + dy*dy;
  if(len2 == 0) return {ax, ay};
  double t = (dx * (px - ax) + dy * (py - ay)) / len2;
  return {ax + t * dx, ay + t * dy};
}

int main(){
  Point a{0,0}, b{4,0}, p{2,3};
  auto [px, py] = projectPointLine(p, a, b);
  cout << "projection = (" << px << ", " << py << ")\n";  // (2, 0)
  return 0;
}
```

7.2 Poligonos

7.1 Convex Hull (monotono)

```
// TC: O(N log N) por sort; el barrido es amortizado O(N) (cada pop_back
    se hace a lo sumo N veces por iter)
// usa: devuelve los puntos del casco convexo en orden CCW (sin
    colineales si cambias <=0 a <0)
vector<Point> convexHull(vector<Point> points) {
  sort(points.begin(), points.end());

  vector<Point> hull;
  for(int iter=0; iter<2; iter++) {
    int S = hull.size();
    for(const Point &c : points) {
      while((int)hull.size() >= S+2) {
        Point a = hull.end()[-2], b = hull.end()[-1];
        // <=0 incluye colineales
        if(a.cross(b, c) <= 0) break;
        hull.pop_back();
      }
      hull.push_back(c);
    }
    hull.pop_back(); // evita duplicar el ultimo punto
    reverse(points.begin(), points.end());
  }

  return hull;
}
```

7.2 Polygon Area

```
int polygonArea(const vector<Point> &poly) {
  int area = 0;
  int n = poly.size();
  for(int i = 0; i < n; i++) {
    area += poly[i] * poly[(i+1) % n];  // Producto cruz
  }
  return abs(area); // este es el doble del area
}
```

7.3 Point in polygon (ray casting)

```
// TC: O(N)
// usa: comprobar si un punto esta dentro de un poligono simple
#include <bits/stdc++.h>
using namespace std;

bool pointInPolygon(const Point& p, const vector<Point>& poly){
  int n = (int)poly.size();
  bool inside = false;
  for(int i=0,j=n-1 ; i<n ; j=i++){
```

```
    if((poly[i].y > p.y) != (poly[j].y > p.y)){
      long long num = 1LL * (p.y - poly[j].y) * (poly[i].x - poly[j].x);
      long long den = poly[i].y - poly[j].y;
      long long x_intersect = poly[j].x + num / den;
      if(p.x < x_intersect) inside = !inside;
    }
  }
  return inside;
}

int main(){
  vector<Point> square = {{0,0},{4,0},{4,4},{0,4}};
  cout << pointInPolygon({2,2}, square) << "\n";  // 1 (inside)
  cout << pointInPolygon({5,5}, square) << "\n";  // 0 (outside)
  return 0;
}
```

7.4 Minkowski sum (convex polygons)

```
// TC: O(N + M)
// usa: suma de Minkowski; util para chequear colision entre dos poligonos convexos
#include <bits/stdc++.h>
using namespace std;

vector<Point> minkowskiSum(const vector<Point>& a, const vector<Point>& b){
  // assumes both are in counter-clockwise order, no repeated last point
  int n = (int)a.size(), m = (int)b.size();
  auto edgesOf = [](const vector<Point>& p){
    vector<Point> e;
    int k = (int)p.size();
    for(int i=0;i<k;i++) e.push_back(p[(i+1)%k] - p[i]);
    return e;
  };
  vector<Point> ea = edgesOf(a), eb = edgesOf(b);
  int i = 0, j = 0;
  vector<Point> result;
  Point cur{a[0].x + b[0].x, a[0].y + b[0].y};
  result.push_back(cur);
  while(i < n && j < m){
    Point pa = ea[i], pb = eb[j];
    long long cr = (long long)pa * pb;
    if(cr >= 0){ cur.x += pa.x; cur.y += pa.y; i++; }
    if(cr <= 0){ cur.x += pb.x; cur.y += pb.y; j++; }
    result.push_back(cur);
  }
  while(i < n){ cur.x += ea[i].x; cur.y += ea[i].y; result.push_back(cur); i++; }
  while(j < m){ cur.x += eb[j].x; cur.y += eb[j].y; result.push_back(cur); j++; }
  return result;
}

int main(){
  vector<Point> A = {{0,0},{2,0},{2,2},{0,2}};  // unit-ish square
  vector<Point> B = {{1,1},{3,1},{3,3},{1,3}};
  auto C = minkowskiSum(A, B);
  cout << "Minkowski sum has " << C.size() << " vertices\n";
  return 0;
}
```

7.3 Herramientas

7.1 Li Chao Tree (long long)

```
// TC: O(log X) per query/insert
// usa: mantener un lower envelope de lineas y consultar en cualquier x
#include <bits/stdc++.h>
using namespace std;

const long long LINF = (1LL<<60);

struct Line{
  long long m, b; // y = m*x + b
  Line(long long m=0, long long b=LINF): m(m), b(b){}
  long long get(long long x) const { return m*x + b; }
};

struct LiChao{
  struct Node{
    Line line;
    Node *l, *r;
    Node(const Line& line): line(line), l(nullptr), r(nullptr){}
  };
  long long X_LO, X_HI;
  Node* root;
  LiChao(long long X_LO, long long X_HI): X_LO(X_LO), X_HI(X_HI){
    root = nullptr;
  }
  void addLine(Line newLine){ addLine(root, X_LO, X_HI, newLine); }
  void addLine(Node*& node, long long lo, long long hi, Line newLine){
    if(!node){
      node = new Node(newLine);
      return;
    }
    long long mid = (lo + hi) >> 1;
    bool left = newLine.get(lo) < node->line.get(lo);
    bool middle = newLine.get(mid) < node->line.get(mid);
    if(middle){
      swap(node->line, newLine);
    }
    if(hi - lo == 0) return;
    if(left != middle){
      addLine(node->l, lo, mid, newLine);
    }else{
      addLine(node->r, mid + 1, hi, newLine);
    }
  }
```

```
  long long query(long long x){ return query(root, X_LO, X_HI, x); }
  long long query(Node* node, long long lo, long long hi, long long x){
    if(!node) return LINF;
    long long res = node->line.get(x);
    if(lo == hi) return res;
    long long mid = (lo + hi) >> 1;
    if(x <= mid) return min(res, query(node->l, lo, mid, x));
    else return min(res, query(node->r, mid + 1, hi, x));
  }
};

int main(){
  LiChao lc(-(long long)1e9, (long long)1e9);
  lc.addLine(Line(1, 0));  // y = x
  lc.addLine(Line(-1, 5)); // y = -x + 5
  cout << "min at x=2: " << lc.query(2) << "\n";   // -2+5 = 3
  cout << "min at x=10: " << lc.query(10) << "\n"; // 10 vs -5 -> -5
  return 0;
}
```

7.2 Sweep line template

```
// TC: depends on problem; usually O((N + E) log N)
// usa: problemas donde se barre una linea/punto y se mantiene una EDD sobre eventos
#include <bits/stdc++.h>
using namespace std;

struct SweepEvent{
  long long x;
  int type; // 0 = add, 1 = query, 2 = remove (problem-specific)
  int id;
  bool operator<(const SweepEvent& o) const { return x < o.x; }
};

// usage:
//   sort events by x
//   maintain a data structure (set, multiset, segment tree) keyed by y
//   process events in order, calling add/query/remove handlers

int main(){
  // skeleton: fill events with x, type, id and process in order
  vector<SweepEvent> evts = {{1, 0, 0}, {3, 1, 1}, {5, 2, 0}};
  sort(evts.begin(), evts.end());
  for(auto& e : evts) cout << "(" << e.x << "," << e.type << "," << e.id << ") ";
  cout << "\n";
  return 0;
}
```

7.3 Closest pair of points (sweepline)

```
// TC: O(N log N)
// usa: encontrar el par de puntos mas cercanos en un plano
// idea: ordenar por x, mantener una "caja" (set ordenado por y) con puntos
//       cuya x esta a <= d del actual; evict y query acotados por d
// nota: hypot(dx,dy) = sqrt(dx^2+dy^2) sin overflow intermedio (mas seguro que sqrt(dx*dx+dy*dy))
#include <bits/stdc++.h>
using namespace std;

double closestPair(vector<Point>& pts){
  sort(pts.begin(), pts.end(), [](const Point& a, const Point& b){
    return a.x < b.x || (a.x == b.x && a.y < b.y);
  });
  double d = 1e18;
  set<pair<long long,long long>> box;
  int j = 0;
  for(int i = 0; i < (int)pts.size(); i++){
    long long xi = pts[i].x, yi = pts[i].y;
    while(j < i && (double)(xi - pts[j].x) > d){
      box.erase({pts[j].y, pts[j].x});
      j++;
    }
    long long ylow  = (long long)floor((double)yi - d);
    long long yhigh = (long long)ceil((double)yi  + d);
    for(auto it = box.lower_bound({ylow, LLONG_MIN});
      it != box.end() && it->first <= yhigh; ++it){
      double dist = hypot((double)(xi - it->second), (double)(yi - it->first));
      if(dist < d) d = dist;
    }
    box.insert({yi, xi});
  }
  return d;
}

int main(){
  vector<Point> pts = {{0,0}, {5,5}, {1,1}, {3,3}, {0,2}};
  cout << "closest distance = " << closestPair(pts) << "\n";  // ~1.41
  return 0;
}
```

7.4 Rotating calipers (polygon diameter)

```
// TC: O(N) (assumes convex polygon sorted counter-clockwise)
// usa: encontrar el par de vertices mas distantes (diametro) en un poligono convexo
#include <bits/stdc++.h>
using namespace std;

long long rotatingCalipers(const vector<Point>& p){
  int n = (int)p.size();
  if(n == 1) return 0;
  if(n == 2){
```

```
    long long dx = p[0].x - p[1].x, dy = p[0].y - p[1].y;
    return dx*dx + dy*dy;
  }
  long long best = 0;
  int k = 1;
  for(int i=0;i<n;i++){
    int ni = (i + 1) % n;
    while(true){
      int nk = (k + 1) % n;
      long long cur = abs((long long)(p[ni] - p[i]) * (p[nk] - p[k]));
      long long nxt = abs((long long)(p[ni] - p[i]) * (p[(nk+1)%n] - p[(k
    +1)%n]));
      if(nxt > cur) k = nk;
      else break;
    }
    long long d1 = 1LL * (p[i].x - p[k].x) * (p[i].x - p[k].x) + 1LL * (p
    [i].y - p[k].y) * (p[i].y - p[k].y);
    long long d2 = 1LL * (p[ni].x - p[k].x) * (p[ni].x - p[k].x) + 1LL *
    (p[ni].y - p[k].y) * (p[ni].y - p[k].y);
    best = max(best, max(d1, d2));
  }
  return best;
}

int main(){
  vector<Point> tri = {{0,0}, {10,0}, {5,8}};  // triangle CCW
  cout << "diameter^2 = " << rotatingCalipers(tri) << "\n";
  return 0;
}
```